

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

30 个 Kernel · 9 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 7 月 20 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试刷题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (~30 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce / Dot Product (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add / Histogram (基础 + float4 向量化 + atomic)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
17 //   Phase 5 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
18 //   Phase 6 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
19 //   Phase 7 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
20 //   Phase 8 — FlashAttention-2split_q (FA-2, 含 online softmax + P@V 寄存器复用)
21 //
22 // =====
23 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
24 // =====
25
26 // ---- GPU 架构速查 ----
27 //
28 // SM (Streaming Multiprocessor) 内部结构:
29 //   - Warp Scheduler x4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
30 //   - Register File: 每 SM 65536 × 32-bit (4 bytes) = 256KB
31 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
32 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
33 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
34 //
35 // Memory Hierarchy 带宽数量级 (H100 参考):
36 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
37 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
38 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
39 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
40 //
41 // 关键瓶颈判断:
42 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
43 //   Compute-bound: AI 足够大, 受限于计算吞吐
44 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
45 //
46 // Occupancy 公式:
47 //   occupancy = active_warps / max_warps_per_SM
48 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory
49 //   → blocks/SM; block 大小 → blocks/SM
50 //
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数

```

```

63 // - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 // - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 // - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 // - Shared memory 有 32 banks × 4 bytes
71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMa)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMa m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 1D~5D 寻址, 低寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPs
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPs/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
102 // AI ≈ 33M / 67M ≈ 0.5 FLOPs/Byte → severely memory-bound
103 //
104 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPs/Byte → memory-bound
105
106 // =====
107 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
108 // =====
109 // 面试要点:
110 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
111 //   memory
112 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
113 //   注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
114 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
115
116 #include <algorithm>
117 #include <cstring>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123 #include <math.h>
124 #include <cublas_v2.h>
125 #include <cuda.h>

```

```

126 #include <cuda/barrier>
127 #include <cuda/ptx>
128
129 #if defined(NOTES_V2_ENABLE_CUDNN)
130 #pragma nv_diag_suppress 128
131 #include <cuda/cudnn_frontend.h>
132 #pragma nv_diag_default 128
133 namespace fe = cudnn_frontend;
134 #endif
135
136 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS) && CUDART_VERSION < 13000
137 #error "NOTES_V2_ENABLE_TMA_MMA_WS requires CUDA Toolkit 13.0 or newer"
138 #endif
139
140 #define INT4(value) (reinterpret_cast<int4 *>(&(value)))[0])
141 #define FLOAT4(value) (reinterpret_cast<float4 *>(&(value)))[0])
142 #define HALF2(value) (reinterpret_cast<half2 *>(&(value)))[0])
143
144 static constexpr int kWarpSize = 32;
145
146 // =====
147 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
148 // =====
149
150 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
151 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
152 // 复杂度 O(logN), N=32 时只需 5 步
153 // 模板参数: T=数据类型, kWarpWidth=segment width (默认 32)
154 // kWarpWidth 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
155 // 当 kWarpWidth < 32 (如 FA 中 kWarpWidth=4) 时, 只有同 segment 的 lane 参与通信
156 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
157 //
158 // 初始: 每个 lane 持有自己的值 v0..v7
159 // lane: 0 1 2 3 4 5 6 7
160 // val: v0 v1 v2 v3 v4 v5 v6 v7
161 //
162 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
163 //
164 //      ┌──────────┐
165 // 对: (0,4) (1,5) (2,6) (3,7)
166 //
167 // lane: 0 1 2 3 4 5 6 7
168 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
169 //
170 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
171 //
172 //      ┌───┐ ┌───┐ ┌───┐ ┌───┐
173 // 对: (0,2) (1,3) (4,6) (5,7)
174 //      ─── 前4个一组 ───      ─── 后4个一组 ───
175 //
176 // lane: 0 1 2 3 4 5 6 7 (每 lane 持有前一轮2个值的和再加本轮配对)
177 // val: Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7} Σ{0,2,4,6} Σ{1,3,5,7}
178 //      = v0+v2+v4+v6 ... (逐步归约)
179 //
180 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
181 //
182 //      ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐ ┌─┐
183 // 对: (0,1) (2,3) (4,5) (6,7)
184 //
185 // lane: 0 1 2 3 4 5 6 7
186 // val: Σall Σall Σall Σall Σall Σall Σall Σall ← 所有 lane 拥有全归约结果!
187 //
188 // mask=0: 循环终止, 归约完成。
189 //
190 // 关键性质:
191 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i

```

```

189 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
190 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
191 // - 0(log2 N) 步完成, 无需 shared memory, 纯寄存器操作
192 // - __shfl_xor_sync 第四个参数 kWarpWidth 限制 segment width:
193 // 当 kWarpWidth=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
194 template <const int kWarpWidth = kWarpSize, typename T = float>
195 __device__ __forceinline__ T warp_reduce_sum(T val) {
196 #pragma unroll
197     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
198         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth);
199     }
200     return val;
201 }
202
203 // Warp Reduce Max — generic
204 template <const int kWarpWidth = kWarpSize, typename T = float>
205 __device__ __forceinline__ T warp_reduce_max(T val) {
206 #pragma unroll
207     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
208         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpWidth));
209     }
210     return val;
211 }
212
213 // =====
214 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
215 // =====
216
217 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
218 // 两级归约流程:
219 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
220 // 2. warp leader (lane=0) 写入 shared memory
221 // 3. syncthreads 后, lane 0~kNumWarps-1 读取 shared memory
222 // 4. 所有 warp 内再做一次 warp_reduce<kNumWarps> → 得到最终结果
223 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<kNumWarps 知道结果)
224 template <const int kNumThreads = 256>
225 __device__ float block_reduce_sum(float val) {
226     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
227     int warp = threadIdx.x / kWarpSize;
228     int lane = threadIdx.x % kWarpSize;
229     __shared__ float shared[kNumWarps];
230
231     float value = warp_reduce_sum<kWarpSize>(val);
232     if (lane == 0)
233         shared[warp] = value;
234     __syncthreads();
235     value = (lane < kNumWarps) ? shared[lane] : 0.0f;
236     value = warp_reduce_sum<kNumWarps>(value);
237     // 关键: broadcast 结果到所有线程, 后续用 result
238     // 做除法等操作时所有线程都能拿到
239     value = __shfl_sync(0xffffffff, value, 0, 32);
240     return value;
241 }
242
243 // Block Reduce Max — FP32 (增强版, 带 broadcast)
244 template <const int kNumThreads = 256>
245 __device__ float block_reduce_max(float val) {
246     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
247     int warp = threadIdx.x / kWarpSize;
248     int lane = threadIdx.x % kWarpSize;
249     __shared__ float shared[kNumWarps];
250
251     float value = warp_reduce_max<kWarpSize>(val);

```

```

252     if (lane == 0)
253         shared[warp] = value;
254     __syncthreads();
255     value = (lane < kNumWarps) ? shared[lane] : -FLT_MAX;
256     value = warp_reduce_max<kNumWarps>(value);
257     value = __shfl_sync(0xffffffff, value, 0, 32);
258     return value;
259 }
260
261 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
262 // 多 block 各自做 warp->smem->warp0 reduce, 然后 atomicAdd 到全局 y
263 // 跨 block 求和的常见模式, 适合 N 较大时使用;
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (256, 1, 1)
266 // source: LeetCUDA/kernels/reduce/block_all_reduce.cu
267 template <const int kNumThreads = 256>
268 __global__ void block_reduce_all(float *a, float *y, int N) {
269     int tid = threadIdx.x;
270     int idx = blockIdx.x * kNumThreads + tid;
271     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
272     __shared__ float shared[kNumWarps];
273
274     float val = (idx < N) ? a[idx] : 0.0f;
275     int warp = tid / kWarpSize;
276     int lane = tid % kWarpSize;
277
278     val = warp_reduce_sum<kWarpSize>(val);
279     if (lane == 0)
280         shared[warp] = val;
281     __syncthreads();
282
283     val = (lane < kNumWarps) ? shared[lane] : 0.0f;
284     if (warp == 0)
285         val = warp_reduce_sum<kNumWarps>(val);
286     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加
287         atomicAdd(y, val);
288 }
289
290 // ---- Dot Product: y = sum(a[i] * b[i]) ----
291 // 核心模式: elementwise 乘法 -> block reduce -> atomicAdd 全局累加
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (256, 1, 1)
294 // source: LeetCUDA/kernels/dot-product/dot_product.cu
295 template <const int kNumThreads = 256>
296 __global__ void dot(float *a, float *b, float *y, int N) {
297     int tid = threadIdx.x;
298     int idx = blockIdx.x * kNumThreads + tid;
299     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
300     __shared__ float shared[kNumWarps];
301
302     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
303     int warp = tid / kWarpSize;
304     int lane = tid % kWarpSize;
305
306     prod = warp_reduce_sum<kWarpSize>(prod);
307     if (lane == 0)
308         shared[warp] = prod;
309     __syncthreads();
310
311     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
312     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
313         prod = warp_reduce_sum<kNumWarps>(prod);
314     if (tid == 0) // tid == 0, not lane 0. 只有 block 内的一个线程负责写回全局结果, 避免重复累加

```

```

315     atomicAdd(y, prod);
316 }
317
318 // Dot Product + float4
319 // Grid: ((N + 255) / 256, 1, 1), 每个block处理256元素, float4向量化后每个线程处理4元素
320 // Block: (64, 1, 1), 256/4=64
321 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
322 // source: LeetCUDA/kernels/dot-product/dot_product.cu
323 template <const int kNumThreads = 256 / 4>
324 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
325     int tid = threadIdx.x;
326     int idx = (blockIdx.x * kNumThreads + tid) * 4;
327     constexpr int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
328     __shared__ float shared[kNumWarps];
329
330     float4 reg_a = FLOAT4(a[idx]);
331     float4 reg_b = FLOAT4(b[idx]);
332     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
333                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
334                       : 0.0f;
335     int warp = tid / kWarpSize;
336     int lane = tid % kWarpSize;
337
338     prod = warp_reduce_sum<kWarpSize>(prod);
339     if (lane == 0)
340         shared[warp] = prod;
341     __syncthreads();
342
343     prod = (lane < kNumWarps) ? shared[lane] : 0.0f;
344     if (warp == 0)
345         prod = warp_reduce_sum<kNumWarps>(prod);
346     if (tid == 0)
347         atomicAdd(y, prod);
348 }
349
350
351 // =====
352 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
353 // =====
354 // 面试要点:
355 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
356 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
357 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
358
359 // ---- ReLU: y = max(0, x) ----
360 // Grid: ((N + 255) / 256, 1, 1)
361 // Block: (256, 1, 1)
362 // source: LeetCUDA/kernels/relu/relu.cu
363 __global__ void relu(float *x, float *y, int N) {
364     int idx = blockIdx.x * blockDim.x + threadIdx.x;
365     if (idx < N)
366         y[idx] = fmaxf(0.0f, x[idx]);
367 }
368
369 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
370 // block(64)x4(float4)=256 元素/block, 与基础版吞吐相同
371 // Grid: ((N + 255) / 256, 1, 1)
372 // Block: (64, 1, 1)
373 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
374 // source: LeetCUDA/kernels/relu/relu.cu
375 __global__ void relu_vec4(float *x, float *y, int N) {
376     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
377     if (idx < N) {

```

```

378     float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
379     float4 reg_y;
380     reg_y.x = fmaxf(0.0f, reg_x.x);
381     reg_y.y = fmaxf(0.0f, reg_x.y);
382     reg_y.z = fmaxf(0.0f, reg_x.z);
383     reg_y.w = fmaxf(0.0f, reg_x.w);
384     FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
385 }
386 }
387
388 // ---- Elementwise Add: c = a + b ----
389 // Grid: ((N + 255) / 256, 1, 1)
390 // Block: (256, 1, 1)
391 // source: LeetCUDA/kernels/elementwise/elementwise.cu
392 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
393     int idx = blockIdx.x * blockDim.x + threadIdx.x;
394     if (idx < N)
395         c[idx] = a[idx] + b[idx];
396 }
397
398 // Elementwise Add + float4 向量化
399 // Grid: ((N + 255) / 256, 1, 1)
400 // Block: (64, 1, 1), block(64)*4(float4)=256 元素/block
401 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
402 // source: LeetCUDA/kernels/elementwise/elementwise.cu
403 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
404     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
405     if ((idx + 3) < N) {
406         float4 reg_a = FLOAT4(a[idx]);
407         float4 reg_b = FLOAT4(b[idx]);
408         float4 reg_c;
409         reg_c.x = reg_a.x + reg_b.x;
410         reg_c.y = reg_a.y + reg_b.y;
411         reg_c.z = reg_a.z + reg_b.z;
412         reg_c.w = reg_a.w + reg_b.w;
413         FLOAT4(c[idx]) = reg_c;
414     } else if (idx < N) {
415         for (int i = 0; (idx + i) < N; i++) {
416             c[idx + i] = a[idx + i] + b[idx + i];
417         }
418     }
419 }
420
421 // ---- Histogram: y[a[i]]++ ----
422 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
423 // Grid: ((N + 255) / 256, 1, 1)
424 // Block: (256, 1, 1)
425 // source: LeetCUDA/kernels/histogram/histogram.cu
426 __global__ void histogram(int *a, int *y, int N) {
427     int idx = blockIdx.x * blockDim.x + threadIdx.x;
428     if (idx < N)
429         atomicAdd(&(y[a[idx]]), 1);
430 }
431
432 // ---- Merge Attn States: 合并 Split-KV Attention 的部分结果 ----
433 // 实现 FlashInfer 论文 (arXiv 2501.01005) Section 2.2 的 attention 合并逻辑。
434 //
435 // 面试要点:
436 //   - 应用场景: Split-KV attention 将序列沿 K 维分段计算 attention,
437 //     每段产出部分结果 (O_i, LSE_i), 本 kernel 将两段按指数权重加权合并。
438 //   - 数学公式 (5 步推导):
439 //     Step 1 — max 归一化 (数值稳定, 与 safe softmax 同理):
440 //         L_max = max(LSE_1, LSE_2)

```



```

441 // Step 2 — 还原指数权重 (un-normalized) :
442 //      w_1 = exp(LSE_1 - L_max), w_2 = exp(LSE_2 - L_max)
443 // Step 3 — 归一化为混合比例:
444 //      alpha = w_1 / (w_1 + w_2), beta = w_2 / (w_1 + w_2)
445 //      满足 alpha + beta = 1
446 // Step 4 — 逐元素加权合并:
447 //      O = alpha * O_1 + beta * O_2
448 // - LSE 布局 [num_heads, num_tokens] (与 FlashAttention-2惯例一致,
449 //   lse[head_idx][token_idx])
450 // - Output 布局 [num_tokens, num_heads, head_size] (token 维在最外,
451 //   展平为 [T0H0, T0H1, ..., T1H0, ...])
452 // -  $-\infty$  LSE  $\rightarrow$   $-\infty$ : 空 attention 段 (causal mask 等导致全部 score
453 //   为  $-\infty$ ) 的 LSE 可能为  $+\infty$ , 替换后  $\exp(-\infty - L_{\max}) = 0$ ,
454 //   该段权重退化为 0
455 // - 128-bit 向量化: uint4 = 4  $\times$  float, 每线程处理 4 个输出元素,
456 //   pack load  $\rightarrow$  FMA  $\rightarrow$  pack store 一条流水线
457 // - AI  $\approx$  3 FLOP / 20 bytes  $\approx$  0.15 FLOPS/Byte  $\rightarrow$  severely memory-bound
458 //
459 // 线程到数据的 3D 映射 (以 num_tokens=2, num_heads=2, head_size=8 为例):
460 // kPackSize = 16 / sizeof(float) = 4 (每线程 4 个 float = 128-bit)
461 // threads_per_head = head_size / 4 = 2
462 // total_threads = num_tokens * num_heads * threads_per_head = 8
463 //
464 // global_idx:      0      1      2      3      4      5      6      7
465 // token_head_idx:  0      0      1      1      2      2      3      3
466 //   (第几个 (token,head) 对, 行优先)
467 // pack_idx:        0      1      0      1      0      1      0      1
468 //   (该 (token,head) 对内第几个 pack)
469 // token_idx:        0      0      0      0      1      1      1      1
470 //   (token_head_idx / num_heads, token 变化慢)
471 // head_idx:         0      0      1      1      0      0      1      1
472 //   (token_head_idx % num_heads, head 变化快)
473 // pack_offset:      0      4      0      4      0      4      0      4
474 //   (pack_idx * 4, 该 pack 在 head_size 中的起始元素偏移)
475 // head_offset:      0      0      8      8      16     16     24     24
476 //   (token_idx * num_heads * head_size + head_idx * head_size,
477 //   该 (token,head) 对在 output 展平数组中的起始偏移)
478 //
479 // Grid: ((total_threads + 127) / 128, 1, 1)
480 // Block: (128, 1, 1)
481 // source: LeetCUDA/kernels/openai-triton/merge-attn-states/cuda_merge_attn_states.cu
482 __global__ void merge_attn_states(
483     float *output,           // [num_tokens, num_heads, head_size]
484     const float *prefix_output, // [num_tokens, num_heads, head_size]
485     const float *prefix_lse,   // [num_heads, num_tokens]
486     const float *suffix_output, // [num_tokens, num_heads, head_size]
487     const float *suffix_lse,   // [num_heads, num_tokens]
488     int num_tokens, int num_heads, int head_size) {
489
490     constexpr int kNumThreads = 128;
491     constexpr int kPackSize = 16 / sizeof(float); // 4 floats = 128-bit
492     using pack_t = uint4;
493
494     // 每个 (token, head) 对需要 threads_per_head 个线程覆盖 head_size 个元素
495     const int threads_per_head = head_size / kPackSize;
496     // 实际有效总线程数, 超过这个数的线程会被忽略, block是按照kNumThreads=128
497     // 来分配的, 那么最后一个block的线程数可能会超过实际需要的线程数
498     const int total_threads = num_tokens * num_heads * threads_per_head;
499
500     const int global_idx = blockIdx.x * kNumThreads + threadIdx.x;
501     if (global_idx >= total_threads)
502         return;
503 
```

```

504 // global_idx → (token_head_idx, pack_idx) → (token_idx, head_idx, pack_offset)
505 // token_head_idx: 第几个 (token, head) 对, 行优先展平
506 const int token_head_idx = global_idx / threads_per_head;
507 // pack_idx: 该 (token, head) 对内第几个 pack, 0 ~ threads_per_head-1
508 const int pack_idx = global_idx % threads_per_head;
509
510 // token_head_idx 分解为 (token_idx, head_idx)
511 const int token_idx = token_head_idx / num_heads; // token 变化慢 (外维)
512 const int head_idx = token_head_idx % num_heads; // head 变化快 (内维)
513
514 // pack_offset: 该 pack 覆盖的元素在 head_size 中的起始偏移 (0, 4, 8, ...)
515 const int pack_offset = pack_idx * kPackSize;
516 // head_offset: 该 (token, head) 对在 output 展平一维数组中的起始偏移
517 const int head_offset = token_idx * num_heads * head_size + head_idx * head_size;
518
519 // 定位到当前 (token, head) 的输出段起始
520 const float *prefix_head = prefix_output + head_offset;
521 const float *suffix_head = suffix_output + head_offset;
522 float *output_head = output + head_offset;
523
524 // LSE 布局 [num_heads, num_tokens]: lse[head_idx][token_idx]
525 float p_lse = prefix_lse[head_idx * num_tokens + token_idx];
526 float s_lse = suffix_lse[head_idx * num_tokens + token_idx];
527
528 // inf → -inf: 空 attention 段 LSE 可能为 +inf, 替换后权重退化为 0
529 p_lse = isinf(p_lse) ? -INFINITY : p_lse;
530 s_lse = isinf(s_lse) ? -INFINITY : s_lse;
531
532 // Step 1: max 归一化 (与 safe softmax 同理, 防 exp 溢出)
533 const float max_lse = fmaxf(p_lse, s_lse);
534 p_lse -= max_lse;
535 s_lse -= max_lse;
536
537 // Step 2-3: 指数还原 → 混合比例 alpha, beta
538 const float p_se = expf(p_lse);
539 const float s_se = expf(s_lse);
540 const float out_se = p_se + s_se;
541 const float p_scale = p_se / out_se; // alpha = w_1 / (w_1 + w_2)
542 const float s_scale = s_se / out_se; // beta = w_2 / (w_1 + w_2)
543
544 // Step 4: 逐元素加权合并 0 = alpha * 0_1 + beta * 0_2
545 // 128-bit 向量化: uint4 load → per-element FMA → uint4 store
546 if (pack_offset < head_size) {
547     pack_t p_pack =
548         reinterpret_cast<const pack_t *>(prefix_head)[pack_idx];
549     pack_t s_pack =
550         reinterpret_cast<const pack_t *>(suffix_head)[pack_idx];
551     pack_t o_pack;
552
553     #pragma unroll
554     for (int i = 0; i < kPackSize; ++i) {
555         const float p_v = reinterpret_cast<const float *>(&p_pack)[i];
556         const float s_v = reinterpret_cast<const float *>(&s_pack)[i];
557         const float o_v = p_v * p_scale + (s_v * s_scale); // FMA
558         reinterpret_cast<float *>(&o_pack)[i] = o_v;
559     }
560
561     reinterpret_cast<pack_t *>(output_head)[pack_idx] = o_pack;
562 }
563 }
564
565 // =====
566 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm

```

```

567 // =====
568 // 面试要点:
569 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
570 //   online (1-pass, 增量更新)
571 //   - Online Softmax 是 FlashAttention-2的数学基础
572 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
573 //   + variance)
574 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
575
576 // =====
577 // Phase 3a: Softmax — 三级递进 (面试核心考点)
578 // =====
579 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点? 」
580 // 回答线索: naive(溢出) → safe → online
581
582 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
583 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
584 // 问题: x 值很大时 exp(x) 溢出为 inf
585 template <const int kNumThreads = 256>
586 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
587 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
588 // source: LeetCUDA/kernels/softmax/softmax.cu
589 __global__ void softmax_per_token(float *x, float *y, int N) {
590     const int tid = threadIdx.x;
591     const int idx = blockIdx.x * blockDim.x + tid;
592
593     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
594     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
595     if (idx < N)
596         y[idx] = exp_val / exp_sum;
597 }
598
599 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
600 // 面试重点: 为什么 Softmax 需要 Safe?
601 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
602 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
603 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
604 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
605 // Grid: (S, 1, 1)
606 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
607 // source: LeetCUDA/kernels/softmax/softmax.cu
608 template <const int kNumThreads = 256>
609 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
610     const int tid = threadIdx.x;
611     const int idx = blockIdx.x * blockDim.x + tid;
612
613     // Pass 1: block reduce max — 找最大值
614     float val = (idx < N) ? x[idx] : -FLT_MAX;
615     float max_val = block_reduce_max<kNumThreads>(val);
616
617     // Pass 2: exp(x - max) → block reduce sum
618     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
619     float exp_sum = block_reduce_sum<kNumThreads>(exp_val);
620
621     if (idx < N)
622         y[idx] = exp_val / exp_sum;
623 }
624
625 // ---- Level 3: Online Safe Softmax (FlashAttention-2的数学基础) ----
626 // 面试重点:
627 // 两种使用场景 (公式不同, 但结果等价) :
628 //   1) 单元素增量更新 (online update, 处理新元素 x_i):
629 //       m_new = max(m_old, x_i)

```

```

630 //      d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
631 //  2) 二元合并 (binary merge, 合并两个部分累加器, warp/block reduce 使用):
632 //      m = max(m1, m2) safe softmax用的max值
633 //      d = d1*exp(m1-m) + d2*exp(m2-m) softmax用的分母值
634 //      当 m1≥m2 时退化为 d1 + d2*exp(m2-m1) (d_bigger + d_smaller*exp(m_smaller - m_bigger))
635 //  算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
636 //  Warp Reduce for Online Softmax — binary merge of two partial (m,d) accumulators.
637 //
638 //  不同于单元元素增量更新公式 (m_new = max(m_old, x_i); d_new = d_old*exp(m_old-m_new) + exp(x_i-m_new)) ,
639 //  warp reduce 中每次合并的是两个**已各自归一化到各自 max 的部分累加器**:
640 //  (m1,d1): max 和  $\sum \exp(x_j - m1)$ , 覆盖集合 S1
641 //  (m2,d2): max 和  $\sum \exp(x_k - m2)$ , 覆盖集合 S2
642 //
643 //  合并公式 (对称, 不依赖"新/旧"概念) :
644 //  m = max(m1, m2)
645 //  d = d1*exp(m1-m) + d2*exp(m2-m) ← m1≥m2 时退化为 d1 + d2*exp(m2-m1)
646 //
647 //  该操作满足结合律 → XOR butterfly 任意归约顺序均保证全局正确结果。
648 struct __align__(8) MD {
649     float m; // running max
650     float d; // running denominator (sum of exp(x - max))
651 };
652
653 template <const int kWarpWidth = kWarpSize>
654 __device__ __forceinline__ MD warp_reduce_md(MD md1) {
655     #pragma unroll
656     for (int mask = kWarpWidth >> 1; mask >= 1; mask >>= 1) {
657         MD md2;
658         md2.m = __shfl_xor_sync(0xffffffff, md1.m, mask, kWarpWidth);
659         md2.d = __shfl_xor_sync(0xffffffff, md1.d, mask, kWarpWidth);
660
661         MD b_m = (md1.m > md2.m) ? md1 : md2; // max
662         MD s_m = (md1.m > md2.m) ? md2 : md1;
663
664         // b_m.d 无需 rescale (其基准 max 已是全局 max) ,
665         // s_m.d 需 rescale: exp(s_m.m - b_m.m)
666         md1.d = b_m.d + s_m.d * __expf(s_m.m - b_m.m);
667         md1.m = b_m.m;
668     }
669     return md1;
670 }
671
672 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
673 // Grid: (S, 1, 1)
674 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
675 // source: LeetCUDA/kernels/softmax/softmax.cu
676 template <const int kNumThreads = 256>
677 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {
678     int tid = threadIdx.x;
679     int idx = blockIdx.x * kNumThreads + threadIdx.x;
680     const int kNumWarps = (kNumThreads + kWarpSize - 1) / kWarpSize;
681     __shared__ MD shared[kNumWarps];
682
683     float val = (idx < N) ? x[idx] : -FLT_MAX;
684     int warp = tid / kWarpSize;
685     int lane = tid % kWarpSize;
686
687     // 初始化: 每个线程持有一个 (max, denom) 对
688     MD md;
689     md.m = idx < N ? val : -FLT_MAX;
690     md.d = idx < N ? 1.0f : 0.0f;
691
692     // 第一级规约: warp_reduce_md 在归约中自动更新 m 和 d

```

```

693 md = warp_reduce_md<kWarpSize>(md);
694
695 if (lane == 0)
696     shared[warp] = md;
697 __syncthreads();
698
699 // 第二级归约: 每个 warp 结果再做一次 warp_reduce_md (复用 block_reduce 模式)
700 md = lane < kNumWarps ? shared[lane] : MD{-FLT_MAX, 0.0f};
701 md = warp_reduce_md<kWarpSize>(md); // 用kWarpSize确保每个lane都能拿到最终结果
702
703 // 用全局 max 和 denom 做最终 softmax
704 float d_inv = __fdividef(1.0f, md.d);
705 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
706 if (idx < N) {
707     y[idx] = __expf(val - md.m) * d_inv;
708 }
709 }
710
711 // =====
712 // Phase 3b: RMS Normalization (1-pass reduce)
713 // =====
714 // 面试要点:
715 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
716 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
717 //   - Llama 系列使用 RMS Norm
718 //   - grid(N, K/K), block(K): 一行一个 block
719 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
720 // Block: (128, 1, 1), kNumThreads=K=128 (K>128 时调整模板参数)
721 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
722 template <const int kNumThreads = 128>
723 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
724     int tid = threadIdx.x;
725     int idx = blockIdx.x * blockDim.x + threadIdx.x;
726     const float epsilon = 1e-5f;
727
728     __shared__ float s_variance;
729     float value = (idx < N * K) ? x[idx] : 0.0f;
730     float variance = value * value;
731     variance = block_reduce_sum<kNumThreads>(variance);
732     if (tid == 0)
733         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
734     __syncthreads();
735     if (idx < N * K)
736         y[idx] = (value * s_variance) * g;
737 }
738
739 // RMS Norm + float4
740 // Grid: (N, 1, 1)
741 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
742 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
743 // source: LeetCUDA/kernels/rms-norm/rms_norm.cu
744 template <const int kNumThreads = 128 / 4>
745 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
746     int tid = threadIdx.x;
747     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
748     const float epsilon = 1e-5f;
749
750     __shared__ float s_variance;
751     float4 reg_x = FLOAT4(x[idx]);
752     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
753                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
754                                     : 0.0f;
755     variance = block_reduce_sum<kNumThreads>(variance);

```

```

756     if (tid == 0)
757         s_variance = rsqrtf(variance / (float)K + epsilon);
758     __syncthreads();
759     float4 reg_y;
760     reg_y.x = reg_x.x * s_variance * g;
761     reg_y.y = reg_x.y * s_variance * g;
762     reg_y.z = reg_x.z * s_variance * g;
763     reg_y.w = reg_x.w * s_variance * g;
764     if (idx < N * K)
765         FLOAT4(y[idx]) = reg_y;
766 }
767
768 // =====
769 // Phase 3c: Layer Normalization (2-pass reduce)
770 // =====
771 // 面试要点:
772 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
773 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)2)/K)
774 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
775 // Grid: (N, 1, 1), 一行一个 block
776 // Block: (128, 1, 1), kNumThreads=K=128
777 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
778 template <const int kNumThreads = 128>
779 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
780     int tid = threadIdx.x;
781     int idx = blockIdx.x * blockDim.x + threadIdx.x;
782     const float epsilon = 1e-5f;
783
784     __shared__ float s_mean;
785     __shared__ float s_variance;
786     float value = (idx < N * K) ? x[idx] : 0.0f;
787
788     // Pass 1: compute mean
789     float sum = block_reduce_sum<kNumThreads>(value);
790     if (tid == 0)
791         s_mean = sum / (float)K;
792     __syncthreads(); // 必须等待 s_mean 对所有线程可见
793
794     // Pass 2: compute variance = (x - mean)2
795     float variance = (value - s_mean) * (value - s_mean);
796     variance = block_reduce_sum<kNumThreads>(variance);
797     if (tid == 0)
798         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
799     __syncthreads(); // 必须等待 s_variance 对所有线程可见
800
801     if (idx < N * K)
802         y[idx] = ((value - s_mean) * s_variance) * g + b;
803 }
804
805 // Layer Norm + float4
806 // Grid: (N, 1, 1)
807 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
808 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
809 // source: LeetCUDA/kernels/layer-norm/layer_norm.cu
810 template <const int kNumThreads = 128 / 4>
811 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N, int K) {
812     int tid = threadIdx.x;
813     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
814     const float epsilon = 1e-5f;
815
816     __shared__ float s_mean;
817     __shared__ float s_variance;
818     float4 reg_x = FLOAT4(x[idx]);

```

```

819 float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
820
821 float sum = block_reduce_sum<kNumThreads>(value);
822 if (tid == 0)
823     s_mean = sum / (float)K;
824 __syncthreads();
825
826 float4 reg_x_hat;
827 reg_x_hat.x = reg_x.x - s_mean;
828 reg_x_hat.y = reg_x.y - s_mean;
829 reg_x_hat.z = reg_x.z - s_mean;
830 reg_x_hat.w = reg_x.w - s_mean;
831 float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
832                 reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
833 variance = block_reduce_sum<kNumThreads>(variance);
834 if (tid == 0)
835     s_variance = rsqrtf(variance / (float)K + epsilon);
836 __syncthreads();
837
838 float4 reg_y;
839 reg_y.x = reg_x_hat.x * s_variance * g + b;
840 reg_y.y = reg_x_hat.y * s_variance * g + b;
841 reg_y.z = reg_x_hat.z * s_variance * g + b;
842 reg_y.w = reg_x_hat.w * s_variance * g + b;
843 if (idx < N * K)
844     FLOAT4(y[idx]) = reg_y;
845 }
846
847 // =====
848 // Phase 4: RoPE — 旋转位置编码 (Rotary Position Embedding)
849 // =====
850 // 面试要点:
851 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
852 //     [x1']   [cos(θ)  -sin(θ)] [x1]
853 //     [x2'] = [sin(θ)   cos(θ)] [x2]
854 //   -  $\theta_i = 1 / (\text{theta}^{(2i/d)})$ , theta=10000.0f (Llama 风格)
855 //   - token_pos = idx / N: token 在序列中的位置
856 //   - token_idx = idx % N: token 内的维度对索引
857 //   - 输入 [seq_len, hidden_size], 输出同形状
858
859 // Grid: ((seq_len * N + 255) / 256, 1, 1)
860 // Block: (256, 1, 1)
861 // source: LeetCUDA/kernels/rope/rope.cu
862 __global__ void rope(float *x, float *out, int seq_len, int N) {
863     int idx = blockIdx.x * blockDim.x + threadIdx.x;
864     float x1 = x[idx * 2];
865     float x2 = x[idx * 2 + 1];
866     int token_pos = idx / N; // 序列位置
867     int token_idx = idx % N; // 维度对索引
868
869     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
870     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
871     float sin_v = sinf(token_pos * exp_v);
872     float cos_v = cosf(token_pos * exp_v);
873
874     // 2D 旋转
875     float out1 = x1 * cos_v - x2 * sin_v;
876     float out2 = x1 * sin_v + x2 * cos_v;
877     out[idx * 2] = out1;
878     out[idx * 2 + 1] = out2;
879 }
880
881 // =====

```



```

882 // Phase 5: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
883 // =====
884 // 面试要点 (Bank Conflict 专题) :
885 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
886 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
887 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
888 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
889 //
890 // 转置的四步演进:
891 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
892 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
893 //   BCF: smem 布局 [kWarpSize_S*4][kWarpSize_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
894 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
895 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
896
897 // 每个线程处理 1 个元素, block(16,16)
898 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
899 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
900 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
901 // Block: (16, 16, 1)
902 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
903 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
904     const int c = blockIdx.x * blockDim.x + threadIdx.x; // col in x
905     const int r = blockIdx.y * blockDim.y + threadIdx.y; // row in x
906     if (r < row && c < col) {
907         // x[r][c] → y[c][r]; x stride=col, y (transposed) stride=row
908         y[c * row + r] = x[r * col + c];
909     }
910 }
911
912 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
913 // Block: (16, 16, 1)
914 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
915 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
916 __global__ void mat_transpose_padded(
917     float *x, float *y, const int row, const int col) {
918     const int tx = threadIdx.x;
919     const int ty = threadIdx.y;
920
921     constexpr int TILE = 16;
922     constexpr int PAD = 1;
923     // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
924     __shared__ float tile[TILE * 4][TILE + PAD]; // 64x16
925
926     // x 空间坐标; 每线程覆盖 4 行 (actual row = x_r * 4)
927     const int x_c = blockIdx.x * TILE + tx;
928     const int x_r = blockIdx.y * TILE + ty;
929
930     if (x_r * 4 < row && x_c < col) {
931         // Step 1: 4 rows × 1 col → smem (row-major);
932 #pragma unroll
933         for (int i = 0; i < 4; ++i) {
934             tile[ty * 4 + i][tx] = x[(x_r * 4 + i) * col + x_c];
935         }
936         __syncthreads();
937
938         // Step 2: Transposed read from smem
939         float4 reg_trans;
940         reg_trans.x = tile[tx * 4 + 0][ty];
941         reg_trans.y = tile[tx * 4 + 1][ty];
942         reg_trans.z = tile[tx * 4 + 2][ty];
943         reg_trans.w = tile[tx * 4 + 3][ty];
944

```



```

945 // y 空间坐标: y_r = x 的列, y_c = x 的行
946 const int y_r = blockIdx.x * TILE + ty; // = x col
947 const int y_c = (blockIdx.y * TILE + tx) * 4; // = x row
948
949 // Coalesced write: y[y_r][y_c .. y_c+3]
950 FLOAT4(y[y_r * row + y_c]) = reg_trans;
951 }
952 }
953
954 // =====
955 // Phase 6: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
956 // =====
957 // 面试要点:
958 // - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
959 // - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
960 // - 不同 K 值对应不同分块策略:
961 //   K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
962 //   K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
963 //   K=16 < 32: 一个 warp 用不满, 用 kRowPerWarp=2 让每个 warp 处理 2 行
964 //
965 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
966
967 // ---- SGEMV K32: 基础 warp-per-row ----
968 // 设计: block(32, 4), blockDim.x=kWarpSize=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 kNumWarps 次)
969 // grid(M/4), 每个 warp 负责一行
970 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
971 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
972 // Block: (32, 4, 1), 每行 1 warp
973 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
974 // source: LeetCUDA/kernels/sgemv/sgemv.cu
975 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
976     int tx = threadIdx.x; // 0~31
977     int ty = threadIdx.y; // 0~3
978     int lane = tx % kWarpSize; // 0~31
979     int m = blockIdx.x * blockDim.y + ty; // 全局行号
980     if (m < M) {
981         float sum = 0.0f;
982         // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
983         // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
984         const int NUM_ITERS = (K + kWarpSize - 1) / kWarpSize;
985 #pragma unroll
986         for (int w = 0; w < NUM_ITERS; ++w) {
987             // 假设K是32的整倍数, m * K 本行的起始地址, x: Kx1
988             int k = w * kWarpSize + lane;
989             sum += a[m * K + k] * x[k];
990         }
991         sum = warp_reduce_sum<kWarpSize>(sum);
992         // 每个 warp 处理一行, lane 0 写回结果
993         if (lane == 0)
994             y[m] = sum;
995     }
996 }
997
998 // ---- SGEMV K128: float4 向量化 ----
999 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
1000 // Grid: ((M + 3) / 4, 1, 1)
1001 // Block: (32, 4, 1)
1002 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
1003 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1004 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
1005     int tx = threadIdx.x; // 0~31
1006     int ty = threadIdx.y; // 0~3
1007     int lane = tx % kWarpSize; // 0~31

```

```

1008 int m = blockDim.y * blockIdx.x + ty;
1009
1010 if (m < M) {
1011     float sum = 0.0f;
1012     // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
1013     const int NUM_ITERS = ((K + kWarpSize - 1) / kWarpSize) + 4 - 1) / 4;
1014 #pragma unroll
1015     for (int w = 0; w < NUM_ITERS; ++w) {
1016         int k = (w * kWarpSize + lane) * 4;
1017         float4 reg_x = FLOAT4(x[k]);
1018         float4 reg_a = FLOAT4(a[m * K + k]);
1019         sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
1020             reg_a.w * reg_x.w);
1021     }
1022     sum = warp_reduce_sum<kWarpSize>(sum);
1023     if (lane == 0)
1024         y[m] = sum;
1025 }
1026 }
1027
1028 // ---- SGEMV K16: K < WarpSize, kRowPerWarp=2 ----
1029 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
1030 // kRowPerWarp=2, kNumLanePerRow=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
1031 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
1032 // Block: (32, 4, 1)
1033 // 注意: 这一版是面向 K=16 的专用写法; kRowPerWarp=2 时一个 warp 同时处理 2 行
1034 // source: LeetCUDA/kernels/sgemv/sgemv.cu
1035 template <const int kRowPerWarp = 2>
1036 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
1037     constexpr int kNumLanePerRow = (kWarpSize + kRowPerWarp - 1) / kRowPerWarp; // 16
1038     int tx = threadIdx.x;
1039     int ty = threadIdx.y;
1040     int lane = tx % kWarpSize;
1041     int k = lane % kNumLanePerRow; // row0: 0~15, t0~t15; row1: 0~15, t16~t31
1042     int m = (blockDim.y * blockIdx.x + ty) * kRowPerWarp + lane / kNumLanePerRow;
1043     if (m < M) {
1044         float sum = A[m * K + k] * x[k];
1045         // 按照kNumLanePerRow=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
1046         sum = warp_reduce_sum<kNumLanePerRow>(sum);
1047         // 注意: 判断条件是 k == 0, 不是 lane == 0!
1048         if (k == 0)
1049             y[m] = sum;
1050     }
1051 }
1052
1053 // =====
1054 // Phase 7: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
1055 // =====
1056 // 面试要点 (GEMM 优化五层金字塔):
1057 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
1058 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
1059 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
1060 //   load/store 指令数 Level 4 — Tensor Core (MMA
1061 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
1062 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
1063 //
1064 // 计算密度递进:
1065 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
1066 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
1067 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
1068
1069 // =====
1070 // Phase 7a: SGEMM (非 Tensor Core 路径)

```

```

1071 // =====
1072
1073 // ---- Level 1: SGEMM — Block Tile 32x32 + K Tile 32 ----
1074 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
1075 // C = A x B, C[M, N] = A[M, K] x B[K, N]
1076 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
1077 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
1078 // Block: (32, 32, 1), 1024 线程
1079 // source: LeetCUDA/kernels/sgemm/sgemm.cu
1080 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
1081     constexpr int BM = 32; // vec 版: 32x4 = 128
1082     constexpr int BN = 32; // vec 版: 32x4 = 128
1083     constexpr int BK = 32;
1084     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
1085
1086     int bx = blockIdx.x;
1087     int by = blockIdx.y;
1088     int tx = threadIdx.x;
1089     int tid = threadIdx.y * blockDim.x + tx;
1090
1091     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
1092     // 技巧: 一般来说 “/” 表示线程不是连续排布的, “%” 表示线程是连续排布的
1093     // 因此, 在需要考虑连续访问的维度使用 “%”, 比如, 连续的线程访问列方向连续的元素
1094     // A[M, K], M的stride=K, K的stride=1 → 线程连续访问 K 维度 → 用 %,
1095     // 线程不连续访问 M 维度 → 用 /;
1096     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载;
1097     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载;
1098     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载;
1099     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载;
1100     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row;
1101     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col;
1102
1103     float sum = 0.f; // 遍历完整的K, slice K;
1104     // 这里不用pragma unroll, 因为K不是编译器常量, 编译器无法展开循环
1105     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1106         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1107         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1108         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
1109         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1110         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1111         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
1112         __syncthreads(); // 确保整个 smem tile 加载完毕
1113
1114     #pragma unroll
1115     for (int k = 0; k < BK; ++k) {
1116         int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
1117         int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1118         sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
1119     }
1120     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
1121 }
1122 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
1123 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
1124 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
1125 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
1126 }
1127
1128 // ---- Level 1+: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
1129 // 在 Level 1 基础上引入两层优化:
1130 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
1131 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
1132 // BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
1133 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major

```

```

1134 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
1135 // 1024 × 16 = 16384 = 128 × 128 ✓
1136 //
1137 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
1138 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
1139 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
1140 //
1141 // 加载映射 (每线程 4 个元素, float4):
1142 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
1143 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
1144 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
1145 //
1146 // △ Bank Conflict 提示 (面试加分点):
1147 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
1148 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
1149 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
1150 //
1151 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
1152 // Block: (32, 32, 1), 1024 线程
1153 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
1154 // source: LeetCUDA/kernels/sgemm/sgemm.cu (vec4 variant)
1155 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
1156     constexpr int BM = 128;
1157     constexpr int BN = 128;
1158     constexpr int BK = 32;
1159     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
1160     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
1161
1162     int bx = blockIdx.x;
1163     int by = blockIdx.y;
1164     int tx = threadIdx.x;
1165     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
1166
1167     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
1168     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
1169     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
1170     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
1171     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
1172     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
1173
1174     int load_gmem_a_m = by * BM + load_smem_a_m;
1175     int load_gmem_b_n = bx * BN + load_smem_b_n;
1176
1177     // 4x4 Thread Tile 基址 (独立于加载映射), 这里compute索引的计算逻辑要和
1178     // load索引的计算逻辑分开, load/compute是可以独立索引的, 理解这点很重要。
1179     // 目标C Tile为[BM,BN]=[128x128], 有32x32线程, 则每个线程处理4x4 tile
1180     // 那么, 就可以不重不漏地覆盖[32x4,32x4]=[128x128]的大小
1181     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
1182     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
1183
1184     float sum[4][4] = {0.f};
1185     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
1186         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
1187         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1188         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]); // s_a [BM,BK]
1189         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
1190         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
1191         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]); // s_b [BK,BN]
1192         __syncthreads();
1193     }
1194     #pragma unroll
1195     for (int k = 0; k < BK; ++k) {
1196         // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA

```

```

1197     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
1198                       s_a[comp_smem_a_m_base + 1][k],
1199                       s_a[comp_smem_a_m_base + 2][k],
1200                       s_a[comp_smem_a_m_base + 3][k]};
1201     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
1202                       s_b[k][comp_smem_b_n_base + 1],
1203                       s_b[k][comp_smem_b_n_base + 2],
1204                       s_b[k][comp_smem_b_n_base + 3]};
1205 #pragma unroll
1206     for (int i = 0; i < 4; ++i) {
1207 #pragma unroll
1208         for (int j = 0; j < 4; ++j) {
1209             sum[i][j] += a_vals[i] * b_vals[j];
1210         }
1211     }
1212 }
1213 __syncthreads();
1214 }
1215
1216 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
1217 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
1218 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
1219 #pragma unroll
1220 for (int i = 0; i < 4; ++i) {
1221     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
1222     float4 reg_c;
1223     reg_c.x = sum[i][0];
1224     reg_c.y = sum[i][1];
1225     reg_c.z = sum[i][2];
1226     reg_c.w = sum[i][3];
1227     FLOAT4(c[store_gmem_c_addr]) = reg_c;
1228 }
1229 }
1230
1231 // =====
1232 // Phase 7b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMA m64n128k16)
1233 // =====
1234 // 面试要点:
1235 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
1236 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
1237 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
1238 //     寄存器)
1239 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
1240 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
1241 //
1242 // ★ TN 布局详解 (面试高频考点):
1243 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
1244 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
1245 //   N = Normal (列优先, BLAS 原生格式)
1246 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
1247 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
1248 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
1249 //   BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
1250 //   视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
1251 //   调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
1252 //   T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
1253 //   N on B: B 已经是 BLAS 原生的 col-major, 无需转置
1254 //
1255 // 记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
1256 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
1257 //   N = col-major (列优先, BLAS native = normal)
1258 //
1259 // LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]

```

```

1260 // - A: row-major [M, K], B: row-major [K, N]
1261 // - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)
1262 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
1263 //
1264 // TN 布局: C[M×N] = A[M×K] × B[K×N], B 以 B^T=[N×K] row-major 存储 (A 行优先, B 列优先)
1265 // - A [M×K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1266 // - B^T [N×K]: row-major → 全局索引 B[n*K+k] 即访问原 B 元素 (k,n) (△ 内维连续的是 K)
1267 // - 优势: B^T 已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
1268 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
1269 // - row.col: A 输入 row-major, B 输入 col-major → 与 TN 布局天然匹配
1270 //
1271 // WGMMA (Warp Group MMA, Hopper+):
1272 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
1273 // - m64n128k16: 一次处理 64×128×16 的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
1274 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
1275 // threads) 做计算
1276 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
1277 //
1278 // =====
1279 // Phase 7b-1: MMA PTX 宏定义
1280 // =====
1281 //
1282 // ---- gmem → smem: cp.async ----
1283 // cp.async.commit_group / wait_group / wait_all 语义 (PTX ISA §9.7.9.25.3) :
1284 // - commit_group: 将此前所有未提交的 cp.async 归入一个新的 async-group (per-thread) 。
1285 // - wait_group N: 阻塞直到最多 N 个 async-group 尚未完成 (即 pending ≤ N) 。
1286 // N=0 → 等所有 group 完成。与 wgmma.wait_group 语义一致。
1287 // - wait_all: 等价于 commit_group + wait_group 0。
1288 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
1289 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
1290 #define CP_ASYNC_WAIT_GROUP(n) \
1291     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
1292 //
1293 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
1294 #define CP_ASYNC_CG(dst, src, bytes) \
1295     asm volatile( \
1296         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
1297         "l"(src), "n"(bytes))
1298 //
1299 // ---- ldmatrix: smem → register (Tensor Core 专用) ----
1300 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
1301 // 每次加载 8×8 的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
1302 // aligned: 要求 128-bit 对齐, trans: 转置加载
1303 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
1304     asm volatile( \
1305         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
1306         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
1307         : "r"(addr))
1308 //
1309 #define LDMATRIX_X2(R0, R1, addr) \
1310     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1311         : "=r"(R0), "=r"(R1) \
1312         : "r"(addr))
1313 //
1314 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
1315 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B → 使用 trans
1316 #define LDMATRIX_X2_T(R0, R1, addr) \
1317     asm volatile( \
1318         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
1319         : "=r"(R0), "=r"(R1) \
1320         : "r"(addr))
1321 //
1322 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----

```



```

1323 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
1324 // row.col: A 是 row-major, B 是 col-major
1325 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
1326 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
1327 // C 矩阵大小 16×8=128 元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
1328 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
1329     asm volatile( \
1330         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
1331         "%4, %5}, {%6, %7}, {%8, %9};\n" \
1332         : "=r"(RD0), "=r"(RD1) \
1333         : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
1334         "r"(RC1))
1335
1336 // ---- MMA 辅助函数 ----
1337 // div_ceil: 整数除法向上取整
1338 #define HOST_DEVICE_INLINE __device__ __host__ inline
1339 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
1340     return (a % b != 0) ? (a / b + 1) : (a / b);
1341 }
1342
1343 // =====
1344 // Phase 7b-2: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 (统一循环版)
1345 // =====
1346 // 面试重点 — Tile Hierarchy:
1347 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
1348 //   MMA Tile (more warps): 2×4=8 个 MMA atom → [2×16, 8×4]=[32,32]
1349 //   VAL Tile (more values): 4×4=16 expand → [32×4, 32×4]=[128,128]
1350 //   实际: kMmaTileM=2, kMmaTileN=4, kValTileM=4, kValTileN=4
1351 //           → BM=16×2×4=128, BN=8×4×4=128, Warps=2×4=8, Threads=8×32=256
1352 //
1353 // ★ TN 布局 (T=A 行优先, N=B 列优先):
1354 //   - A[M][K]: row-major → 全局索引 A[m*K + k], smem s_a[BM][BK]
1355 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major) → 全局索引 B[n*K+k]
1356 //   - ldmatrix A: 用 x4 (非转置), A row-major 原生匹配
1357 //   - ldmatrix B: 用 x2 (非转置), B^T row-major 逐行加载 = B 的列, 天然匹配 MMA row.col
1358 //   - MMA 指令: mma.sync.aligned.m16n8k16.row.col → 天然匹配 TN 布局
1359 //
1360 // ★ 统一循环设计 (k 从 0 开始, 消除尾端重复代码):
1361 //   - k 从 0 开始: 每次迭代 k 直接对应 tile k, sel = k % kStages (零偏移)
1362 //   - 加载语义为"预取未来": 迭代 k 加载 tile (k+kStages-1) 供后续使用
1363 //   - cp.async 条件化: 仅当 k+kStages-1 < NUM_K_TILES 时加载
1364 //   - WAIT_GROUP 自适应: 满载期用 kStages-2, 尾部排空用 0
1365 //
1366 // ★ Block Swizzle: 把 C 的逻辑 tile 布局改写为紧凑的二维发射窗口, 增加 A/B tile 的 L2 reuse 机会。
1367 //   C tile 坐标为 (by, bx), by 沿 M 方向, bx 沿 N 方向; 一个 bx 读取同一块 B^T[bx*BN:(bx+1)*BN, :]
1368 //   (TN 布局中 B^T[N][K] 为 row-major), 一个 by 读取同一块 A[by*BM:(by+1)*BM, :]。
1369 //   下图只画 4 个逻辑上相邻的 CTA; SM0~SM3 仅表示可能的发射序列, 不是硬件 SM 绑定或调度保证。
1370 //
1371 //   No Swizzle: grid = (tiles_n, tiles_m, 1), blockIdx.x = bx。
1372 //   C-Matrix (同一 by 行; CTA 先在很宽的 N 方向范围内展开):
1373 //
1374 //       N / bx → 0          1          2          3          ...
1375 //       M ↓
1376 //       by = 0  |  SM0   |  SM1   |  SM2   |  SM3   |  ...
1377 //               |  A 0,B0 |  A 0,B1 |  A 0,B2 |  A 0,B3 |
1378 //               +-----+
1379 //
1380 //   同一 by 的 CTA 复用 A[by], 但分别读取 B^T 0、B^T 1 等不同 B tile。只有 scheduler
1381 //   推进到下一条 by 行、再次遇到相同 bx 时, 才有机会复用 B; 当 tiles_n 很大时, 这段时间内
1382 //   L2 更容易被其他 B tile 挤占。
1383 //
1384 //   Thread Block Swizzle: 先把 N 切成 S 个连续窗口; 一个 z slice 只含 grid_x 个 bx。
1385 //   下图用 grid_x=2 展示一个窄窗口, scheduler 更早进入下一条 by 行:

```

```

1386 //
1387 //      N / local x → 0      1
1388 //      M ↓
1389 //      by = 0      

|     |     |
|-----|-----|
| SM0 | SM1 |
| SM2 | SM3 |

 → A 0; B^T 0, B^T 1
1390 //
1391 //      by = 1      

|     |     |
|-----|-----|
| SM2 | SM3 |
|-----|-----|

 → A 1; B^T 0, B^T 1
1392 //
1393 //
1394 // 对于这个 2x2 窗口：同一行横向复用 A (SM0/SM1、SM2/SM3)，同一列纵向复用 B
1395 // (SM0/SM2、SM1/SM3)。真实 grid_x 不必等于 2，但缩小它会缩短再次访问相同 bx 的距离。
1396 // 此优化只提高 scheduler 在相近时间执行这些 CTA、命中 L2 的机会，不保证 CTA 的 z 顺序、
1397 // 缓存驻留或 L2 hit。
1398 //
1399 // Launch (kBlockSwizzle=false, 当前默认路径)：
1400 // grid = (ceil(N / BN), ceil(M / BM), 1), blockIdx.x 直接就是 N-tile 编号 bx。
1401 // Launch (kBlockSwizzle=true, 需由 launch 端显式构造 3D grid)：
1402 // tiles_n = ceil(N / BN), S = ceil(N / 2048)
1403 // grid = (ceil(tiles_n / S), ceil(M / BM), S)
1404 // bx = blockIdx.z * grid.x + blockIdx.x, col_start = bx * BN。
1405 // 例如 N=8192、BN=128: tiles_n=64, S=4, grid.x=16; z=0/1/2/3 分别覆盖
1406 // bx=0..15/16..31/32..47/48..63, 即 columns [0,2048)/[2048,4096)/[4096,6144)/[6144,8192)。
1407 // grid.x*grid.z 可能产生 bx>=tiles_n 的冗余 CTA。当前 kernel 仍要求 M/N/K 分别按 BM/BN/BK
1408 // 对齐; ceil 只保证逻辑 tile 编号不遗漏, 并不使部分尾 tile 变为安全。
1409 // Block: (256, 1, 1), 8 warps
1410 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1411 // =====
1412 template <const int kMmaM = 16,          // MMA atom M dim (m16n8k16)
1413           const int kMmaN = 8,          // MMA atom N dim
1414           const int kMmaK = 16,         // MMA atom K dim, also BK tile = kMmaK
1415           const int kMmaTileM = 2,      // warps along M, 2 → warp tile M = 32
1416           const int kMmaTileN = 4,      // warps along N, 4 → warp tile N = 32
1417           const int kValTileM = 4,      // value-repeat along M, BM = 16*2*4 = 128
1418           const int kValTileN = 4,      // value-repeat along N, BN = 8*4*4 = 128
1419           const int kStages = 3,        // cp.async pipeline depth
1420           const int kBlockSwizzle = 0>  // 1 enables 3D grid swizzle for L2 locality
1421 __global__ void __launch_bounds__(256)
1422   hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1423   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1424   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1425   // Block Swizzle: 0 时 bx=blockIdx.x; 1 时将 (blockIdx.z, blockIdx.x)
1426   // 线性化为原始 N-tile 编号 bx=z*gridDim.x+x。by 始终是 M-tile 编号。
1427   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1428   const int by = blockIdx.y;
1429   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1430   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1431   constexpr int BK = kMmaK; // 16
1432
1433   // Dynamic shared memory: kStages 个 stage 的 A 和 B
1434   // TN 布局: s_a[BM][BK]=[128][16](A row-major), s_b[BN][BK]=[128][16](B^T
1435   // row-major, 即 B col-major [K×N] 在 smem 中按 B^T[N×K] 存储)
1436   // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1437   // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict。这里先保留最简 PAD=0 版本。
1438   extern __shared__ half smem[];
1439   half *s_a = smem;
1440   half *s_b = smem + kStages * BM * BK; // A 和 B 连续存放
1441   constexpr int s_a_stage_offset = BM * BK; // 128*16
1442   constexpr int s_b_stage_offset = BN * BK; // 128*16 △ BN(128)×BK(16) = B^T row-major
1443   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1444   const int warp_id = tid / kWarpSize; // 0~7
1445   const int lane_id = tid % kWarpSize; // 0~31
1446   // warp_m变化快(0->0,1->1), warp_n变化慢([0,1]->0,[2,3]->1,...), 因此,
1447   // 这种2x4的MMA(Warp) layout是按照col major的顺序来排列MMA0-MMA7的
1448   //      N direction (warp_n)

```



```

1449 //          0      1      2      3
1450 // M direction (warp_m) 0 MMA0   MMA2   MMA4   MMA6
1451 //          1 MMA1   MMA3   MMA5   MMA7
1452 // MMA的排布方式不是唯一的，现在这样排是因为结合MMA/VAL Tile之后，刚好能覆盖整个
1453 // C Tile[128,128]。只要调整MMA/VAL Tile，这里的排布方式也可以跟着调整。要注意的
1454 // 是，MMA0-7逻辑上是可以认为是并行执行的，各自的计算结果累计加到对应的C Tile位置上。
1455 const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1456 const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1457
1458 // 线程到 global memory 的映射（用于加载 A 和 B）共 256 个线程
1459 // TN 布局关键：A[m*K+k] 是 row-major，B^T[n*K+k] 是 row-major（内维连续的是 K）
1460 int load_smem_a_m = tid / 2; // 0~127
1461 int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1462 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向（row-major 的行）
1463 int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向（row-major 的列）
1464 int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1465 int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1466 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1467     return;
1468
1469 // 8个Warps(MMAs)的排布是M方向2个，N方向4个，则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32]。
1470 // CUDA中每个block能放的线程数是有上限的（warp数量有限），一般为4或者8个warps。为了能处理更大的C tile，
1471 // 需要把MMA Atom的tile再M方向和N方向各自重复4次，得到[128,128]的C block tile，这就是Value Tile的作用。
1472 // MMA Tile对应到cutlass cute中的TiledMMA的概念，Value Tile对应到cutlass cute中的PermuteMNK的概念。
1473 uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1474
1475 // CVTA：一次转换 smem 基地址，避免每次 cp.async 都做转换
1476 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1477 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1478
1479 // 预加载前 (kStages-1) 个 stage
1480 // TN 布局：A 的 gmem 索引用 m*K+k(row-major)，B^T 用 n*K+k(row-major，即 B col-major [K×N])
1481 #pragma unroll
1482 for (int k = 0; k < (kStages - 1); ++k) {
1483     int load_gmem_a_k = k * BK + load_smem_a_k;
1484     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1485     uint32_t load_smem_a_ptr = (smem_a_base_ptr +
1486         (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) * sizeof(half)
1487     );
1488     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1489
1490     int load_gmem_b_k = k * BK + load_smem_b_k;
1491     // B^T: [n][k] row-major (即 B[k][n] col-major) △
1492     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1493     uint32_t load_smem_b_ptr = (smem_b_base_ptr +
1494         (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) * sizeof(half)
1495     );
1496     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1497
1498     CP_ASYNC_COMMIT_GROUP();
1499 }
1500
1501 CP_ASYNC_WAIT_GROUP(kStages - 2); // 允许有 kStages-2 个group未完成
1502 __syncthreads();
1503
1504 // 统一循环：k 从 0 开始，每次迭代负责 tile k（加载 + 计算合并为单循环）
1505 const int NUM_K_TILES = div_ceil(K, BK);
1506 // 此处不用pragma unroll，因为 K 不是编译期常量，因此NUM_K_TILES 也不是编译期常量
1507 for (int k = 0; k < NUM_K_TILES; ++k) {
1508     int smem_sel = k % kStages; // 计算 tile k 的 stage
1509     int smem_sel_next = (k + kStages - 1) % kStages; // 预加载目标 stage
1510
1511     // 条件加载：预加载 tile (k+kStages-1) 供将来使用。因为在prefetch loop中已经

```

```

1512 // load了(kStages-1) 个 stage, 因此这里是从 tile (k+kStages-1) 开始预加载
1513 // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用 n*K+k
1514 // (row-major, 内维连续的是 K)
1515 if (k + kStages - 1 < NUM_K_TILES) {
1516     int load_gmem_a_k = (k + kStages - 1) * BK + load_smem_a_k;
1517     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1518     int load_gmem_b_k = (k + kStages - 1) * BK + load_smem_b_k;
1519     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k]
1520
1521     uint32_t load_smem_a_ptr =
1522         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1523             load_smem_a_m * BK + load_smem_a_k) *
1524             sizeof(half));
1525     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1526
1527     uint32_t load_smem_b_ptr =
1528         (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1529             load_smem_b_n * BK + load_smem_b_k) *
1530             sizeof(half));
1531     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1532     CP_ASYNC.COMMIT_GROUP();
1533 }
1534
1535 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1536 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major; B 用 x2 (非转置), smem 中
1537 // B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1538 uint32_t RA[kValTileM][4]; // [4][4], M方向4次repeat, A regs = 4 uint32_t per MMA
1539 uint32_t RB[kValTileN][2]; // [4][2], N方向4次repeat, B regs = 2 uint32_t per MMA
1540
1541 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1542 #pragma unroll
1543 for (int i = 0; i < kValTileM; ++i) {
1544     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1545     // 其中 warp_m {0,1}; warp_m_0 offsets {0,16,32,48}; warp_m_1 offsets {64,80,96,112}
1546     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1547     // {0,16,32,...,112} + {0~15} = {0~127}, 按照col-major的顺序访问A的4个8x8 matrix (16x16)
1548     // ldmatrix.{...}.x4.{...} 需要warp内32个线程都参与, 每个线程提供一个有效的, 不重叠的addr
1549     int lane_smem_a_m = warp_smem_a_m + lane_id % 16; // t{0...15}=0~15, t{16...31}=0~15
1550     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8, t{0...15}=0, t{16...31}=8
1551     uint32_t lane_smem_a_ptr =
1552         (smem_a_base_ptr +
1553             (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1554             sizeof(half));
1555     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1556 }
1557
1558 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1559 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1560 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1561 #pragma unroll
1562 for (int j = 0; j < kValTileN; ++j) {
1563     // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1564     // warp_n_0 offsets {0,8,16,24}; warp_n_1 offsets {32,40,48,56}
1565     // warp_n_2 offsets {64,72,80,88}; warp_n_3 offsets {96,104,112,120}
1566     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1567     // {0,8,...,120} + {0~7} = {0~127}, 按照row-major的顺序访问B^T的2个8x8 matrix (8x16)
1568     // ldmatrix.{...}.x2.{...} 需要warp内前16个线程参与, 后16个线程传的addr会被忽略
1569     int lane_smem_b_n = warp_smem_b_n + lane_id % 8; // t{0...7}=0~7, t{8...15}=0~7
1570     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // t{0...7}=0, t{8...15}=8
1571     uint32_t lane_smem_b_ptr =
1572         (smem_b_base_ptr +
1573             (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1574             sizeof(half));

```

```

1575     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1576 }
1577
1578 // MMA compute: 每个Warp(MMA) 在M方向重复kValTileM(4)次, 在N方向重复kValTileN(4)次
1579 #pragma unroll
1580 for (int i = 0; i < kValTileM; ++i) {
1581     #pragma unroll
1582     for (int j = 0; j < kValTileN; ++j) {
1583         MMA16816(RC[i][j][0], RC[i][j][1], // C fragment
1584                 RA[i][0], RA[i][1], RA[i][2], RA[i][3], // A fragment
1585                 RB[j][0], RB[j][1], // B fragment
1586                 RC[i][j][0], RC[i][j][1]);
1587     }
1588 }
1589
1590 // 自适应等待: 流水线满载期用 kStages-2, 尾部排空用 0
1591 if (k + kStages - 1 < NUM_K_TILES) {
1592     CP_ASYNC_WAIT_GROUP(kStages - 2);
1593 } else { // 对于尾部的 k, 等待所有剩余的 cp.async 完成
1594     CP_ASYNC_WAIT_GROUP(0);
1595 }
1596 __syncthreads();
1597 }
1598
1599 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1600 {
1601     for (int i = 0; i < kValTileM; ++i) {
1602         // RC[kValTileM][kValTileN][2] 中 RC[...] [...] [2] 中保存了2个 uint32
1603         // 寄存器, 每个uint32 寄存器表示两个临近的fp16值: {c0,c1}, 然后RC[...] [...] [0]
1604         // 和RC[...] [...] [1]代表的是按照col-major排布的2个8x8子矩阵上 (不同物理行, 跨8x8)
1605         // 同一个位置上的元素, 也就是, 实际代表了2个不同的行的元素, 因此要分开RC0, RC1;
1606         // RC0表示第一行, 8个half, 可以用4个uint32寄存器来装; 同理, RC1表示第二行。
1607         uint32_t RC0[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1608         uint32_t RC1[kValTileN][4]; // 32 bits x 4 = 128 bits = 8 half
1609         // =====
1610         // MMA m16n8k16 C fragment — a single 16x8 matrix (registers per warp):
1611         // Thread t holds 4 half values → RC[0] for rows 0-7, RC[1] for rows 8-15.
1612         // Mapping: row = t/4, col-pair = t%4 (each uint32 = 2 half values).
1613         //
1614         //      cols: c0-1    c2-3    c4-5    c6-7
1615         // r0:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1616         // r1:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1617         // r2:  T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[0]
1618         // ...                               |
1619         // r7:  T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1620         // r8:  T0{c0,c1}  T1{c2,c3}  T2{c4,c5}  T3{c6,c7}  ↵
1621         // r9:  T4{c0,c1}  T5{c2,c3}  T6{c4,c5}  T7{c6,c7}  |
1622         // r10: T8{c0,c1}  T9{c2,c3}  T10{c4,c5} T11{c6,c7} | RC[1]
1623         // ...                               |
1624         // r15: T28{c0,c1} T29{c2,c3} T30{c4,c5} T31{c6,c7} ↵
1625         //
1626         // Within a 4-lane group (e.g. T0-T3 for row 0):
1627         // lane+0 holds {c0,c1}, lane+1 holds {c2,c3},
1628         // lane+2 holds {c4,c5}, lane+3 holds {c6,c7}.
1629         // shfl 从 lane+1/+2/+3 各收 2 个 half 到 lane+0, 4 次 shuffle 凑齐
1630         // 一行 8 个 half = {c0,c1,c2,c3,c4,c5,c6,c7}, 再由 lane%4==0 128-bit store.
1631         // =====
1632     #pragma unroll
1633     for (int j = 0; j < kValTileN; ++j) {
1634         RC0[j][0] = RC[i][j][0];
1635         RC1[j][0] = RC[i][j][1];
1636         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1637         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);

```

```

1638     RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1639     RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1640     RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1641     RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1642 }
1643 // 每 4 个 lane 中只有 lane 0 做 128-bit store
1644 // lane_id / 4 → 行号映射 (每 4 个连续 lane 负责上8x8矩阵和下8x8矩阵各一行) :
1645 //
1646 // | lane_id | lane_id/4 | RC[0] 的行 | RC[1] 的行 |
1647 // |-----|-----|-----|-----|
1648 // | 0~3    | 0        | row 0     | row 8     |
1649 // | 4~7    | 1        | row 1     | row 9     |
1650 // | 8~11   | 2        | row 2     | row 10    |
1651 // | 12~15  | 3        | row 3     | row 11    |
1652 // | 16~19  | 4        | row 4     | row 12    |
1653 // | 20~23  | 5        | row 5     | row 13    |
1654 // | 24~27  | 6        | row 6     | row 14    |
1655 // | 28~31  | 7        | row 7     | row 15    |
1656 //
1657 if (lane_id % 4 == 0) {
1658     // {0,1} * (16 * 4) + i * 16 = {0,64} + {0,16,32,48} = {0,16,32,48,64,80,96,112}
1659     int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM; // smem row
1660     // 这里用 lane_id / 4 → {0,1,2,3,4,5,6,7}, 对应 RC0/RC1 (+8) 的行号, 表示2个8x8矩阵的行号
1661     int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4; // gmem row
1662 #pragma unroll
1663     for (int j = 0; j < kValTileN; ++j) {
1664         // {0,...,3} * (8 * 4) + j * 8 = {0,32,64,96} + {0,8,16,24} = {0,8,...,120}
1665         int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN; // smem col
1666         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n; // gmem col
1667         int store_gmem_c_addr_0 = store_lane_gmem_c_m * N + store_lane_gmem_c_n; // 1-th 8x8 matrix
1668         int store_gmem_c_addr_1 = (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n; // 2-th 8x8 matrix
1669         // 128-bit store: 一次写入 8 个 half
1670         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
1671             *reinterpret_cast<float4*>(&RC0[j][0]);
1672         *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
1673             *reinterpret_cast<float4*>(&RC1[j][0]);
1674     }
1675 }
1676 }
1677 }
1678 }
1679
1680 // =====
1681 // Phase 7b-3: HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局
1682 // + XOR swizzle 消除 smem bank conflict (统一循环版)
1683 // =====
1684 // 面试要点 (Swizzle 原理 — smem Bank Conflict 消除) :
1685 // - 问题: ldmatrix 以 8x8 片段 (m8n8) 从 smem 加载 16x16 的矩阵。同一 warp 中
1686 //   不同 lane 访问的地址在 bank 上产生冲突 → 串行化 (most common: 2/4-way)。
1687 // - XOR swizzle 方案: 对列地址做 `((j>>3) ^ (i>>2)) % 2 << 3` 的 XOR 置换,
1688 //   将连续 4 行的 bank 映射打散。每 4 行为一组的 pattern 交替:
1689 //   rows 0~3: 列 0→地址偏移 0, 列 8→地址偏移 8
1690 //   rows 4~7: 列 0→地址偏移 8, 列 8→地址偏移 0 (XOR 翻转)
1691 //   rows 8~11: repeat rows 0~3 pattern
1692 //   rows 12~15: repeat rows 4~7 pattern
1693 // - 效果: 原来 n-way 的 bank conflict 降低到 1-way (fully conflict-free)。
1694 // - 优势: 无需 smem PAD (不浪费空间), 原理上完全消除特定 pattern 的 bank conflict。
1695 // - 局限性: 要求 kColStride ≤ 16 (即 BK ≤ 16), kStep ∈ {4,8}。
1696 //   对 MMA m16n8k16 的 BK=16 而言刚好满足。
1697 //
1698 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle.cu
1699 // 参考: https://zhuanlan.zhihu.com/p/4746910252
1700

```

```

1701 // ---- Swizzle 辅助函数 ----
1702
1703 // i: row index; j: col index.
1704 // Returns the chunk-level (8 fp16 = 16 B per chunk) column swizzle, i.e.
1705 // 0 / 8 / 16 / ... that the caller adds to the chunk-aligned base column.
1706 // The within-chunk offset (`j & 7`) is preserved by the caller.
1707 //
1708 // kColStride matches the equivalent CuTe ``Swizzle<B, M, S>`` swizzle
1709 // hardware mode used by TMA bulk-tensor descriptors:
1710 // * kColStride = 16 fp16 = 32 B/row -> Swizzle<1, 4, 3> = SWIZZLE_32B
1711 // * kColStride = 32 fp16 = 64 B/row -> Swizzle<2, 4, 3> = SWIZZLE_64B
1712 // * kColStride = 64 fp16 = 128 B/row -> Swizzle<3, 4, 3> = SWIZZLE_128B
1713 // The XOR mask is derived from the byte-address bit positions used by the
1714 // underlying CuTe swizzle pattern: bits {4..(4+B-1)} XOR bits
1715 // {7..(7+B-1)} of the absolute byte address. Translating to (i, j) with
1716 // row stride ``kColStride * 2`` bytes:
1717 // * 32B (B=1) : (j>>3) ^ (i>>2) [chunk in {0, 1}]
1718 // * 64B (B=2) : chunk(2b) ^ {(i>>1)&1, (i>>2)&1} [{0..3}]
1719 // * 128B (B=3) : chunk(3b) ^ {i&1, (i>>1)&1, (i>>2)&1} [{0..7}]
1720 // source: LeetCUDA/ffpa-attn/cuffpa/swizzle.cuh
1721 template <const int kColStride = 16, const int kStep = 8>
1722 static __device__ __forceinline__ int permuted(int i, int j) {
1723     static_assert(kColStride == 16 || kColStride == 32 || kColStride == 64 ||
1724         kColStride == 8,
1725         "kColStride must be one of {8, 16, 32, 64} (matches "
1726         "SWIZZLE_32B/64B/128B + the "
1727         "kStep=4 narrow legacy case).");
1728     static_assert(kStep == 4 || kStep == 8, "kStep must be 8 or 4.");
1729     static_assert(kColStride % kStep == 0,
1730         "kColStride must be multiple of kStep.");
1731     if constexpr (kStep == 4) {
1732         static_assert(kColStride <= 16, "kStep=4 only supports (kColStride <= 16).");
1733         return (((j >> 2) ^ (i >> 2)) % (kColStride >> 2)) << 2;
1734     } else if constexpr (kColStride == 16) {
1735         return (((j >> 3) ^ (i >> 2)) & 1) << 3; // SWIZZLE_32B
1736     } else if constexpr (kColStride == 32) {
1737         const int chunk = (j >> 3) & 3;
1738         const int xor_mask = ((i >> 1) & 1) | (((i >> 2) & 1) << 1);
1739         return (chunk ^ xor_mask) << 3; // SWIZZLE_64B
1740     } else {
1741         // kColStride == 64
1742         const int chunk = (j >> 3) & 7;
1743         const int xor_mask =
1744             (i & 1) | (((i >> 1) & 1) << 1) | (((i >> 2) & 1) << 2);
1745         return (chunk ^ xor_mask) << 3; // SWIZZLE_128B
1746     }
1747 }
1748 // Manually SMEM swizzling for bank conflict free.
1749 // -----
1750 // [INFO] Assert smem store layout col_stride <= 16, prefer 16. |
1751 // [INFO] For logical_col_stride > 16, we have to permute the |
1752 // [INFO] smem store layout using col major ZigZag method: |
1753 // [INFO] e.g., --> Q smem logical layout [Br][64]. |
1754 // [INFO] --> col major ZigZag permuted --> |
1755 // [INFO] --> Q smem store layout [4][Br][16]. |
1756 // -----
1757 // -----swizzle layout-----
1758 // -----logical col 0~64, step 8-----
1759 // -----smem col 0~16, step 8-----
1760 // -----
1761 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1762 // |row 0 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |

```

```

1764 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1765 // |row 1 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1766 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1767 // |row 2 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1768 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1769 // |row 3 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1770 // -----
1771 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1772 // |row 4 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1773 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1774 // |row 5 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1775 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1776 // |row 6 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1777 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1778 // |row 7 | 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1779 // -----
1780 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1781 // |row 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1782 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1783 // |row 9 | 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1784 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1785 // |row 10| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1786 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1787 // |row 11| 0 | 8 | 0 | 8 | 0 | 8 | 0 | 8 |
1788 // -----
1789 // |bank |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |b 0~3 |b 4~7 |
1790 // |row 12| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1791 // |bank |b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|b 8~11|b12~15|
1792 // |row 13| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1793 // |bank |b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|b16~19|b20~23|
1794 // |row 14| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1795 // |bank |b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|b24~27|b28~31|
1796 // |row 15| 8 | 0 | 8 | 0 | 8 | 0 | 8 | 0 |
1797 // -----
1798 template <const int kColStride = 16>
1799 static __device__ __forceinline__ int swizzle(int i, int j) {
1800     return permuted<kColStride, 8>(i, j);
1801 }
1802
1803 // =====
1804 // Phase 7b-3: HGEMM MMA — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
1805 // + Register Double Buffering (kValTileK=2, BK=32 统一 tile)
1806 // =====
1807 // 在 Phase 7b-2 的 smem XOR swizzle 基础上增加寄存器双缓冲:
1808 // - kValTileK=2: 每个 BK tile 包含 2 个 kMmaK slice (BK = kMmaK * kValTileK = 32)
1809 // - BK=32 统一 tile: kMmaK=0 和 kMmaK=1 数据连续存放在同一个 BK=32 宽的 smem tile 中
1810 // - RA[2][kValTileM][4] / RB[2][kValTileN][2]: 双份寄存器乒乓切换
1811 // - ldmatrix 与 MMA 计算重叠: 加载 k_step=1 的同时用另一组寄存器做 k_step=0 计算
1812 //
1813 // ★ smem 布局:
1814 // s_a: [stage0][stage1][stage2], 每个 stage = BM × BK = 128 × 32
1815 // s_b: 紧接 s_a 之后
1816 // 每个 stage 内 k_step=0 列偏移 0, k_step=1 列偏移 +kMmaK(=16)
1817 //
1818 // ★ 寄存器双缓冲时间线 (每 k 迭代内):
1819 // Step 1: G→S cp.async 预取 stage(k+kStages-1) 全部 k_step (条件)
1820 // Step 2: MMA k_step=0 (用 reg[load], 已在上轮 post-wait 中 ldmatrix)
1821 // Step 3: for k_step=1..kValTileK-1: ldmatrix(列偏移 k_step*kMmaK)→reg[store], flip, MMA→reg[load]
1822 // Step 4: adaptive wait + __syncthreads
1823 // Step 5: S→R ldmatrix 预加载 stage(k+1) k_step=0 → reg[store] (条件)
1824 //
1825 // 参考: LeetCUDA/kernels/hgemm/mma/swizzle/hgemm_mma_stage_tn_swizzle_x2.cu
1826 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle

```



```

1827 // Block: (256, 1, 1), 8 warps
1828 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
1829           const int kMmaN = 8,           // MMA atom N dim
1830           const int kMmaK = 16,          // MMA atom K dim
1831           const int kMmaTileM = 2,       // warps along M, 2 → warp tile M = 32
1832           const int kMmaTileN = 4,       // warps along N, 4 → warp tile N = 32
1833           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
1834           const int kValTileN = 4,       // value-repeat along N, BN = 8*4*4 = 128
1835           const int kValTileK = 2,       // MMA_K slices per BK tile, BK = kMmaK*2 = 32
1836           const int kStages = 3,         // cp.async pipeline depth
1837           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
1838 __global__ void __launch_bounds__(256)
1839   hgemm_mma_stages_tn_swizzle(half *A, half *B, half *C, int M, int N, int K) {
1840   static_assert(kValTileK == 2, "Only support kValTileK=2 for register double buffering");
1841   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
1842   static_assert(kStages >= 2, "kStages must be >= 2 for cp.async pipeline");
1843   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1844   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
1845   const int by = blockIdx.y;
1846   constexpr int BM = kMmaM * kMmaTileM * kValTileM; // 16*2*4=128
1847   constexpr int BN = kMmaN * kMmaTileN * kValTileN; // 8*4*4=128
1848   constexpr int BK = kMmaK * kValTileK;             // 16*2=32
1849
1850   // kStages stages, each with BM×BK for A, BN×BK for B
1851   // smem: kValTileK 个 kMmaK slice 连续存放在同一个 BK=32 宽 tile 中
1852   extern __shared__ half smem[];
1853   half *s_a = smem;
1854   half *s_b = smem + kStages * BM * BK;
1855   constexpr int s_a_stage_offset = BM * BK;
1856   constexpr int s_b_stage_offset = BN * BK;
1857
1858   const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1859   const int warp_id = tid / kWarpSize;
1860   const int lane_id = tid % kWarpSize;
1861   const int warp_m = warp_id % kMmaTileM; // 0,1 (M 方向 2 个 warp) kMmaTileM = 2
1862   const int warp_n = warp_id / kMmaTileM; // 0,1,2,3 (N 方向 4 个 warp)
1863
1864   // 线程到 global memory 的映射 (用于加载 A 和 B) 共 256 个线程
1865   // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1866   // 注意: smem_a_k 和 smem_b_k 依然使用 0/8, 虽然BK=16*2=32, 在后续的kValTileK循环中
1867   // 会加上 k_step*kMmaK=0/16, 最终得到 smem 中的列偏移 0/8/16/24, 正好覆盖 BK=32
1868   int load_smem_a_m = tid / 2;           // 0~127
1869   int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8
1870   int load_smem_b_n = tid / 2;           // 0~127 → B^T 的 N 方向 (row-major 的行)
1871   int load_smem_b_k = (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1872   int load_gmem_a_m = by * BM + load_smem_a_m; // C/A 全局行号 = M 方向的 tile 起始 + 线程偏移
1873   int load_gmem_b_n = bx * BN + load_smem_b_n; // C/B 全局列号 = N 方向的 tile 起始 + 线程偏移
1874   if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1875     return;
1876
1877   // 8个Warps(MMAs)的排布是M方向2个, N方向4个, 则MMA Atom一次性能处理的tile大小是[2*16,4*8]=[32,32].
1878   // CUDA中每个block能放的线程数是有上限的 (warp数量有限), 一般为4或者8个warps。为了能处理更大的C tile,
1879   // 需要把MMA Atom的tile再M方向和N方向各自重复4次, 得到[128,128]的C block tile, 这就是Value Tile的作用。
1880   // MMA Tile对应到cutlass cute中的TiledMMA的概念, Value Tile对应到cutlass cute中的PermuteMNMK的概念。
1881   uint32_t RC[kValTileM][kValTileN][2] = {0}; // 初始化为 0
1882
1883   uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1884   uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1885
1886   // Prefetch (kStages-1) stages: load all k_step slices for each early stage
1887   #pragma unroll
1888   for (int k = 0; k < (kStages - 1); ++k) {
1889   #pragma unroll

```



```

1890 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1891     int load_gmem_a_k = k * BK + (k_step * kMmaK) + load_smem_a_k;
1892     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1893     int load_gmem_b_k = k * BK + (k_step * kMmaK) + load_smem_b_k;
1894     int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1895
1896     uint32_t load_smem_a_ptr =
1897         (smem_a_base_ptr +
1898          (k * s_a_stage_offset + load_smem_a_m * BK +
1899           k_step * kMmaK +
1900            swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1901           sizeof(half));
1902     CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1903
1904     uint32_t load_smem_b_ptr =
1905         (smem_b_base_ptr +
1906          (k * s_b_stage_offset + load_smem_b_n * BK +
1907           k_step * kMmaK +
1908            swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1909           sizeof(half));
1910     CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1911 }
1912 CP_ASYNC.COMMIT_GROUP();
1913 }
1914
1915 CP_ASYNC.WAIT_GROUP(kStages - 2);
1916 __syncthreads();
1917
1918 // Register double buffers: RA[0/1] for k_step=0/1 A data, RB[0/1] for B data
1919 uint32_t RA[2][kValTileM][4];
1920 uint32_t RB[2][kValTileN][2];
1921 int reg_st_idx = 0; // write target for ldmatrix
1922 int reg_ld_idx = 1; // read source for MMA
1923
1924 // Initial ldmatrix: load stage 0, S -> R, k_step=0 (0~15列) -> reg[0]
1925 // 此时, reg_st_idx = 0, 对0位置的寄存器buffer做初始化。
1926 #pragma unroll
1927 for (int i = 0; i < kValTileM; ++i) {
1928     int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
1929     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1930     int lane_smem_a_k = (lane_id / 16) * 8;
1931     uint32_t lane_smem_a_ptr =
1932         (smem_a_base_ptr +
1933          (0 * s_a_stage_offset + lane_smem_a_m * BK +
1934           swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
1935           sizeof(half));
1936     LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
1937                 RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
1938                 lane_smem_a_ptr);
1939 }
1940 #pragma unroll
1941 for (int j = 0; j < kValTileN; ++j) {
1942     int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
1943     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1944     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1945     uint32_t lane_smem_b_ptr =
1946         (smem_b_base_ptr +
1947          (0 * s_b_stage_offset + lane_smem_b_n * BK +
1948           swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
1949           sizeof(half));
1950     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
1951                 lane_smem_b_ptr);
1952 }

```

```

1953 // 统一循环: k 从 0 开始, 条件 G→S / S→R / wait 处理所有边界情况
1954 // BK = kMmaK * kValTileK = 32, 每个 k 迭代覆盖 BK=32 个 K 元素
1955 const int NUM_K_TILES = div_ceil(K, BK);
1956 for (int k = 0; k < NUM_K_TILES; ++k) {
1957     int smem_sel = k % kStages;
1958     int smem_sel_next = (k + kStages - 1) % kStages;
1959
1960     // G→S: 条件预取 stage(k+kStages-1) 全部 k_step slice
1961     if (k + kStages - 1 < NUM_K_TILES) {
1962 #pragma unroll
1963         for (int k_step = 0; k_step < kValTileK; ++k_step) {
1964             int load_gmem_a_k =
1965                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_a_k;
1966             int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
1967             int load_gmem_b_k =
1968                 (k + kStages - 1) * BK + (k_step * kMmaK) + load_smem_b_k;
1969             int load_gmem_b_addr = load_gmem_b_n * K + load_gmem_b_k;
1970
1971             uint32_t load_smem_a_ptr =
1972                 (smem_a_base_ptr +
1973                  (smem_sel_next * s_a_stage_offset + load_smem_a_m * BK +
1974                   k_step * kMmaK +
1975                   swizzle<kMmaK>(load_smem_a_m, load_smem_a_k)) *
1976                  sizeof(half));
1977             CP_ASYNC.CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1978
1979             uint32_t load_smem_b_ptr =
1980                 (smem_b_base_ptr +
1981                  (smem_sel_next * s_b_stage_offset + load_smem_b_n * BK +
1982                   k_step * kMmaK +
1983                   swizzle<kMmaK>(load_smem_b_n, load_smem_b_k)) *
1984                  sizeof(half));
1985             CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1986         }
1987     }
1988     CP_ASYNC.COMMIT_GROUP();
1989 }
1990
1991 // 内层 k_step 循环: flip → ldmatrix(k_step+1) → MMA, 乒乓交替
1992 // 初始: st=0, ld=1, reg[0]=preload k_step=0; reg[1]=未初始化
1993 // 每轮: flip→ldmatrix next k_step→reg[st], MMA reg[ld] (k_step+1 的 ldmatrix 条件化)
1994
1995 #pragma unroll
1996 for (int k_step = 0; k_step < kValTileK; ++k_step) {
1997     reg_st_idx ^= 1; // 0 → 1; 1 → 0
1998     reg_ld_idx ^= 1; // 1 → 0; 0 → 1
1999
2000     if (k_step + 1 < kValTileK) {
2001         int smem_k_offset = (k_step + 1) * kMmaK;
2002 #pragma unroll
2003         for (int i = 0; i < kValTileM; ++i) {
2004             int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2005             int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2006             int lane_smem_a_k = (lane_id / 16) * 8;
2007             uint32_t lane_smem_a_ptr =
2008                 (smem_a_base_ptr +
2009                  (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2010                   smem_k_offset +
2011                   swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2012                  sizeof(half));
2013             LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2014                         RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2015                         lane_smem_a_ptr);

```

```

2016     }
2017 #pragma unroll
2018     for (int j = 0; j < kValTileN; ++j) {
2019         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2020         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2021         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2022         uint32_t lane_smem_b_ptr =
2023             (smem_b_base_ptr +
2024              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +
2025               smem_k_offset +
2026               swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2027               sizeof(half));
2028         LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2029                     lane_smem_b_ptr);
2030     }
2031 }
2032
2033 #pragma unroll
2034     for (int i = 0; i < kValTileM; ++i) {
2035 #pragma unroll
2036         for (int j = 0; j < kValTileN; ++j) {
2037             HMMMA16816(RC[i][j][0], RC[i][j][1],
2038                        RA[reg_ld_idx][i][0], RA[reg_ld_idx][i][1],
2039                        RA[reg_ld_idx][i][2], RA[reg_ld_idx][i][3],
2040                        RB[reg_ld_idx][j][0], RB[reg_ld_idx][j][1],
2041                        RC[i][j][0], RC[i][j][1]);
2042         }
2043     }
2044 }
2045
2046 // 自适应等待: 满载期用 kStages-2, 尾部排空用 0
2047 if (k + kStages - 1 < NUM_K_TILES) {
2048     CP_ASYNC_WAIT_GROUP(kStages - 2);
2049 } else {
2050     CP_ASYNC_WAIT_GROUP(0);
2051 }
2052 __syncthreads();
2053
2054 // 从下一阶段加载 k_step=0 数据, 供下一轮 k 迭代使用, 此时 reg_st_idx=0
2055 if (k + 1 < NUM_K_TILES) {
2056     smem_sel = (k + 1) % kStages; // old smem_sel + 1
2057 #pragma unroll
2058     for (int i = 0; i < kValTileM; ++i) {
2059         int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2060         int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
2061         int lane_smem_a_k = (lane_id / 16) * 8;
2062         uint32_t lane_smem_a_ptr =
2063             (smem_a_base_ptr +
2064              (smem_sel * s_a_stage_offset + lane_smem_a_m * BK +
2065               swizzle<kMmaK>(lane_smem_a_m, lane_smem_a_k)) *
2066               sizeof(half));
2067         LDMATRIX_X4(RA[reg_st_idx][i][0], RA[reg_st_idx][i][1],
2068                    RA[reg_st_idx][i][2], RA[reg_st_idx][i][3],
2069                    lane_smem_a_ptr);
2070     }
2071 #pragma unroll
2072     for (int j = 0; j < kValTileN; ++j) {
2073         int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2074         int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
2075         int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
2076         uint32_t lane_smem_b_ptr =
2077             (smem_b_base_ptr +
2078              (smem_sel * s_b_stage_offset + lane_smem_b_n * BK +

```

```

2079         swizzle<kMmaK>(lane_smem_b_n, lane_smem_b_k)) *
2080         sizeof(half));
2081     LDMATRIX_X2(RB[reg_st_idx][j][0], RB[reg_st_idx][j][1],
2082         lane_smem_b_ptr);
2083 }
2084 }
2085 }
2086
2087 // Epilogue: 复用 RA[2][4][4] 寄存器做 warp shuffle → 128-bit collective store
2088 // 做RC的warp shuffle正好需要[2][4][4]的寄存器空间, RA[2][4][4]正好可以复用
2089 for (int i = 0; i < kValTileM; ++i) {
2090 #pragma unroll
2091     for (int j = 0; j < kValTileN; ++j) {
2092         RA[0][j][0] = RC[i][j][0];
2093         RA[1][j][0] = RC[i][j][1];
2094         RA[0][j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
2095         RA[0][j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
2096         RA[0][j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
2097         RA[1][j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
2098         RA[1][j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
2099         RA[1][j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
2100     }
2101     if (lane_id % 4 == 0) {
2102         int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
2103         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
2104 #pragma unroll
2105         for (int j = 0; j < kValTileN; ++j) {
2106             int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
2107             int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
2108             int store_gmem_c_addr_0 =
2109                 store_lane_gmem_c_m * N + store_lane_gmem_c_n;
2110             int store_gmem_c_addr_1 =
2111                 (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
2112             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
2113                 *reinterpret_cast<float4*>(&RA[0][j][0]);
2114             *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
2115                 *reinterpret_cast<float4*>(&RA[1][j][0]);
2116         }
2117     }
2118 }
2119 }
2120
2121 // =====
2122 // Phase 7c: HGEMM CuTe — CUTLASS CuTe DSL 实现 (SM80+, Tensor Core 全自动调度)
2123 // =====
2124 // 面试要点 (CuTe vs 手写 MMA PTX) :
2125 // - CuTe (CUTLASS Templates) 是 NVIDIA CUTLASS 3.x 的核心 DSL, 提供编译期
2126 //   Tensor 抽象, 自动推导 MMA、Copy、Swizzle 的线程-数据映射
2127 // - 与手写 MMA PTX (Phase 7b) 的对比:
2128 //   - 手写 MMA: 手动计算 smem 地址、手动 ldmatrix/mma PTX、手动 swizzle、手动 epilogue shuffle
2129 //   - CuTe: 声明 TiledMMA / TiledCopy / SmemLayout, 编译器自动推导线程-数据映射,
2130 //     cute::copy / cute::gemm 自动展开为高效指令序列
2131 // - 核心抽象 ("CuTe 五要素") :
2132 //   1. Tensor: 全局/共享/寄存器数据的逻辑视图 = (ptr, shape, stride)
2133 //   2. TiledMMA: 描述 MMA 指令 + warp/warpgroup 排布 + value tile 的 compile-time type
2134 //   3. TiledCopy: 描述数据搬运 (G→S, S→R, R→S, S→G) 的线程-元素映射
2135 //   4. Layout + Swizzle: 描述 smem 的数据排布和 bank conflict 消除
2136 //   5. Pipeline: cp.async + multistage, 由 cute::copy 自动管理
2137 //
2138 // CuTe kernel 的参数推导链 (launch wrapper 负责实例化, kernel 只接收实例化后的类型) :
2139 //   MMA Atom (SM80_16x8x16_F16F16F16F16_TN)
2140 //   → TiledMMA (EURepeat=MxNxK, ValTile=MxNxK)
2141 //   → TiledCopy (G2S/S2R/R2S/S2G, 每种有 ThrLayout + ValLayout)

```

```

2142 //      → SmemLayout (Atom + Swizzle + tile_to_shape + Stage)
2143 //
2144 // 调参口诀（面试速记）：
2145 //      BM/BN/BK 越大 → smem 越大 → occupancy 越低，但 K 循环更少 → 指令开销更低
2146 //      Swizzle<B,M,S> 的核心不是改变逻辑 Tensor 的 shape，而是把一维 offset
2147 //      重新解释为一个二维逻辑空间，再把二维坐标映射回 bank-conflict-free 的物理 offset：
2148 //          1. 连续的  $2^M$  个元素组成二维空间中的一个基本元素；（元素宽度）
2149 //          2. 连续的  $2^S$  个基本元素组成一行；（列数）
2150 //          3. 二维空间包含  $2^B$  行；（行数）
2151 //          4. 对二维坐标做列置换： $icol' = irow \wedge icol$ ；
2152 //          5. 保留基本元素内部的低 M 位，再将置换后的二维坐标编码回 offset。
2153 //      因此 M 描述基本元素宽度，S 描述二维空间的列数，B 描述二维空间的行数。
2154 //      CuTe 的位级实现等价于：
2155 //          offset' = offset ^ ((offset & YYY) >> S),
2156 //          YYY = ((1 << B) - 1) << (M + S)。
2157 //      对 Swizzle<3,3,3>：每个基本元素有  $2^3=8$  个值，每行有  $2^3=8$  个基本元素，
2158 //      二维空间有  $2^3=8$  行；元素地址位 [6:8] XOR 到 [3:5]，低 3 位保持不变。
2159 //      一个完整 swizzle 周期覆盖  $2^{(M+S+B)}=2^9=512$  个元素（FP16 下为 1024B）。
2160 //      kStage: 2=最低延迟，3/4=更好隐藏延迟 → 权衡 smem 占用
2161 //      EURepeat: 在 M/N/K 方向重复 MMA atom 的执行单元布局，决定 TiledMMA 的线程排布
2162 //      与逻辑 tile 组织；MMA atom 越多，需要的线程数就越多
2163 //
2164 // ★ 与手写 MMA Swizzle (Phase 7b-3) 的本质区别：
2165 //      - 手写 swizzle: 在 smem 地址计算时手动 XOR 列索引
2166 //      - CuTe Swizzle: SmemLayout 声明式指定地址位的 XOR pattern, cute::copy 自动应用
2167 //      - CuTe 优势: 类型安全，编译器可在编译期推导映射；布局变更通常只需修改类型定义
2168 //
2169 // ★ 本实现的 Tile 配置：
2170 //      - gmem/smem tile: BM=128, BN=256, BK=32, kStage=2; 这是 A/B tile 的目标 shape。
2171 //      - MMA atom: SM80_16x8x16_F16F16F16F16_TN; EURepeat=2x2x1, MMA_P_T=32x32x16。
2172 //      - TiledMMA 的逻辑 MMA tile 是 32x32x16, G2S/S2R copy 再把它映射到更大的 BMxBN tile;
2173 //      BN=256 不是由“8x2x16”直接推导出的 MMA tile，而是本 wrapper 选择的 B tile 尺寸。
2174 //      - Smem Swizzle<3,3,3>: 512-element address period 的 XOR 重排，针对 ldmatrix
2175 //      的访问模式降低 bank conflict; 512 是 swizzle 周期，不等于 A atom 的逻辑元素数。
2176 //      - Threads:  $\text{size}(\text{MMA}\{\}) = 128$  (4 warps × 2x2 EU repeat)。
2177 //      - Dynamic smem: Stage=2 时 A[128x32] + B[256x32] 共 49,152 bytes (约 48 KiB)。
2178 //
2179 // 参考: LeetCUDA/kernels/hgemm/cutlass/hgemm_mma_stage_tn_cute.cu
2180 // 参考: https://zhuanlan.zhihu.com/p/671419093 (CuTe Swizzle 详解)
2181 // =====
2182 //
2183 #if defined(NOTES_V2_ENABLE_CUTE)
2184 //
2185 #include <cute/tensor.hpp>
2186 //
2187 // =====
2188 // Phase 7c-1: CuTe HGEMM Kernel — TN 布局 + Smem Swizzle + Multistage Pipeline
2189 // =====
2190 // TN 布局:  $C[M \times N] = A[M \times K] \times B^T[N \times K]$ 
2191 //      -  $A[M][K]$  row-major,  $B^T[N][K]$  row-major (等价 B col-major  $[K \times N]$ )
2192 //      - CuTe 中通过 make_tensor(make_gmem_ptr(ptr), shape, stride) 描述
2193 //
2194 // Pipeline 流程 (stage 是 K tile 的循环缓冲槽，不是一条完整计算链)：
2195 //      1. PREFETCH: 预加载 kStage-1 个 K tile (G→S via cp.async)，填满 stage。
2196 //      2. 主循环 over K tiles:
2197 //          a. 从当前 stage 做 S→R: cute::copy(...) → ldmatrix;
2198 //          b. 对当前 K slice 做 MMA: cute::gemm(...) → mma.sync;
2199 //          c. 在处理当前 tile 的首个 MMA slice 时，异步提交下一个 tile 的 G→S;
2200 //          d. 当前 tile 的最后一个 K slice 完成后等待并切换 smem stage。
2201 //      3. Epilogue: R→S (UniversalCopy 写入 C scratchpad) → S→G (128-bit store)。
2202 //
2203 // 面试对比 — 手写 MMA vs CuTe 代码量：
2204 //      手写: ldmatrix PTX 地址计算 + swizzle XOR + mma PTX + epilogue shuffle (~200行)

```

```

2205 // CuTe: partition + copy + gemm (~80行), 其余由 launch wrapper 的类型推导完成
2206 template <typename T,                                // element type (half)
2207         int BM, int BN, int BK,                      // block tile: BM=128, BN=256, BK=32
2208         int kStage,                                  // cp.async pipeline depth (2)
2209         typename TiledMMA,                          // MMA: SM80_16x8x16_F16F16F16_TN, EURepeat=2x2x1
2210         typename G2SCopyA,                          // G→S A: SM80_CP_ASYNC_CACHEGLOBAL<uint128_t>
2211         typename G2SCopyB,                          // G→S B: same as G2SCopyA
2212         typename SmemLayoutA,                       // smem A: Swizzle<3,3,3> + 8×BK atom → (BM,BK,kStage)
2213         typename SmemLayoutB,                       // smem B: same atom → (BN,BK,kStage)
2214         typename SmemLayoutC,                       // C scratchpad: Swizzle<3,3,3> + 32×32 atom, pipe=4
2215         typename S2RCopyAtomA,                     // S→R A: SM75_U32x4_LDSM_N (ldmatrix.x4)
2216         typename S2RCopyAtomB,                     // S→R B: same as S2RCopyAtomA
2217         typename R2SCopyAtomC,                     // R→S C: UniversalCopy<int> (32-bit store)
2218         typename S2GCopyAtomC,                     // S→G C: UniversalCopy<uint128_t> (128-bit wide store)
2219         typename S2GCopyC,                         // S→G TiledCopy: ThrLayout{32,4} × ValLayout{1,8}
2220         const int BlockSwizzle = 0>                // 1 enables 3D grid swizzle for L2 locality
2221 __global__ void hgemm_mma_stages_tn_cute(T *Aptr, T *Bptr, T *Dptr, int m,
2222                                         int n, int k) {
2223     using namespace cute;
2224     static_assert(BlockSwizzle == 0 || BlockSwizzle == 1, "BlockSwizzle must be 0 or 1");
2225     static_assert(kStage >= 2, "kStage must be >= 2 for cp.async pipeline");
2226
2227     // 动态 shared memory: KStage 个 A/B tile; C epilogue 复用当前 A stage 的空间。
2228     extern __shared__ T shm_data[];
2229     T *Ashm = shm_data;
2230     T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2231
2232     int idx = threadIdx.x;
2233     // BlockSwizzle: 0/1 控制是否启用 thread block swizzle, 改善 L2 cache 局部性
2234     int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2235     int iy = blockIdx.y; // M 方向 block 索引
2236     // 当前 kernel 只有 CTA 级边界判断, 没有 tile 内的逐元素 predication; 调用方需保证
2237     // M % BM == 0、N % BN == 0、K % BK == 0, 才能避免边界 tile 的越界访问或未写回。
2238     if (iy * BM >= m || ix * BN >= n) return;
2239
2240     // CuTe Tensor 抽象: (ptr, shape, stride) 三元组描述数据布局
2241     // make_stride(leading_dim, Int<1>{}) → row-major: 同行相邻元素步长=1, 跨行步长=leading_dim
2242     Tensor A = make_tensor(make_gmem_ptr(Aptr), make_shape(m, k),
2243                             make_stride(k, Int<1>{}));
2244     Tensor B = make_tensor(make_gmem_ptr(Bptr), make_shape(n, k),
2245                             make_stride(k, Int<1>{}));
2246     Tensor D = make_tensor(make_gmem_ptr(Dptr), make_shape(m, n),
2247                             make_stride(n, Int<1>{}));
2248
2249     // local_tile: 按 tiler 切出当前 CTA 的逻辑 tile; 返回 Tensor 仍保留未切出的 remainder mode。
2250     // make_coord(iy, _): 固定 M/N 方向的 CTA 坐标, _ 保留 K 方向的 tile remainder:
2251     // gA: (BM, BK, num_tile_k), gB: (BN, BK, num_tile_k), gD: (BM, BN)。
2252     Tensor gA = local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), make_coord(iy, _));
2253     Tensor gB = local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), make_coord(ix, _));
2254     Tensor gD = local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), make_coord(iy, ix));
2255
2256     // Shared memory Tensor: 按 SmemLayout 解读 smem 区域
2257     auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2258     auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2259
2260     // TiledMMA partition: 将 MMA 的 A/B/C tile 映射到各线程的寄存器 fragment
2261     TiledMMA tiled_mma;
2262     auto thr_mma = tiled_mma.get_slice(idx);
2263     auto tCrA = thr_mma.partition_fragment_A(gA(_, _, 0)); // (MMA, MMA_M, MMA_K)
2264     auto tCrB = thr_mma.partition_fragment_B(gB(_, _, 0)); // (MMA, MMA_N, MMA_K)
2265     auto tCrD = thr_mma.partition_fragment_C(gD);           // (MMA, MMA_M, MMA_N)
2266     clear(tCrD); // 累加器清零
2267

```



```

2268 // G2S TiledCopy: 描述 global → shared memory 的数据搬运 (128-bit cp.async)。
2269 G2SCopyA g2s_tiled_copy_a;
2270 auto g2s_thr_copy_a = g2s_tiled_copy_a.get_slice(idx);
2271 auto tAgA_copy = g2s_thr_copy_a.partition_S(gA); // (CPY, CPY_M, CPY_K, num_tile_k)
2272 auto tAsA_copy = g2s_thr_copy_a.partition_D(sA); // (CPY, CPY_M, CPY_K, kStage)
2273
2274 G2SCopyB g2s_tiled_copy_b;
2275 auto g2s_thr_copy_b = g2s_tiled_copy_b.get_slice(idx);
2276 auto tBgB_copy = g2s_thr_copy_b.partition_S(gB); // (CPY, CPY_N, CPY_K, num_tile_k)
2277 auto tBsB_copy = g2s_thr_copy_b.partition_D(sB);
2278
2279 // S2R TiledCopy: 描述 shared → register 的数据搬运 (使用 ldmatrix)
2280 // make_tiled_copy_A/B: 根据 TiledMMA 自动推导 S2R copy 的线程-数据映射
2281 auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2282 auto s2r_thr_copy_a = s2r_tiled_copy_a.get_slice(idx);
2283 auto tAsA = s2r_thr_copy_a.partition_S(sA); // (CPY, CPY_M, CPY_K, kStage)
2284 auto tCrA_view = s2r_thr_copy_a.retile_D(tCrA); // (CPY, CPY_M, CPY_K) — 与 rA 寄存器布局对齐
2285
2286 auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2287 auto s2r_thr_copy_b = s2r_tiled_copy_b.get_slice(idx);
2288 auto tBsB = s2r_thr_copy_b.partition_S(sB);
2289 auto tCrB_view = s2r_thr_copy_b.retile_D(tCrB);
2290
2291 // PREFETCH: 预加载前 (kStage - 1) 个 K tile, 填满循环缓冲; 每次 copy 提交一个异步 G→S 操作。
2292 int itile_to_read = 0, ismem_read = 0, ismem_write = 0;
2293 #pragma unroll
2294 for (int istage = 0; istage < kStage - 1; ++istage) {
2295     cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, istage),
2296               tAsA_copy(_, _, _, istage));
2297     cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, istage),
2298               tBsB_copy(_, _, _, istage));
2299     cp_async_fence();
2300     ++itile_to_read;
2301     ++ismem_write;
2302 }
2303 cp_async_wait<kStage - 2>();
2304 __syncthreads();
2305
2306 // K 维第一层循环 — 按 BK 大小将 K 切分为 num_k_tiles 个 tile。
2307 // 外层 k_tile 遍历这些 BK-tile, 内层 k_step 遍历 tile 内的 MMA_K slice。
2308 // 当前实现使用整除, 要求 K % BK == 0; 没有为尾部 K tile 做 predication/padding。
2309 int num_k_tiles = k / BK;
2310 // 循环前预加载首轮数据: 将第一个 K tile 的第 0 个 K slice 从 smem 加载到寄存器。
2311 cute::copy(s2r_tiled_copy_a, tAsA(_, _, 0, ismem_read), tCrA_view(_, _, 0));
2312 cute::copy(s2r_tiled_copy_b, tBsB(_, _, 0, ismem_read), tCrB_view(_, _, 0));
2313
2314 #pragma unroll 1
2315 for (int k_tile = 0; k_tile < num_k_tiles; ++k_tile) {
2316     // num_k_steps: BK tile 内 K 方向的 MMA 迭代次数, 取自 fragment 的第三 mode (K-mode)。
2317     //
2318     // 为什么只显式展开 K? MMA_M 和 MMA_N 去哪了?
2319     // tCrA=(MMA, MMA_M, MMA_K), tCrB=(MMA, MMA_N, MMA_K), tCrD=(MMA, MMA_M, MMA_N)
2320     // - K 方向: 每个 k_step 需要从 smem 加载**不同的** A/B 数据 (ldmatrix),
2321     //   所以 K 循环必须显式写, 每次迭代做 S→R copy + cute::gemm。
2322     // - M/N 方向: 同一个 k_step 内, 所有 M、N 位置的 MMA atom 共享
2323     //   同一批寄存器数据。cute::gemm 根据 tiled_mma 的类型信息 (EURepeat=2×2)
2324     //   在编译期自动展开为覆盖全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列,
2325     //   无需手动写 M/N 循环。——这等价于手写 MMA 中的:
2326     //   for (i=0; i<kValTileM; ++i)
2327     //       for (j=0; j<kValTileN; ++j)
2328     //           HMMA16816(RC[i][j][0], RC[i][j][1], ...);
2329     //
2330     // CUTLASS 推导链 (mma_atom.hpp) :

```



```

2331 // make_tiled_mma 将 MMA_P_T::<2> (即 kMmaPK) 存入 PermutationMnK
2332 // → TiledMMA::permutation_mnk<2>() 返回 kMmaPK
2333 // → thrfrg_A 对 A(M,K) 按 (permutation_mnk<0>(), permutation_mnk<2>())
2334 // 做 logical_divide, 即按 (kMmaPM, kMmaPK) tile 化 K 维:
2335 // K 被分为 BK/kMmaPK 个 perm-K slice
2336 // 随后按 AtomShape_MNK<2> (MMA_K_atom) 做 zipped_divide
2337 // 每个 perm-K 含 kMmaEURepeatK 个 atom-K slice
2338 // → partition_A 选当前线程后, K-mode size = BK / kMmaPK = 32 / 16 = 2
2339 // 本配置: kMmaPK=16 (MMA_K_atom=16 × kMmaEURepeatK=1), BK=32 → num_k_steps=2
2340 int num_k_steps = size<2>(tCrA);
2341
2342 #pragma unroll
2343 for (int k_step = 0; k_step < num_k_steps; ++k_step) {
2344     int k_step_next = (k_step + 1) % num_k_steps;
2345
2346     if (k_step == num_k_steps - 1) {
2347         // 等待下一个 K tile 的 smem 数据就绪, 然后切换到下一个 stage。
2348         cp_async_wait<kStage - 2>();
2349         __syncthreads();
2350         ismem_read = (ismem_read + 1) % kStage;
2351     }
2352
2353     // S→R: 加载下一组 A/B fragment 到寄存器。
2354     //
2355     // cute::copy 原生支持多 mode——理论上可以一次加载全部 K slice:
2356     // cute::copy(s2r_tiled_copy_a, tAsA(_, _, _, ismem_read), tCrA_view);
2357     // 但这里刻意逐 k_step_next 加载, 目的是做**寄存器双缓冲**::
2358     // 当前 k_step 做 MMA 计算时, ldmatrix 预加载 k_step_next 的数据,
2359     // 让 S→R 延迟与 MMA 计算重叠。
2360     // 如果一次性加载全部 K slice, S→R 和 MMA 就会彻底串行。
2361     cute::copy(s2r_tiled_copy_a, tAsA(_, _, k_step_next, ismem_read),
2362               tCrA_view(_, _, k_step_next));
2363     cute::copy(s2r_tiled_copy_b, tBsB(_, _, k_step_next, ismem_read),
2364               tCrB_view(_, _, k_step_next));
2365
2366     if (k_step == 0) {
2367         // G→S: 提交下一个 K tile (整个 BK) 的异步拷贝。
2368         if (itile_to_read < num_k_tiles) {
2369             cute::copy(g2s_tiled_copy_a, tAgA_copy(_, _, _, itile_to_read),
2370                       tAsA_copy(_, _, _, ismem_write));
2371             cute::copy(g2s_tiled_copy_b, tBgB_copy(_, _, _, itile_to_read),
2372                       tBsB_copy(_, _, _, ismem_write));
2373             ++itile_to_read;
2374             ismem_write = (ismem_write + 1) % kStage;
2375         }
2376         cp_async_fence();
2377     }
2378
2379     // MMA: cute::gemm 内部根据 tiled_mma 的类型信息, 自动展开为覆盖
2380     // 全部 (MMA_M, MMA_N) 对的 mma.sync 指令序列, 累加到 tCrD。
2381     cute::gemm(tiled_mma, tCrD, tCrA(_, _, k_step), tCrB(_, _, k_step), tCrD);
2382 }
2383 }
2384
2385 // Epilogue: D 寄存器 → shared memory → global memory。
2386 // 使用当前 A stage 作为 C scratchpad, 避免为 epilogue 单独分配完整的 C shared memory。
2387 // 这里是“复用 storage”, 不是把 A Tensor 转换成 C Tensor:
2388 // sA 的逻辑 shape 是 (BM, BK, kStage)=(128,32,2), 而 sA(_,_,ismem_read)
2389 // 只选出其中一个 A-stage 的 storage 起点; sC 随后以完全独立的 SmemLayoutC
2390 // 解释这同一段地址。当前 wrapper 的固定配置下:
2391 // 一个 A stage: 128 × 32 = 4096 个 half;
2392 // C scratchpad: 32 × 32 × 4 = 4096 个 half。
2393 // 两者容量刚好相等, 但 logical shape 和 layout 不是同一个: A 的 layout atom

```

```

2394 // 是 8x32, C 的 layout atom 是 32x32, 二者都含 Swizzle<3,3,3>, 却不能把
2395 // C 数据再按 SmemLayoutA 读取。launch wrapper 的 static_assert 用当前配置
2396 // 检查 C scratchpad 能放进一个 A pipe; 此处只别名该单个 stage, 不是整块双缓冲 sA。
2397 //
2398 // mainloop 已结束, 不会再通过 sA 的 A/B load view 消费这个 pipe 的旧 A 数据。
2399 // 从这一行开始, 对这块地址的所有访问都通过 SmemLayoutC 派生的 sC 完成: R2S
2400 // 用它写 C, S2G 用它读 C。因此 C 的 swizzle/address mapping 在写和读两侧一致。
2401 auto sC = make_tensor(sA(_, _, ismem_read).data(), SmemLayoutC{});
2402
2403 // R2S TiledCopy: 寄存器 → shared memory (使用 UniversalCopy 做类型转换)
2404 // R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>; 以 32-bit 粒度搬运 T 数据
2405 auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2406 auto r2s_thr_copy_c = r2s_tiled_copy_c.get_slice(idx);
2407 // tCrD 是 TiledMMA 产生的 accumulator fragment, 逻辑 shape 为
2408 // (MMA, MMA_M, MMA_N)。它的寄存器元素已经就位, 但该 layout 是为 MMA
2409 // accumulator 服务的, 并不直接按 R2S copy atom 的 source-value 顺序表达。
2410 //
2411 // retile_S 中的 S 指“这个 TiledCopy 的 source side”, 不是 shared memory。
2412 // 它基于 r2s_tiled_copy_c 的 reference layout 创建一个共享 tCrD.data() 的
2413 // zero-copy layout view, 将逻辑坐标表示为 (CPY, CPY_M, CPY_N)。这一步不读取
2414 // 或写入寄存器/共享内存, 不进行 dtype conversion, 也不交换 lane 间数据; 真正的
2415 // R2S 数据搬运发生在下面的 cute::copy(r2s_tiled_copy_c, ..., tCsC_r2s(...))。
2416 // 可将它与主循环的 s2r_thr_copy_a.retile_D(tCrA) 对照: 二者都是为了让已有
2417 // fragment 的 per-thread value ordering 匹配某个 TiledCopy 的 source/destination
2418 // 角色, 而不是另一次 memory transfer。
2419 // tCrD: (MMA, MMA_M, MMA_N) -> retile -> tCrC_r2s: (CPY, CPY_M, CPY_N)
2420 auto tCrC_r2s = r2s_thr_copy_c.retile_S(tCrD); // (CPY, CPY_M, CPY_N)
2421 // get_slice(idx) 已固定 CTA 内当前 logical copy thread; partition_D 再按
2422 // R2S TiledCopy 的 destination mapping 切分 sC。推导链:
2423 // MMA_P_T = Tile<kMmaPM, kMmaPN, kMmaPK> = Tile<32, 32, 16>
2424 // → TiledMMA::tile_size<0>(mma) = 32, tile_size<1>(mma) = 32
2425 // → make_tiled_copy_C 的 tiler = (32, 32)
2426 // → SmemLayoutC 的逻辑 M×N = (kMmaPM, kMmaPN) = (32, 32) 恰好等于该 tiler
2427 // → partition_D: zipped_divide(sC, tiler=(32,32)) 后 M/N 维无 remainder
2428 // → 结果 shape 的 M/N remainder = _1, _1
2429 // 最后一维 pipe=4 来自 SmemLayoutC 的第三维 kSmemLayoutCBatch, 而不是
2430 // “thread-level tensor 自动只剩 CPY”这一条规则。_1 表示该逻辑 mode 的 extent
2431 // 是 1, 并非该 mode 被删除或 C tile 没有 M/N 覆盖。
2432 auto tCsC_r2s = r2s_thr_copy_c.partition_D(sC); // (CPY, _1, _1, pipe)
2433
2434 // S2G TiledCopy: shared memory → global memory (128-bit store)
2435 // S2GCopyAtomC(Copy_Atom<UniversalCopy<cute::uint128_t>, T>) -> S2GCopyC
2436 S2GCopyC s2g_tiled_copy_c;
2437 auto s2g_thr_copy_c = s2g_tiled_copy_c.get_slice(idx);
2438 // tCsC_s2g 与 tCsC_r2s 都从同一个 sC 产生, 因而保留相同的 pipe=4; 前者是
2439 // S2G source mapping, 后者是 R2S destination mapping。tCgC_s2g 则面向完整
2440 // gD=(BM,BN)=(128,256), 相对于 S2G copy tiler 仍有非平凡的 M/N repetition,
2441 // 所以它保留 CPY_M、CPY_N。这里的 CPY/CPY_M/CPY_N 都是编译期 copy-partition
2442 // 的逻辑 mode, 不应直接理解为固定的 lane 数、warp 数或物理连续维度。
2443 auto tCsC_s2g = s2g_thr_copy_c.partition_S(sC); // (CPY, _1, _1, pipe)
2444 auto tCgC_s2g = s2g_thr_copy_c.partition_D(gD); // (CPY, CPY_M, CPY_N)
2445
2446 // group_modes<B,E> 将 layout 的半开区间 [B,E) 组合成一个嵌套 mode:
2447 // group_modes<1,3>(Tensor(CPY, CPY_M, CPY_N))
2448 // → Tensor(CPY, (CPY_M, CPY_N))
2449 // 它只重组 Tensor 的 layout, 不分配内存、不移动 data(), 也不改变元素访问的物理地址。
2450 // 第二个 mode 仍然保存原来 CPY_M/CPY_N 的层次关系, 只是现在可以用一个坐标
2451 // i+j 访问这个组合 mode; 因此这里的 group_modes 更准确地说“合并 mode”,
2452 // 而不是把底层数据拷贝或无条件 flatten 成普通一维数组。
2453 // 从 type/shape 结构看, 结果确实是 Tensor(CPY, (CPY_M, CPY_N)); 但 grouped
2454 // mode 的 cardinality 是两个子 mode 的乘积, 因此 size<1>(…) 可作为一个标量
2455 // 范围遍历其全部 logical coordinate。于是 (_, i+j) 中的 i+j 是该 nested mode
2456 // 的线性 logical coordinate, 不是在 C++ 中对二维 tuple 做加法, 更不是 raw

```

```

2457 // pointer offset; 最终物理地址仍由各自 Tensor 的 grouped layout 计算。
2458 //
2459 // 这里为什么要对 tCrC_r2s 和 tCgC_s2g 同时 group?
2460 //   - tCrC_r2s: 每个线程持有的寄存器 C fragment, 原始形状为 (CPY, CPY_M, CPY_N);
2461 //   - tCgC_s2g: 该线程对应的 global-memory C 输出位置, 形状与源 Tensor 对齐;
2462 //   - 两者使用相同的 [1,3] 合并后, 源/目的 Tensor 仍保持相同的坐标空间,
2463 //     cute::copy 可以按相同的 (CPY, i+j) 坐标完成寄存器到 global 的对应搬运。
2464 //
2465 // tCsC_r2s/tCsC_s2g 没有 group 第 3 个 pipe mode, 因为它们还需要显式保留
2466 // shared-memory C scratchpad 的 pipeline 维度; step=size<3>(tCsC_r2s) 表示
2467 // 每一轮可以复用的 C shared-memory pipeline 深度。外层 i 在合并后的 C 元素空间中
2468 // 按 step 前进, 内层 j 选择当前 pipeline slot: 寄存器先写入 sC(_,0,0,j),
2469 // 再从同一个 slot 写回 global memory。
2470 // 当前 kSmemLayoutCBatch=4, 因此 step=4。代码未对 i+j 做尾部 predication,
2471 // 这个循环依赖当前静态 layout 中 grouped extent 能被 pipe depth 整除; 不应将
2472 // “每轮正好处理 4 个 fragment” 误认为任意 tile/copy 配置都自动成立的通用规则。
2473 auto tCgC_s2gx = group_modes<1, 3>(tCgC_s2g);
2474 auto tCrC_r2sx = group_modes<1, 3>(tCrC_r2s);
2475
2476 int step = size<3>(tCsC_r2s); // C scratchpad 的 pipe 数 (由 kSmemLayoutCBatch=4 指定)
2477 // 双层循环的语义 (注意 step 与 CPY_N 无关):
2478 //   group_modes<1,3> 之后, tCrC_r2sx 的 shape 为 (CPY, (CPY_M, CPY_N))。
2479 //   size<1>(tCrC_r2sx) = CPY_M * CPY_N, 即该线程需处理的 C fragment 总数。
2480 //   外循环以 step=4 为步长, 内循环 j=0..3 每次处理 1 个 fragment, 将其写入
2481 //   第 j 号 C scratchpad pipe slot, 再读出写回 global。
2482 //
2483 //   这里 step=4 是 scratchpad pipeline 深度, 不是 CPY_N。CPY_N 的值取决于
2484 //   TiledMMA 的 C-layout 如何将 (128,256) 的 gD tile 分配到 128 个线程上,
2485 //   通常 ≠ 4。循环之所以成立, 是因为 grouped mode 用线性坐标 i+j 遍历整个
2486 //   CPY_M * CPY_N 的扁平空间——它不再区分 M 和 N 方向, 也不要求 CPY_N == step。
2487 //   唯一前提是 CPY_M * CPY_N 能被 step 整除 (当前静态 layout 编译期保证)。
2488 //
2489 //   第 j 号 pipe slot 在 sC 上的物理地址由 R2S partition_D / S2G partition_S
2490 //   各自推导; 因为二者都从同一个 sC (SmemLayoutC) 出发, pipe mode 保持一致,
2491 //   写入和读取命中同一段 shared memory。
2492 #pragma unroll
2493 for (int i = 0; i < size<1>(tCrC_r2sx); i += step) {
2494     // 每轮处理 step 个 fragment, 内层 j 选 pipe slot。
2495 #pragma unroll
2496     for (int j = 0; j < step; ++j) {
2497         // 这两行是 R2R staging: make_tensor_like<T> 分配当前线程私有、拥有 storage
2498         // 的 register Tensor; 普通 cute::copy 将 accumulator fragment materialize
2499         // 成输出元素类型 T 的临时 payload。
2500         //
2501         // cute::copy (无 copy-atom 的普通版) 内部按元素做
2502         //   dst(i) = static_cast<T>(static_cast<SrcType>(src(i)))
2503         // 因此当 accumulator 与 T 类型不同时, 它自动完成 dtype 转换; 当二者相同
2504         // (如本 kernel 的 f16 accumulator + T=half), cast 退化为 no-op。
2505         //
2506         // 即使类型一致, t 还有一个不可省略的作用: layout 归一化。
2507         // tCrC_r2sx(_, i+j) 的 layout 是 retile_S → group_modes → slice 层层
2508         // 叠加的 composed layout, 而 make_tensor_like<T> + cute::copy 把它
2509         // 物化到一个拥有简单 strides 的 owning register Tensor 中。后续
2510         // r2s_tiled_copy_c 的 copy_unpack 需要按 copy-atom 的 val-layout
2511         // 拆分 source 元素, 简单 owning layout 比复杂的 composed layout 更容易
2512         // 被编译器推导内联。上游同源实现将其概括为"cope with accumulator and
2513         // output data type difference", 实际承担了类型适配和 layout 归一化两个职责。
2514         //
2515         // 这不是 retile_S 的一部分, 也不是 warp shuffle: 源/目的都在当前线程的
2516         // register 中, 代码没有显式 __shfl_sync。不能仅根据这段 C++ 断言最终 SASS
2517         // 绝不会含 shuffle; 但语义上的跨线程 regrouping 不由此 R2R copy 承担, 而是
2518         // 由随后的 R2S → shared memory → S2G 路径完成。若特定 accumulator/output
2519         // type 与 R2S copy-atom contract 已直接兼容, 可以设计直接 R2S 的变体; 本例

```

```

2520 // 保持同源实现的通用 staging 写法。
2521 auto t = make_tensor_like<T>(tCrC_r2sx(_, i + j));
2522 cute::copy(tCrC_r2sx(_, i + j), t);
2523 // R2S 的 copy atom 才在此处将 thread-local payload 写入 j 号 C scratchpad
2524 // slot; SmemLayoutC 定义写入后的跨线程地址重组。
2525 //
2526 // cute::copy(r2s_tiled_copy_c, src, dst) 的调用链路:
2527 //   r2s_tiled_copy_c 是 TiledCopy, 但继承自 Copy_Atom<UniversalCopy<int>,T>
2528 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2529 //   → src(t) 与 dst(tCsC_r2s(_,0,0,j)) 都是 rank-1 → 直接 copy_atom.call
2530 //   → copy_atom.call 内部: 若 size(src)==NumValSrc (atom 编译期 val-count),
2531 //     走 copy_unpack; 否则递归剥 mode, 最终都落到 UniversalCopy<int>::copy
2532 //     → 硬件层面就是逐 uint32_t 的 register-to-smem store [arch/copy.hpp:~L46]
2533 //
2534 // 这里没有任何运行时"查找表": t 的第 i 个元素之所以对应 dst 的第 i 个
2535 // shared memory offset, 是因为 pre-partition 阶段已经把映射烧进 layout 了:
2536 //   - t 的 layout 来自 retile_S → group_modes → slice → make_tensor_like
2537 //     → 元素顺序已按 R2S atom 的 source-side value ordering 排好
2538 //   - tCsC_r2s 的 layout 来自 r2s_thr_copy_c.partition_D(sC)
2539 //     → TiledCopy::tidfrg_D 用 LayoutCopy_TV + Tiler_MN + ValLayoutDst
2540 //     计算了当前线程每个 val 在 smem 中的物理 offset
2541 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2542 // 因此逐元素 copy 天然把正确的数据写到了正确的地址。
2543 cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2544 }
2545 // 这是 CTA 范围的 rendezvous: 所有线程完成当前批 R2S 写入后, S2G 才能从
2546 // sC 的重组布局读取完整数据。它承担了手写版中显式跨 lane 拼接所需的协作边界。
2547 __syncthreads();
2548
2549 // S→G: 从同一个 pipe slot 读回 global memory, 保持源/目的坐标一一对应。
2550 #pragma unroll
2551 for (int j = 0; j < step; ++j) {
2552 // S2GCopyC 的 atom 为 UniversalCopy<uint128_t>: 与 R2S 的 UniversalCopy<int>
2553 // (32-bit) 不同, 它一次搬运 128-bit (8 个 half), 映射为一条 wide store
2554 // (如 st.global.v4.f32 或 st.global.b128)。
2555 //
2556 // cute::copy(s2g_tiled_copy_c, src, dst) 的调用链路:
2557 //   s2g_tiled_copy_c 是 TiledCopy, 继承自 Copy_Atom<UniversalCopy<uint128_t>,T>
2558 //   → 匹配 copy(Copy_Atom<...> const&, src, dst) 重载 [copy.hpp:~L190]
2559 //   → src=tCsC_s2g(_,0,0,j) 与 dst=tCgC_s2gx(_,i+j) 都是 rank-1
2560 //   → 直接 copy_atom.call(src, dst)
2561 //   → copy_atom.call 内部: size(src)==NumValSrc → copy_unpack
2562 //   → 逐 128-bit chunk 调用 UniversalCopy<uint128_t>::copy
2563 //   → 硬件层面就是 shared-memory-to-global wide store [arch/copy.hpp:~L46]
2564 //
2565 // S2G 的 pre-partition 与 R2S 对称, 但 source/destination 角色互换:
2566 //   - tCsC_s2g 来自 s2g_thr_copy_c.partition_S(sC): TiledCopy::tidfrg_S
2567 //     用 LayoutCopy_TV + Tiler_MN + ValLayoutSrc 计算了当前线程每个 val
2568 //     在 sC (shared memory) 中的物理 offset。
2569 //   S2GCopyC 的 tiler = product(ThrLayout{32,4}, ValLayout{1,8})
2570 //     = (32×1, 4×8) = (32, 32) — 恰好与 R2S tiler 相同,
2571 //   因此 tCsC_s2g 也是 (CPY, _1, _1, pipe), _1,_1 来自 sC=(32,32,4) 无
2572 //   M/N remainder。
2573 //   - tCgC_s2gx(_, i+j) 来自 s2g_thr_copy_c.partition_D(gD) → group_modes
2574 //     → slice。gD=(128,256) 相对于 tiler (32,32) 有 4×8 个 tile repetition,
2575 //     因此 tCgC_s2g 为 (CPY, CPY_M, CPY_N), group_modes<1,3> 合并 M/N
2576 //     repetition 后得到 (CPY, (CPY_M,CPY_N)), 再用线性坐标 i+j 切片。
2577 // 两者共享同一个 copy atom 的 ValLayoutSrc/ValLayoutDst 定义,
2578 // 因此逐 chunk copy 天然从正确的 smem 地址读到正确的 gmem 地址。
2579 cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i + j));
2580 }
2581 // 所有线程都完成当前 pipe slots 的 S2G 读取后, 下一轮 R2S 才能覆盖这 4 个
2582 // scratchpad slots, 避免一部分线程仍在读取旧数据时被其他线程提前重写。

```

```

2583     __syncthreads();
2584 }
2585
2586 // 与 hgemm_mma_stages_tn 的手写 epilogue 对照：手写版必须显式处理 MMA fragment
2587 // 的物理分布，使用 RC0/RC1 暂存，再用 __shfl_sync 从同一 warp 的相邻 lane 收齐
2588 // 8 个 half，并由 lane_id % 4 == 0 的线程做 float4 (128-bit) global store。
2589 // CuTe 版没有把这些 lane 映射写在 kernel 源码中：retile_S 表达 accumulator
2590 // fragment 与 R2S copy 的对应，R2S/sC/S2G 通过 shared-memory scratchpad 完成
2591 // CTA 范围的数据重组，S2GCopyC 以 uint128_t 表达 wide store。两者的共同目标都是
2592 // 将分散在计算线程寄存器中的 C fragment 变为适合连续 global-memory 写回的布局；
2593 // 区别是手写版直接编排 shuffle/store，CuTe 版将映射交给 TiledMMA/TiledCopy 类型推导。
2594 }
2595
2596 // =====
2597 // Phase 7c-2: CuTe HGEMM Launch Wrapper — 类型实例化 + Grid/Block 配置
2598 // =====
2599 // CuTe 的核心设计理念：“类型即配置” (Type-level configuration)
2600 // 所有的 MMA atom、TiledCopy、SmemLayout、Swizzle 都是 **编译期类型**，
2601 // kernel 接收这些类型的实例（通常为空 struct），在编译期完成全部映射推导。
2602 //
2603 // 以下每个类型定义后面都标注了它在 kernel body 中的对应变量/用法，形成 1:1 对照。
2604 //
2605 // 面试常问：「CuTe 为什么比手写 PTX 简洁？」
2606 // 答：手写需要：
2607 //   1) 手动计算每线程的 smem 地址偏移
2608 //   2) 手动计算 ldmatrix 的 lane 映射
2609 //   3) 手动插入 swizzle XOR 到每个地址计算点
2610 //   4) 手动做 epilogue 的 warp shuffle 编排
2611 // CuTe 的 partition + copy + gemm 通过 TiledCopy/TiledMMA 的类型信息
2612 // 在编译期自动完成上述全部推导，生成与手写等价的 PTX 指令序列。
2613 // =====
2614 template <typename T, const int Stages = 2, const int BlockSwizzle = 0>
2615 void launch_hgemm_mma_stages_tn_cute(T *a, T *b, T *c, int M, int N, int K) {
2616     using namespace cute;
2617
2618     // — Tile 尺寸 (kernel 模板参数 BM/BN/BK/kStage 的值) —
2619     //
2620     // kernel 用法对照：
2621     //   BM=128:  local_tile(A, make_tile(Int<BM>{}, Int<BK>{}), ...) → gA=(BM,BK,num_k_tiles)
2622     //             local_tile(D, make_tile(Int<BM>{}, Int<BN>{}), ...) → gD=(BM,BN)
2623     //   BN=256:  local_tile(B, make_tile(Int<BN>{}, Int<BK>{}), ...) → gB=(BN,BK,num_k_tiles)
2624     //             local_tile(D, ...) → gD 的列方向
2625     //   BK=32:   num_k_tiles = k / BK; 每个 BK tile 内 num_k_steps = BK/kMmaPK = 2 个 MMA_K slice
2626     //   kStage=2: sA/sB 的第三维 = (BM,BK,kStage); cp_async_wait<kStage-2>() 流水线同步
2627     //   kSmemLayoutCBatch=4: step = size<3>(tCsC_r2s) → C scratchpad pipe 深度 = 4
2628     auto BM = Int<128>{};
2629     auto BN = Int<256>{};
2630     auto BK = Int<32>{};
2631     auto kStage = Int<Stages>{};
2632     auto kSmemLayoutCBatch = Int<4>{};
2633
2634     // — SmemLayoutA / SmemLayoutB —
2635     // kernel 用法：
2636     // auto sA = make_tensor(make_smem_ptr(Ashm), SmemLayoutA{}); // (BM, BK, kStage)
2637     // auto sB = make_tensor(make_smem_ptr(Bshm), SmemLayoutB{}); // (BN, BK, kStage)
2638     // SmemLayout 由三部分组成：Swizzle<3,3,3> + atom layout(8×BK) + tile_to_shape 扩展到完整 tile+stage。
2639     //
2640     // 对当前 base layout shape=(8,32), stride=(32,1), T=half 的例子：
2641     //   - M=3: 连续 2^3=8 个 half 组成一个基本元素，即 16 bytes；
2642     //   - S=3: 连续 2^3=8 个基本元素组成一行，即 128 bytes；
2643     //   - B=3: 共有 2^3=8 行。
2644     // 于是一个 8×32 的逻辑 tile 正好对应 8 行 × 128B，覆盖全部 32 个 shared-memory bank。
2645     // Swizzle 对每个基本元素的列坐标执行 icol' = irow ^ icol，

```



```

2646 // 再将 (irow, icol') 和基本元素内部的 3 个低位编码回物理 offset。
2647 // 其位级形式为 offset' = offset ^ ((offset & (0b111 << 6)) >> 3):
2648 // 地址位 [6:8] XOR 到 [3:5], 地址位 [0:2] 不参与置换。
2649 // composition(Swizzle, base_layout) 的语义是 R(c) = Swizzle(base_layout(c)):
2650 // 逻辑坐标仍由 base_layout 给出, 只有最终的 shared-memory offset 被重排。
2651 // tile_to_shape: 通过 blocked product 重复这个带 swizzle 的 block layout,
2652 // 使结果 shape 匹配 BM×BK×kStage; 默认按目标 shape 的 mode order 重复各维。
2653 using SmemLayoutAtom = decltype(composition(
2654     Swizzle<3, 3, 3>{},
2655     make_layout(make_shape(Int<8>{}, Int<BK>{}),
2656         make_stride(Int<BK>{}, Int<1>{}))));
2657 using SmemLayoutA = decltype(tile_to_shape(
2658     SmemLayoutAtom{}, make_shape(Int<BM>{}, Int<BK>{}, Int<KStage>{})));
2659 using SmemLayoutB = decltype(tile_to_shape(
2660     SmemLayoutAtom{}, make_shape(Int<BN>{}, Int<BK>{}, Int<KStage>{})));
2661
2662 // — MMA (TiledMMA) —
2663 // kernel 用法:
2664 // TiledMMA tiled_mma;
2665 // auto thr_mma = tiled_mma.get_slice(threadIdx.x); // 每线程的 MMA slice
2666 // auto tCrA = thr_mma.partition_fragment_A(gA(_,_,0)); // (MMA, MMA_M, MMA_K)
2667 // auto tCrB = thr_mma.partition_fragment_B(gB(_,_,0)); // (MMA, MMA_N, MMA_K)
2668 // auto tCrD = thr_mma.partition_fragment_C(gD); // (MMA, MMA_M, MMA_N)
2669 // cute::gemm(tiled_mma, tCrD, tCrA(_,_,k_step), tCrB(_,_,k_step), tCrD);
2670 // MMA 还决定 S2R/R2S copy 的 tiler: make_tiled_copy_A/B/C 都依赖 tiled_mma 的
2671 // tile_size<0/1>(mma) = (32,32) 来推导线程→数据映射。
2672 //
2673 // 推导链: SM80_16x8x16_F16F16F16F16_TN → MMA_Atom
2674 // → make_tiled_mma(atom, EURepeat{2,2,1}, ValTile{32,32,16})
2675 // → TiledMMA: 128 threads = 4 warps × (2×2 EU slices), 逻辑 MMA tile = 32×32×16
2676 // TN = A row-major, B col-major; 因此传给本 kernel 的 B 指针实际指向
2677 // B^T[N,K] 的 row-major 存储 (等价于 GEMM 语义中的 B[K,N] col-major)。
2678 using mma_op = SM80_16x8x16_F16F16F16F16_TN;
2679 using mma_traits = MMA_Traits<mma_op>;
2680 using mma_atom = MMA_Atom<mma_traits>;
2681 using mma_atom_shape = mma_traits::Shape_MNK; // (Int<16>, Int<8>, Int<16>)
2682
2683 static constexpr int kMmaEURepeatM = 2;
2684 static constexpr int kMmaEURepeatN = 2;
2685 static constexpr int kMmaEURepeatK = 1;
2686 static constexpr int kMmaPM = 1 * kMmaEURepeatM * get<0>(mma_atom_shape{}); // 32
2687 static constexpr int kMmaPN = 2 * kMmaEURepeatN * get<1>(mma_atom_shape{}); // 32
2688 static constexpr int kMmaPK = 1 * kMmaEURepeatK * get<2>(mma_atom_shape{}); // 16
2689 // kMmaPK=16 → kernel 中 num_k_steps = size<2>(tCrA) = BK/kMmaPK = 32/16 = 2
2690
2691 using MMA_EU_RepeatT = decltype(make_layout(make_shape(
2692     Int<kMmaEURepeatM>{}, Int<kMmaEURepeatN>{}, Int<kMmaEURepeatK>{})));
2693 using MMA_P_T = Tile<Int<kMmaPM>, Int<kMmaPN>, Int<kMmaPK>>;
2694 using MMA = decltype(make_tiled_mma(mma_atom{}, MMA_EU_RepeatT{}, MMA_P_T{}));
2695
2696 // — G2SCopyA / G2SCopyB —
2697 // kernel 用法:
2698 // G2SCopyA g2s_tiled_copy_a; G2SCopyB g2s_tiled_copy_b;
2699 // cute::copy(g2s_tiled_copy_a, tAgA_copy(_,_,_,istage), tAsA_copy(_,_,_,istage));
2700 // cute::copy(g2s_tiled_copy_b, tBgB_copy(_,_,_,istage), tBsB_copy(_,_,_,istage));
2701 // G→S: 128-bit cp.async. ThrLayout{32,4} × ValLayout{1,8}:
2702 // 128 个线程, 每线程搬运 1×8=8 个 half = 128 bits。
2703 using g2s_copy_op = SM80_CP_ASYNC_CACHEGLOBAL<cute::uint128_t>;
2704 using g2s_copy_traits = Copy_Traits<g2s_copy_op>;
2705 using g2s_copy_atom = Copy_Atom<g2s_copy_traits, T>;
2706 using G2SCopyA = decltype(make_tiled_copy(
2707     g2s_copy_atom{},
2708     make_layout(make_shape(Int<32>{}, Int<4>{}),

```

```

2709         make_stride(Int<4>{}, Int<1>{})),
2710         make_layout(make_shape(Int<1>{}, Int<8>{}))));
2711 using G2SCopyB = G2SCopyA;
2712
2713 // — S2RCopyAtomA / S2RCopyAtomB —
2714 // kernel 用法:
2715 // auto s2r_tiled_copy_a = make_tiled_copy_A(S2RCopyAtomA{}, tiled_mma);
2716 // auto s2r_tiled_copy_b = make_tiled_copy_B(S2RCopyAtomB{}, tiled_mma);
2717 // cute::copy(s2r_tiled_copy_a, tAsA(_,_,k_step_next,ismem_read), tCrA_view(_,_,k_step_next));
2718 // cute::copy(s2r_tiled_copy_b, tBsB(_,_,k_step_next,ismem_read), tCrB_view(_,_,k_step_next));
2719 // S→R: ldmatrix (SM75_U32x4_LDSM_N)。make_tiled_copy_A/B 根据 TiledMMA 自动推导
2720 // 与 MMA fragment 对齐的 shared-register 映射, 无需手动指定 ThrLayout/ValLayout。
2721 using s2r_copy_op = SM75_U32x4_LDSM_N;
2722 using s2r_copy_traits = Copy_Traits<s2r_copy_op>;
2723 using s2r_copy_atom = Copy_Atom<s2r_copy_traits, T>;
2724 using S2RCopyAtomA = s2r_copy_atom;
2725 using S2RCopyAtomB = s2r_copy_atom;
2726
2727 // — SmemLayoutC —
2728 // kernel 用法:
2729 // auto sC = make_tensor(sA(_,_,ismem_read).data(), SmemLayoutC{});
2730 // 复用当前 A stage 的空间作为 C scratchpad
2731 // sC shape = (kMmaPM, kMmaPN, kSmemLayoutCBatch) = (32, 32, 4)
2732 // pipe 数 4 由 kSmemLayoutCBatch 指定 → step = size<3>(tCsC_r2s) = 4
2733 // 与 A/B 相同的 Swizzle<3,3,3> 规则, 但 base layout 是 32×32 (MMA tile 大小),
2734 // 再扩展为 4 个 epilogue scratchpad pipe slots。这 4 个 slots 不等同于主循环
2735 // 的 kStage K-tile pipeline。
2736 using SmemLayoutAtomC = decltype(composition(
2737     Swizzle<3, 3, 3>{},
2738     make_layout(make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}),
2739         make_stride(Int<kMmaPN>{}, Int<1>{}))));
2740 using SmemLayoutC = decltype(tile_to_shape(
2741     SmemLayoutAtomC{},
2742     make_shape(Int<kMmaPM>{}, Int<kMmaPN>{}, Int<kSmemLayoutCBatch>{})));
2743
2744 // — R2SCopyAtomC —
2745 // kernel 用法:
2746 // auto r2s_tiled_copy_c = make_tiled_copy_C(R2SCopyAtomC{}, tiled_mma);
2747 // cute::copy(r2s_tiled_copy_c, t, tCsC_r2s(_, 0, 0, j));
2748 // R→S: UniversalCopy<int> 以 32-bit 粒度将寄存器 payload 写入 C scratchpad。
2749 // make_tiled_copy_C 从 TiledMMA 的 tile_size<0/1>(mma)=(32,32) 推导 tiler,
2750 // 因此 R2S copy 的 tiler = (32,32), 与 SmemLayoutC 的 (32,32) 恰好匹配。
2751 using R2SCopyAtomC = Copy_Atom<UniversalCopy<int>, T>;
2752
2753 // — S2GCopyC —
2754 // kernel 用法:
2755 // S2GCopyC s2g_tiled_copy_c;
2756 // cute::copy(s2g_tiled_copy_c, tCsC_s2g(_, 0, 0, j), tCgC_s2gx(_, i+j));
2757 // S→G: UniversalCopy<uint128_t> 以 128-bit 粒度做 shared-global wide store。
2758 // ThrLayout{32,4} × ValLayout{1,8} → tiler = product((32,4),(1,8)) = (32,32),
2759 // 与 R2S tiler 相同, 因此 partition_S(sC) 得到的 pipe 维度一致。
2760 using S2GCopyAtomC = Copy_Atom<UniversalCopy<uint128_t>, T>;
2761 using S2GCopyC = decltype(make_tiled_copy(
2762     S2GCopyAtomC{},
2763     make_layout(make_shape(Int<32>{}, Int<4>{}),
2764         make_stride(Int<4>{}, Int<1>{})),
2765     make_layout(make_shape(Int<1>{}, Int<8>{}))));
2766
2767 // — Grid/Block 配置 —
2768 // kernel 用法:
2769 // int idx = threadIdx.x; // 0..127
2770 // int ix = ((int)BlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
2771 // int iy = blockIdx.y; // M 方向 CTA 坐标

```



```

2772 // if (iy * BM >= m || ix * BN >= n) return; // CTA 级边界保护
2773 // 由于 kernel 没有 tile 内的逐元素 predication, 调用方需保证
2774 // M % BM == 0, N % BN == 0, K % BK == 0。
2775 // 当不启用 BlockSwizzle 时, BZ=1 退化为纯 2D grid, 行为不变。
2776 constexpr int kSwizzleStride = 2048;
2777 int BX = (N + BN - 1) / BN;
2778 int BY = (M + BM - 1) / BM;
2779 int BZ = BlockSwizzle ? (N + kSwizzleStride - 1) / kSwizzleStride : 1;
2780 BX = BlockSwizzle ? (BX + BZ - 1) / BZ : BX;
2781 dim3 block(size(MMA{})); // 128 threads (= size(TiledMMA))
2782 dim3 grid(BX, BY, BZ);
2783
2784 // — Dynamic Shared Memory 大小 —
2785 // kernel 用法:
2786 // extern __shared__ T shm_data[];
2787 // T *Ashm = shm_data;
2788 // T *Bshm = shm_data + cute::cosize(SmemLayoutA{});
2789 // cosize(SmemLayoutA) = BM*BK*kStage = 128*32*2 = 8192 个 T
2790 // cosize(SmemLayoutB) = BN*BK*kStage = 256*32*2 = 16384 个 T
2791 // 合计 A+B = 24576 个 T; C epilogue 复用 A stage (128*32 = 4096 个 T)。
2792 // static_assert 保证 C scratchpad (kMmaPM*kMmaPN*kSmemLayoutCBatch = 32*32*4 = 4096)
2793 // 能放进一个 A stage (128*32 = 4096)。
2794 static constexpr int shm_size_AB =
2795     cute::cosize(SmemLayoutA{}) + cute::cosize(SmemLayoutB{});
2796 static constexpr int shm_size_C = cute::cosize(SmemLayoutC{});
2797 static_assert(size<0>(SmemLayoutA{}) * size<1>(SmemLayoutA{}) >= size(SmemLayoutC{}),
2798     "C shared memory must fit within one A pipe");
2799 static constexpr int kShmSize =
2800     cute::max(shm_size_AB, shm_size_C) * sizeof(T);
2801
2802 cudaFuncSetAttribute(
2803     hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2804         SmemLayoutA, SmemLayoutB, SmemLayoutC,
2805         S2RCopyAtomA, S2RCopyAtomB, R2SCopyAtomC,
2806         S2GCopyAtomC, S2GCopyC, BlockSwizzle>,
2807     cudaFuncAttributeMaxDynamicSharedMemorySize, kShmSize);
2808
2809 hgemm_mma_stages_tn_cute<T, BM, BN, BK, KStage, MMA, G2SCopyA, G2SCopyB,
2810     SmemLayoutA, SmemLayoutB, SmemLayoutC, S2RCopyAtomA,
2811     S2RCopyAtomB, R2SCopyAtomC, S2GCopyAtomC, S2GCopyC,
2812     BlockSwizzle>
2813     <<<grid, block, kShmSize>>>(a, b, c, M, N, K);
2814 }
2815
2816 #endif /* NOTES_V2_ENABLE_CUTE */
2817
2818 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
2819 // CUDA 13.2 promotes these TMA operations to cuda::ptx. Keep the older
2820 // experimental wrappers so the Hopper path remains buildable on prior toolkits.
2821 __device__ __forceinline__ void tma_fence_proxy_async_shared_cta() {
2822     #if CUDART_VERSION >= 13020
2823         cuda::ptx::fence_proxy_async(cuda::ptx::space_shared);
2824     #else
2825         cuda::device::experimental::fence_proxy_async_shared_cta();
2826     #endif
2827 }
2828
2829 __device__ __forceinline__ void tma_load_2d(
2830     void *dst, const CUTensorMap *tensor_map, int minor_coord, int major_coord,
2831     cuda::barrier<cuda::thread_scope_block> &barrier) {
2832     #if CUDART_VERSION >= 13020
2833         const int32_t coords[]{minor_coord, major_coord};
2834         auto *barrier_handle = cuda::device::barrier_native_handle(barrier);

```

```

2835     cuda::ptx::cp_async_bulk_tensor(cuda::ptx::space_cluster,
2836                                     cuda::ptx::space_global, dst, tensor_map,
2837                                     coords, barrier_handle);
2838 #else
2839     cuda::device::experimental::cp_async_bulk_tensor_2d_global_to_shared(
2840         dst, tensor_map, minor_coord, major_coord, barrier);
2841 #endif
2842 }
2843
2844 __device__ __forceinline__ void tma_arrive_expect_tx(
2845     cuda::barrier<cuda::thread_scope_block> &barrier, uint32_t bytes) {
2846 #if CUDART_VERSION >= 13020
2847     auto *barrier_handle = cuda::device::barrier_native_handle(barrier);
2848     [[maybe_unused]] auto token = cuda::ptx::mbarrier_arrive_expect_tx(
2849         cuda::ptx::sem_release, cuda::ptx::scope_cta, cuda::ptx::space_shared,
2850         barrier_handle, bytes);
2851 #else
2852     [[maybe_unused]] auto token =
2853         cuda::device::barrier_arrive_tx(barrier, 1, bytes);
2854 #endif
2855 }
2856 #endif
2857
2858 #if defined(NOTES_V2_ENABLE_WGMMA)
2859 // =====
2860 // Phase 7d: HGEMM WGMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
2861 // =====
2862 // 面试要点 (WGMMA vs MMA 对比) :
2863 //   - MMA: warp 级 (32 threads) , 同步执行
2864 //   - WGMMA: warpgroup 级 (128 threads = 4 warps) , 异步执行 (fire-and-forget)
2865 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA m16n8k16的 4×16=64倍)
2866 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
2867 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMA, 通过 barrier 同步
2868 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
2869
2870 // ---- WGMMA 辅助函数 ----
2871 // wgmma.commit_group / wait_group 语义 (PTX ISA §9.7.15.7) :
2872 //   - commit_group: 将此前所有未提交的 wgmma.mma_async 归入一个新的 wgmma-group
2873 //     (per-warpgroup, 故需 .sync.aligned 确保所有线程同步执行) 。
2874 //   - wait_group N: 阻塞直到最多 N 个 wgmma-group 尚未完成 (pending ≤ N) 。
2875 //     N=0 → 等所有 group 完成。★ 与 cp.async.wait_group 语义一致 (PTX ISA §9.7.9.25.3.3) ,
2876 //     都是 "wait until only N or fewer groups are pending"。
2877 #define WGMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
2878 #define WGMMA_COMMIT_GROUP() \
2879     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
2880 #define WGMMA_WAIT_GROUP(n) \
2881     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :: "n"(n) : "memory")
2882
2883 // SMEM_DESC_ENCODE: 将 byte 偏移编码为 WGMMA descriptor 位域中的 14-bit 值。
2884 // 编码规则 (PTX ISA §9.7.15.5.1.2.2) :
2885 //   encoded = (x & 0x3FFFF) >> 4
2886 // 其中 0x3FFFF = 2^18 - 1 = 262143, 只保留低 18 bit。
2887 // >>4 是因为 shared memory 地址/偏移必须 16B 对齐 (2^4) , 低 4 bit 恒为 0,
2888 // 可省略以扩大可表示范围。
2889 // 解码: decoded = encoded << 4 (回到原始 byte 值) 。
2890 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4
2891
2892 // make_smem_desc: 构造 WGMMA (warpgroup MMA) 的 64-bit shared memory 矩阵描述符。
2893 // 硬件 WGMMA (如 wgmma.mma_async.m64n128k16) 需要 descA/descB 两个 64-bit 描述符,
2894 // 来定位 shared memory 中的 A/B tile 并告知 swizzle 模式。
2895 //
2896 // 64-bit descriptor 位域布局 (参见 PTX ISA §9.7.15.5.1.2.2 & CUTLASS GmmaDescriptor) :
2897 // -----

```

```

2898 // | 位域 | 范围 | 大小 | 含义
2899 // |-----|-----|-----|-----
2900 // | start_address | [0, 14) | 14 | smem 基址编码
2901 // | (unused) | [14, 16) | 2 | -
2902 // | leading_byte_offset | [16, 30) | 14 | 主要维度 (K) 的字节步长; swizzle 下硬件未使用 (assumed=1)
2903 // | (unused) | [30, 32) | 2 | -
2904 // | stride_byte_offset | [32, 46) | 14 | 跨越维度 (M/N) 的字节步长: K-Major 下为 8-row stripe 间距
2905 // | (unused) | [46, 49) | 3 | -
2906 // | base_offset | [49, 52) | 3 | swizzle pattern 偏移
2907 // | (unused) | [52, 62) | 10 | -
2908 // | layout_type | [62, 64) | 2 | swizzle 模式
2909 // -----
2910 // layout_type: 0=None, 1=128B swizzle, 2=64B swizzle, 3=32B swizzle
2911 //
2912 // K-Major + 128B swizzle + half(16-bit) 的几何推导 (PTX ISA §9.7.15.5.1.2) :
2913 // - 128B swizzle atom 在 128-bit 单位下为 8×8
2914 // - 归一化到 half(2B) 后: atom 覆盖 8×(128/16)=64 个 K 方向元素 × 8 个 M 方向元素
2915 // - 即每个 atom = 64(K) × 8(M) 个 half = 64×8×2 = 1024 bytes
2916 // - 这是 stride_byte_offset=1024 的来源
2917 //
2918 // - leading_byte_offset 在 K-Major swizzle 下 unused (hardware assumes 1) ,
2919 // 此处填 16 仅为占位, 编码后为 1。
2920 //
2921 // - base_offset=0 (bits [49,52) 未显式置位) , 表示 smem ptr 已 1024B 对齐。
2922 // 若未对齐需计算 (pattern_start_addr >> 7) & 0x7。
2923 //
2924 // 参考: CUTLASS GmmaDescriptor (cute/arch/mma_sm90_desc.hpp)
2925 // 注: descA/descB 共用此函数; 对 B 而言物理存储为 [BN, BK], 详见 WgmmaSMem 注释。
2926 __device__ inline uint64_t make_smem_desc(half *ptr) {
2927 // __cvta_generic_to_shared: 将通用地址空间中的指针转换为 shared memory 地址
2928 // (一个 32-bit 的 smem byte 偏移量, 相对于当前 CTA 的 shared memory 基址)。
2929 uint32_t addr = static_cast<uint32_t>(__cvta_generic_to_shared(ptr));
2930
2931 // 从全零开始: 所有位域初始化为 0。
2932 uint64_t desc = 0x0000000000000000;
2933
2934 // — bits [0, 14): start_address —
2935 // SMEM_DESC_ENCODE(addr) 将 addr 右移 4 位 (丢弃低 4 个 0 bit) ,
2936 // 只保留 14 位。硬件解码时左移 4 位恢复完整的 16B-aligned 地址。
2937 // 可寻址范围: 2^(14+4) = 2^18 = 256K 个 half = 512 KB 共享内存。
2938 desc |= SMEM_DESC_ENCODE(addr);
2939
2940 // — bits [16, 30): leading_byte_offset —
2941 // 原始值 16 (字节), 编码后为 (16 & 0x3FFFF) >> 4 = 1。
2942 // << 16 将编码值放到 bits [16, 30)。
2943 // K-Major + 128B swizzle 下该字段硬件未使用 (assumed to be 1, 参见 PTX ISA §9.7.15.5.1.2.1.1) ,
2944 // 此处填 16 仅为占位, 使编码值为 1 满足硬件预期。
2945 // 若为 MN-Major 或 INTERLEAVE 布局, LBO 有实际含义:
2946 // 对 INTERLEAVE: 同一 8×2 brick 内第一列到第二列的字节偏移。
2947 // 对 MN-Major swizzle: 偏移从首 (swizzle-byte-size/16) 行到下一组行。
2948 desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16;
2949
2950 // — bits [32, 46): stride_byte_offset —
2951 // 原始值 1024 (字节), 编码后为 (1024 & 0x3FFFF) >> 4 = 64。
2952 // << 32 将编码值放到 bits [32, 46)。
2953 // K-Major 下定义为“从首 8 行到次 8 行的字节偏移” (PTX ISA §9.7.15.5.1.2.1.2) 。
2954 // 1024 的来源 (K-Major + 128B swizzle + half) :
2955 // 一个 128B swizzle atom 覆盖 64(K) × 8(M) 个 half。
2956 // 从 1 个 8-row stripe 到下一个 stripe 需要跨越 8 × 64 × 2 = 1024 bytes。
2957 // 若为 MN-Major: SBO 是从首列到下一列的偏移 (8 列一组) 。
2958 desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32;
2959
2960 // — bits [62, 64): layout_type (swizzle mode) —

```

```

2961 // 1llu << 62 = 二进制 01 在 bits [62, 64) = layout_type = 1。
2962 // 1 = 128B swizzle mode。
2963 // 其他取值: 0=None, 2=64B swizzle, 3=32B swizzle。
2964 // 128B swizzle 将 smem 中连续的行按 128B 粒度进行 XOR 重映射,
2965 // 确保同一 warp 内各线程访问不同 bank 的同一偏移时不会产生 bank conflict。
2966 desc |= 1llu << 62;
2967
2968 return desc;
2969 }
2970
2971 // ---- WGMMA PTX 指令宏 ----
2972 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
2973 // m64n128k16: M=64, N=128, K=16。一次 WGMMA 计算  $D[64 \times 128] += A[64 \times 16] \times B[16 \times 128]$ 。
2974 // f16.f16.f16: A=f16, B=f16, D=f16 (累加器也是 f16 精度)。
2975 // 每线程 32 个 uint32 输出:  $D=64 \times 128=8192$  half, warpgroup 128 线程分担,
2976 // 每线程 64 half = 32 uint32。
2977 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成, 64-bit 编码)。
2978 // ScaleD: 0=清零累加器再写入, 1=与累加器中的旧值累加 (K 维迭代必须用 1)。
2979 // ScaleA/ScaleB: 1=正常符号, -1=翻转符号 (此处始终用 1)。
2980 // TransA/TransB: 0=K-major (K 维连续), 1=MN-major (M/N 维连续)。本实现始终用 0。
2981 // 参考: PTX ISA §9.7.15.4 (wgmma.mma_async)
2982 #define WGMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
2983                                     TransB) \
2984 { \
2985     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
2986     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
2987     asm volatile( \
2988         "{\n" \
2989         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
2990         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
2991         "%8, %9, %10, %11, %12, %13, %14, %15, " \
2992         "%16, %17, %18, %19, %20, %21, %22, %23, " \
2993         "%24, %25, %26, %27, %28, %29, %30, %31}, " \
2994         "%32, " \
2995         "%33, " \
2996         "%34, %35, %36, %37, %38;\n" \
2997         "}\n" \
2998         : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
2999         "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
3000         "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
3001         "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
3002         "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
3003         "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
3004         "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
3005         "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
3006         : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
3007         "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
3008         "n"(int32_t(TransB))); \
3009 }
3010
3011 // ---- TMA Shared Memory Layout ----
3012 // Multi-stage pipeline: kStages 个 stage, 每个 stage 存储  $A[BK \times BK] + B[BK \times BN]$ 
3013 //
3014 // 每个 stage 包含两块 smem:
3015 //   A tile:  $[BM, BK]$  row-major  $\rightarrow BK \times BM$  个 half, 地址连续
3016 //   B tile: TMA 按  $[BN, BK]$  物理写入 (详见下方  $\star$ ),  $BN \times BK$  个 half
3017 //
3018 //  $\star$  关键区别 — WGMMA 的 B 物理存储 vs 逻辑视图:
3019 //
3020 //   上层数据: 源矩阵  $B^T [N, K]$  row-major (TN 布局, B 的列以行优先形式存储)。
3021 //   物理存储: TMA 将  $B^T$  的 tile 写入 smem, 物理布局为  $[BN, BK]$  row-major
3022 //               (BN=128 行, BK=64 列, 每行 64 个连续的 half)。
3023 //   TMA 的 smem_box_shape = (BK=64, BN=128) 定义了每个 tile 的 shape,

```

```

3024 //      TMA 按行写入, 行内 BK 个 half 连续 → 物理上就是 [BN][BK]。
3025 //      逻辑视图: WGMMMA 指令 (wgmma.mma_async.m64n128k16, PTX ISA §9.7.15.4)
3026 //      通过 imm-trans-b=0 指定 B 为 K-major, 即逻辑上按 [BK, BN] 寻址。
3027 //      实际 [BN, BK] → 逻辑 [BK, BN] 的"转置"是通过 descB 中的 128B
3028 //      swizzle (layout_type=1) 在硬件地址重映射层完成的, 无需软件转置。
3029 //
3030 //      stride_byte_offset=1024 的来源 (K-major + 128B swizzle + f16, PTX ISA §9.7.15.5.1.2.1.2) :
3031 //      128B swizzle atom = 8(N) × 8(K) × 128-bit = 8 rows × 64 个 half。
3032 //      stride_byte_offset = 从当前 8-row stripe 到下一个 8-row stripe 的字节偏移
3033 //      = 8 rows × (BK=64) halves × 2 bytes = 1024。
3034 //      这个值恰好等于物理 [BN, BK] 布局中连续 8 行的跨度 ← 进一步证实物理存储是 [BN, BK]。
3035 //
3036 //      与 MMA(TN) 的对比:
3037 //      MMA (TN): smem 存 B^T [N, K] row-major, ldmatrix 按列解出 col-major B fragment。
3038 //      WGMMMA: smem 物理存 [BN, BK] (TMA 直写), WGMMMA 通过 swizzle 硬件以 K-major [BK, BN] 逻辑视图读取。
3039 //      简言之: swizzle 桥接了 "物理 row-major [BN, BK]" 和 "逻辑 K-major [BK, BN]" 的差异。
3040 template <int BM, int BN, int BK, int QSIZE> struct WgmmaSMem {
3041     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
3042     // B tile 注记:
3043     // - 物理存储: TMA 从 B^T[N,K] row-major 搬入, 物理上是 [BN, BK] row-major (BN 行, BK 列)
3044     // - 逻辑视图: WGMMMA 通过 descB 的 128B swizzle (PTX ISA §9.7.15.5.1.2) 将地址重映射,
3045     //   以 K-major [BK, BN] 逻辑视图供 wgmma.mma_async (imm-trans-b=0, PTX ISA §9.7.15.4) 读取
3046     // - B[BN * BK * QSIZE] 与 B[BK * BN * QSIZE] 数值等价 (BN×BK = BK×BN),
3047     //   但写 BN*BK 更能体现物理存储形态
3048     alignas(128) half B[BN * BK * QSIZE];
3049 };
3050
3051 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
3052 // 面试重点 — Warp Specialization (Hopper 最核心的编程模型变化):
3053 //
3054 // 背景: 传统 GEMM kernel 中, 所有线程同步地"加载→计算→存回",
3055 // 数据搬运和计算无法重叠。Hopper 的 TMA + WGMMMA 支持将工作拆分为两个
3056 // warpgroup, 让数据搬运和矩阵乘完全异步执行:
3057 //
3058 // WG0 (128 threads, Producer): 仅 thread 0 提交 TMA 2D 拷贝指令,
3059 //   将 A/B tile 从 HBM 搬到 shared memory。
3060 // WG1 (128 threads, Consumer): 所有 128 个线程参与 WGMMMA 矩阵乘。
3061 //
3062 // Producer 和 Consumer 通过 cuda::barrier (CTA 级别) 同步:
3063 // - full[stage]: Producer 发信号表示 stage stage 的数据已就绪, 可被使用
3064 // - empty[stage]: Consumer 发信号表示 stage stage 的使用完毕, 可以被覆盖
3065 //
3066 // 本节重点理解:
3067 // 1) 为什么 Producer 只需要 thread 0? TMA 是硬件 DMA 指令, 一次提交
3068 //   即可搬运整个 2D tile, 无需所有线程参与。
3069 // 2) barrier 的 arrive count = 129: 128 个 Consumer 线程 + 1 个 Producer 提交线程。
3070 // 3) 多 stage pipeline 使 Consumer 计算 stage stage 的同时,
3071 //   Producer 可以搬运 stage (stage+1), 隐藏 HBM→SMEM 延迟。
3072 //
3073 // Tile Hierarchy (与 MMA m16n8k16 kernel 对比):
3074 // WGMMMA Atom: m64n128k16 (一次处理 64×128×16, 是 MMA 的 64 倍)
3075 // K Tile: BK=64, 每个 K tile 包含 BK/kWgmmaK=4 个 WGMMMA atom
3076 // M Tile: BM=128, 每个 M tile 包含 BM/kWgmmaM=2 个 WGMMMA atom
3077 // N Tile: BN=128, 单个 kWgmmaN=128 即可覆盖, 无需在 N 方向分块
3078 // Block Tile: C[128,128] = A[128,64] × B[64,128]
3079 // Threads: 256 = 2 warpgroups × 128 threads/warpgroup
3080 // Producer(WG0): 128 threads (仅 thread 0 做 TMA 提交)
3081 // Consumer(WG1): 128 threads = 4 warps (全部参与 WGMMMA 和写回)
3082 //
3083 // kStages=3: 3 级流水线。Consumer 滞后 Producer 最多 2 步, 确保 HBM→SMEM
3084 // 的延迟被计算完全掩盖。
3085 //
3086 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle

```



```

3087 // - grid.z = 5 个 swizzle 分区, 将连续 block 打散到不同 N 区域改善 L2 命中
3088 // Block: (256, 1, 1), 2 warpgroups
3089 // source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
3090 template <const int kWmmaM = 64,           // WGMMMA atom M dim (m64n128k16)
3091           const int kWmmaN = 128,         // WGMMMA atom N dim
3092           const int kWmmaK = 16,          // WGMMMA atom K dim
3093           const int BM = 128,             // block tile M, 2 × kWmmaM
3094           const int BN = 128,            // block tile N, 1 × kWmmaN
3095           const int BK = 64,             // block tile K, 4 × kWmmaK
3096           const int kNumThreads = 256,    // 2 warpgroups × 128 threads
3097           const int kStages = 3,          // TMA pipeline depth (full/empty barriers)
3098           const int kBlockSwizzle = 0>    // 1 enables 3D grid swizzle for L2 locality
3099 __global__ void __launch_bounds__(kNumThreads)
3100   hgemm_wgmma_stages_tn(
3101     int M, int N, int K, half *C,
3102     const UTensorMap *__restrict__ tensorMapA,
3103     const UTensorMap *__restrict__ tensorMapB) {
3104   static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3105   // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
3106   // 对 row-major [M, K] 矩阵, TMA shape 参数写的是 (K, M) 而不是 (M, K),
3107   // 也就是TMA descriptor中把连续的维度写在最内层, 非连续的维度写在最外层。
3108   // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
3109   // 不展开宿主侧 create_tensor_map 细节。
3110
3111   // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
3112   // bx = blockIdx.z * gridDim.x + blockIdx.x, 将相邻 block 打散到不同 N 区域
3113   const int bx = ((int)kBlockSwizzle) * blockIdx.z * gridDim.x + blockIdx.x;
3114   const int by = blockIdx.y;
3115   constexpr int kConsumerThreads = kNumThreads / 2; // 128 threads = 1 warpgroup
3116   // kNumConsumers = (kNumThreads / 128) - 1 = 2 - 1 = 1 (1 个 consumer WG)
3117   // 这里的 -1 是因为 2 个 warpgroup 中 1 个是 producer, 剩余都是 consumer
3118   constexpr int kNumConsumers = (kNumThreads / kConsumerThreads) - 1; // 1 consumer WG
3119   // kWarpgroupM = BM / kNumConsumers: 每个 consumer warpgroup 负责的 M 行数
3120   // 当只有一个 consumer 时, 它负责全部 BM=128 行
3121   constexpr int kWarpgroupM = BM / kNumConsumers; // 128
3122
3123   // 边界检查: 确保当前 block 不超出 M/N 范围
3124   if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3125     return;
3126
3127   // ---- Shared Memory 分配 ----
3128   // 动态 shared memory (由 host 侧通过 kernel launch 的 smem 参数指定大小)
3129   // TMA + WGMMMA 仅需 __align__(128), 而非 __align__(1024)。原因:
3130   //   make_smem_desc() 构造 WGMMMA descriptor 时, start_address 字段
3131   //   完整编码了 __cvta_generic_to_shared 的绝对 32-bit 偏移量,
3132   //   硬件根据该绝对地址自动推导并补偿 swizzle phase 偏移 (等价于
3133   //   descriptor 内置的 base_offset 机制)。因此即使 smem 基址未落到
3134   //   1024B 边界, WGMMMA 硬件也能正确读取 TMA 写入的数据。
3135   // 对比 hgemm_tma_mma_ws_tn: 消费者使用 ldmatrix + 手写 swizzle
3136   //   函数 tma_swizzle_128B(), 该函数无法感知 smem 绝对地址, 硬编码
3137   //   了 phase=0 的假设, 因此必须 __align__(1024) 来保证 phase 确实为 0。
3138   // 从健壮性角度看本 kernel 也应该用 __align__(1024); 这里保留 128 是
3139   // 为了突出两种消费者的特点与对比。
3140   extern __shared__ __align__(128) uint8_t smem_tma_wgmma_ws[];
3141   WgmmaSMem<BM, BN, BK, kStages> &s =
3142     *reinterpret_cast<WgmmaSMem<BM, BN, BK, kStages>*>(smem_tma_wgmma_ws);
3143   half *s_a = s.A;
3144   half *s_b = s.B;
3145
3146   // ---- cuda::barrier 初始化 ----
3147   //
3148   // ★ Barrier 机制速查 (arrive / wait 核心语义):
3149   //

```



```

3150 // cuda::barrier 是一个 CTA 级同步原语, 基于 **phase (奇偶交替)** 工作:
3151 //
3152 //   init(bar, N): 设置 arrive_count = N。
3153 //   含义: 每 phase 需要 N 个线程各调用一次 arrive(), barrier 才翻转 phase。
3154 //
3155 //   arrive(): 线程声明"我已到达此 barrier 点"。
3156 //   - 非阻塞, 立即返回一个 token (包含当前 phase 值)。
3157 //   - 不关心"谁"到达, 只关心"到达次数"是否达到 arrive_count。
3158 //
3159 //   wait(token): 线程阻塞, 直到 barrier 的当前 phase ≠ token 中的 phase。
3160 //   - 即: 等待 phase 翻转。
3161 //   - Phase 翻转条件: (1) arrive 次数达到 arrive_count
3162 //                     (2) 若使用了 barrier_arrive_tx, 还需 TMA 字节全部写完
3163 //
3164 //   典型用法: b.wait(b.arrive())
3165 //   - arrive() 注册自己到达, 拿到 token (当前 phase = P)
3166 //   - wait(token) 阻塞直到 phase 翻转 (P → P+1)
3167 //   - 如果自己是第 N 个到达者 → phase 立即翻转 → wait 立即返回 (不阻塞)
3168 //   - 如果自己不是最后一个 → wait 阻塞, 直到第 N 个到达者触发翻转
3169 //
3170 // ★ 本 kernel 的 Pipeline 同步协议 (kStages=3, arrive_count=129) :
3171 //
3172 //   Producer (1 thread)           Consumer (128 threads)
3173 //   ────────────                   ────────────
3174 //                                     C0: arrive(empty[*]) ×128 [init: 标记所有 stage 为空]
3175 //   ┌ for each k_tile: ─┐           ┌ for each k_tile: ─┐
3176 //   │ P1: arrive+wait(empty[q]) │   │ C1: arrive+wait(full[q]) │
3177 //   │   ↓ 等 stage q 被消费完   │   │   ↓ 等 TMA 数据就绪     │
3178 //   │ P2: TMA(A[q]) + TMA(B[q]) │   │ C2: WGMMMA 计算         │
3179 //   │   ↓ 异步拷贝             │   │ C3: wait WGMMMA 完成     │
3180 //   │ P3: arrive_tx(full[q])    │   │ C4: arrive(empty[q])    │
3181 //   │   ↑ 通知: stage q 数据就绪 │   │   ↑ 通知: stage q 已消费完 │
3182 //   └──────────────────┘           └──────────────────┘
3183 //
3184 //   关键不变式 (invariant) :
3185 //   - Producer 不会覆盖 Consumer 正在读的 stage
3186 //   - Consumer 不会读 Producer 还没写完的 stage
3187 //   - 通过 full/empty 两个 barrier 的 phase 交替来保证
3188 //
3189 //   每个 stage 有两个 barrier:
3190 //   full[stage]: TMA 数据就绪信号。Producer 发 (arrive_tx), Consumer 等 (wait)。
3191 //   empty[stage]: Stage 空闲信号。Consumer 发 (arrive), Producer 等 (wait)。
3192 //
3193 //   arrive_count = 128 (consumer) + 1 (producer) = 129:
3194 //   每 phase, 128 个 consumer 线程 + 1 个 producer 线程都要 arrive 一次。
3195 #pragma nv_diag_suppress static_var_with_dynamic_init
3196 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3197 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3198 #pragma nv_diag_default static_var_with_dynamic_init
3199
3200 // K 方向总 tile 数。要求 K 能被 BK 整除, 否则尾 tile 被丢弃。
3201 const int NUM_K_TILES = div_ceil(K, BK);
3202 const int wg_idx = threadIdx.x / kConsumerThreads; // 0=Producer, 1=Consumer
3203 const int wg_tid = threadIdx.x % kConsumerThreads; // 0~127 within warpgroup
3204
3205 // 初始化 barriers (仅 thread 0 执行)
3206 if (threadIdx.x == 0) {
3207     for (int i = 0; i < kStages; ++i) {
3208         // arrive_count = 129: 128 consumer + 1 producer
3209         // init 是 CUDA barrier 的 hidden friend 函数 (定义在 cuda::barrier 类体内),
3210         // 只能通过 ADL (Argument-Dependent Lookup) 调用。因为第一个参数 &full[i] 的类型是
3211         // cuda::barrier<cuda::thread_scope_block>*, 编译器通过 ADL 在 cuda 命名空间中自动找到它。
3212         init(&full[i], kConsumerThreads + 1); // 128 consumer + 1 producer

```

```

3213     init(&empty[i], kConsumerThreads + 1); // same
3214 }
3215 // fence_proxy_async_shared_cta: 确保 barrier 在 smem 中的初始化
3216 // 对 async proxy (TMA/WGMMMA) 可见。
3217 tma_fence_proxy_async_shared_cta();
3218 }
3219 __syncthreads();
3220
3221 // =====
3222 // ★ Warp Specialization 并发模型 — 为什么只有 if/else 没有 while?
3223 // =====
3224 //
3225 // 256 个线程在同一个 SM 上**并发执行**。if/else 只是分工，不是先后顺序：
3226 //   - WG0 (threadIdx.x 0~127): 全部走 if 分支，做 Producer。
3227 //   - WG1 (threadIdx.x 128~255): 全部走 else 分支，做 Consumer。
3228 //
3229 // SM 的 warp scheduler 会交替调度来自 WG0 和 WG1 的 warp，两者**同时**推进：
3230 //   Producer: for (k_iter = 0 .. NUM_K_TILES) { P1→P2→P3 } ← 自己的循环
3231 //   Consumer: for (k_iter = 0 .. NUM_K_TILES) { C1→C2→C3→C4 } ← 自己的循环
3232 //
3233 // 两个 warpgroups 各自独立迭代 K-tiles，通过 full[] / empty[] barrier 同步：
3234 //   - Producer 领先 Consumer 太多 → wait(empty[q]) 阻塞，等 Consumer 消费完
3235 //   - Consumer 领先 Producer 太多 → wait(full[q]) 阻塞，等 TMA 搬运完
3236 //
3237 // 这是生产者-消费者模型的 GPU 实现：不需要共享循环变量，barrier 的 phase
3238 // 交替天然保证了流水线串行化 (at most kStages-1 steps ahead)。
3239 // =====
3240 // Producer Warpgroup (WG0, threadIdx.x 0~127)
3241 // 职责：提交 TMA 2D 拷贝，将 A/B tile 从 HBM 异步搬运到 SMEM。
3242 // 只有 wg_tid==0 执行实际拷贝提交，其余 127 个线程空闲。
3243 // =====
3244 if (wg_idx == 0) {
3245     if (wg_tid == 0) {
3246         // stage: 当前操作的 stage 索引 (round-robin 0 -> 1 -> 2 -> 0 -> ...)
3247         int stage = 0;
3248         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3249             if (stage == kStages)
3250                 stage = 0;
3251
3252             // Step P1: 等待 stage stage 变为"空" (可被覆盖写入)
3253             //
3254             // 模式 empty[stage].wait(empty[stage].arrive()):
3255             //   - arrive(): Producer (1 个线程) 在 empty barrier 上注册到达，
3256             //     获得当前 phase 的 token。如果 Consumer 已经在 C4 步骤积累了
3257             //     128 次 arrive (来自上一轮迭代或 C0 初始化)，则加上这次共 129 次
3258             //     → phase 立即翻转。
3259             //   - wait(token): 阻塞直到 barrier phase ≠ token 中的 phase。
3260             //     由于 arrive() 是第 129 次到达，phase 在 arrive 时已翻转，
3261             //     wait 看到 phase 已变 → 立即返回 (首轮不阻塞)。
3262             //
3263             // 语义：Producer 说"我准备好写 stage stage 了，Consumer 用完没有？"
3264             //     如果 Consumer 还没用完 → phase 未翻转 → wait 阻塞等待。
3265             //     如果 Consumer 已用完 → phase 已翻转 → wait 立即返回。
3266             empty[stage].wait(empty[stage].arrive());
3267
3268             // Step P2: 提交 TMA 2D 拷贝指令
3269             // cp_async_bulk_tensor_2d_global_to_shared:
3270             //   参数1(dst): smem 目标地址 (当前 stage 的 A/B tile 起始位置)
3271             //   参数2(tensorMap): Host 预创建的 TMA descriptor (描述源矩阵的 shape/stride/dtype)
3272             //   参数3/4(coords): (k_offset, m_offset) 或 (k_offset, n_offset) 的全局坐标
3273             //   参数5(barrier): 拷贝完成后自动 arrive 到此 barrier
3274             //
3275             // TMA 是硬件 DMA 引擎：一次指令提交即可搬运整个 2D tile,

```

```

3276 // 无需线程逐元素搬运，零寄存器开销。
3277 // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
3278 tma_load_2d(&s_a[stage * BK * BM], tensorMapA, k * BK, by * BM,
3279             full[stage]);
3280
3281 // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
3282 tma_load_2d(&s_b[stage * BK * BN], tensorMapB, k * BK, bx * BN,
3283             full[stage]);
3284
3285 // Step P3: 通知 Consumer: stage stage 的 TMA 数据已就绪
3286 //
3287 // barrier_arrive_tx(bar, arrive_count_update, byte_count):
3288 //   - 在 bar 上注册 1 次到达 (arrive_count_update=1)
3289 //   - 同时声明预期有 byte_count 字节将通过 async copy (TMA) 写入 smem
3290 //   - Phase 翻转条件:
3291 //       (a) 总 arrive 次数达到 129 (128 consumer + 1 producer)
3292 //       (b) 所有声明的 async 字节已写入 smem
3293 //   两个条件都满足后 phase 才翻转, Consumer 的 wait() 才返回。
3294 //   - 即: Consumer 不会在 TMA 数据完整到达前就开始读 smem。
3295 tma_arrive_expect_tx(full[stage], (BK * BN + BK * BM) * sizeof(half));
3296 }
3297 }
3298 }
3299 // =====
3300 // Consumer Warpgroup (WG1, threadIdx.x 128~255)
3301 // 职责: 等待 TMA 数据就绪 -> 发射 WGMMMA 做矩阵乘 -> 积累结果 -> 写回 C
3302 // 所有 128 个线程 (4 warps) 全部参与。
3303 // =====
3304 else {
3305     // Step C0: Consumer 初始化 — 标记所有 stage 为"空" (可被 Producer 写入)
3306     //
3307     // 所有 128 个 Consumer 线程对每个 stage 的 empty barrier 调用 arrive()。
3308     // 这是 Pipeline 的"预热"步骤——没有它, Producer 的 empty[stage].wait()
3309     // 在第一轮会永远阻塞 (因为 Producer 的 1 次 arrive 不足以凑够 129)。
3310     //
3311     // 注意: 此时每个 empty[i] 只有 128 次 arrive, 未达 129, phase 不翻转。
3312     // Producer 后续的 empty[stage].arrive() 作为第 129 次, 触发 phase 翻转。
3313     for (int i = 0; i < kStages; ++i) {
3314         [[maybe_unused]] auto token = empty[i].arrive();
3315     }
3316
3317     // 累加器寄存器声明
3318     // d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4]:
3319     //   - d[0][*][*]: M 方向第 1 个 WGMMMA atom (rows 0~63)
3320     //   - d[1][*][*]: M 方向第 2 个 WGMMMA atom (rows 64~127)
3321     //   - d[*][g][*]: N 方向第 g 组 16 列
3322     //   - d[*][*][0..3]: 4 条 uint32 寄存器, 共 8 个 half (覆盖 16x16 子块)
3323     // 每个线程总共 2 * 8 * 4 = 2 * 32 = 64 uint32 = 128 half (每次WGMMMA Atom 32 uint32 输出)
3324     // 128 线程 * 128 half = 16384 half = 128 * 128 = BM*BN (刚好覆盖整个 C tile)
3325     uint32_t d[kWarpgroupM / kWgmmaM][kWgmmaN / 16][4] = {};
3326
3327     int stage = 0;
3328     // K 维外循环: 沿 K tile 迭代 (BK=64, 每个 K tile 做 4 次 WGMMMA 累加)
3329     for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3330         if (stage == kStages)
3331             stage = 0;
3332
3333         // Step C1: 等待 TMA 数据就绪 (full 信号)
3334         //
3335         // 模式 full[stage].wait(full[stage].arrive()):
3336         //   - 128 个 Consumer 线程各调用 arrive(), 共 128 次到达。
3337         //   - 当前 phase 累积: 128/129, phase 尚未翻转。
3338         //   - wait(token) 阻塞, 直到 Producer 的 barrier_arrive_tx (Step P3)

```

```

3339 // 贡献第 129 次到达 + TMA 字节全部写完 → phase 翻转 → wait 返回。
3340 //
3341 // 语义: Consumer 说"我准备好读 stage stage 了, 数据到了没有?"
3342 full[stage].wait(full[stage].arrive());
3343
3344 // Step C2: 发射 WGMMMA 指令序列
3345 //
3346 // WGMMMA 是异步指令 (fire-and-forget), 发射后立即返回, 不等待计算完成。
3347 // 标准流程: FENCE → 发射 WGMMMA → COMMIT → WAIT。
3348 //
3349 // WGMMMA_FENCE (wgmma.fence.sync.aligned) :
3350 // - 确保 TMA 写入 smem 的数据对 async proxy (WGMMMA) 可见
3351 // - 确保累加器寄存器 d[] 已准备好接收 WGMMMA 输出
3352 // - 本质是 proxy 间的内存序 (memory ordering), 不是传统 __syncthreads
3353 //
3354 // 每个 WGMMMA_M64N128K16_F16F16F16 指令:
3355 // D[64×128] += A[64×16] × B[16×128]
3356 // A/B 均为 K-major (imm-trans=0), 由 descA/descB 描述 smem 中的布局。
3357 // ScaleD=1: 累加。K 维有 BK/kWgmmaK=4 次迭代, 每次都用 ScaleD=1。
3358 WGMMMA_FENCE();
3359
3360 // M 维迭代: BM/kWgmmaM = 128/64 = 2 个 WGMMMA atom
3361 // 每个 atom 处理 64 行 M 方向数据。
3362 #pragma unroll
3363 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3364     // wgmma_sA 指向当前 stage 中 A tile 的第 m 个 64*BK 子块
3365     // s_a 布局: [kStages][BM][BK] -> stage * BK*BM + BK * (m*64)
3366     half *wgmma_sA = s_a + stage * BK * BM + BK * m * kWgmmaM;
3367
3368     // K 维迭代: BK/kWgmmaK = 64/16 = 4 次 WGMMMA (累加)
3369     // 每次处理 K=16 维的矩阵乘, 4 次累加后覆盖完整的 BK=64。
3370 #pragma unroll
3371     for (int k_step = 0; k_step < BK / kWgmmaK; ++k_step) {
3372         // 第 k_step 次 WGMMMA:
3373         // A: wgmma_sA + k_step * kWgmmaK (A 的 K 维起始位置)
3374         // B: s_b + stage * BK * BN + k_step * kWgmmaK (B 的 K 维起始位置)
3375         // 注意 B 的 smem 布局是 [BK, BN] row-major, K-major 读取时从
3376         // 第 k_it*16 行开始取 16 行, 每行 BN=128 列。
3377         // ScaleD=1: 累加 (K 维迭代需要累积结果)
3378         WGMMMA_M64N128K16_F16F16F16(d[m], wgmma_sA + k_step * kWgmmaK,
3379                                     s_b + stage * BK * BN + k_step * kWgmmaK,
3380                                     1, // ScaleD=1: accumulate (不清零)
3381                                     1, 1, 0, 0);
3382     }
3383 }
3384
3385 // Step C3: 提交并等待 WGMMMA 完成
3386 //
3387 // WGMMMA_COMMIT_GROUP (wgmma.commit_group.sync.aligned):
3388 // 将自上一次 COMMIT 以来发射的所有 WGMMMA 归为一组, 提交到 async proxy 执行。
3389 // 类比 cp.async.commit_group, 但 scope 是 warpgroup 而非 per-thread。
3390 //
3391 // WGMMMA_WAIT_GROUP(0) (wgmma.wait_group.sync.aligned 0):
3392 // 阻塞直到最多 N 个 group 尚未完成 (pending ≤ N)。N=0 即等待所有 group。
3393 // ★ 语义与 cp.async.wait_group 完全一致 (PTX ISA §9.7.15.7.3 vs §9.7.9.25.3.3) :
3394 // 两者都是 "wait until only N or fewer of the most recent groups are pending"。
3395 // 此处用 0 是因为 Consumer 在本次迭代中只 commit 了 1 个 group,
3396 // 必须等它全部完成才能进入下一步 (读下一 stage 的数据)。
3397 WGMMMA_COMMIT_GROUP();
3398 WGMMMA_WAIT_GROUP(0);
3399
3400 // Step C4: 释放 stage stage — 通知 Producer 可以覆盖写入
3401 //

```

```

3402 // 128 个 Consumer 线程在 empty[stage] 上 arrive(), 为 **下一 phase** 积累到达次数。
3403 // 这 128 次 arrive 不会立即翻转 phase (还需 Producer 在下一轮的 1 次 arrive)。
3404 //
3405 // 语义: Consumer 说"stage stage 我已经读完了, 你可以放心覆盖了。"
3406 [[maybe_unused]] auto token = empty[stage].arrive();
3407 }
3408
3409 // =====
3410 // Epilogue: 将寄存器中的累加结果写回 global memory C
3411 // =====
3412 //
3413 // WGMMMA m64n128k16.f16.f16.f16 的输出寄存器映射:
3414 //
3415 // 对 warpgroup 内每个线程, 输出 64 个 half = 32 个 uint32。
3416 // 这些 half 分布在 d[2][8][4] 数组中:
3417 // d[m_it][g][0]: (row, col) 和 (row, col+2) 位置的 2 个 half
3418 // d[m_it][g][1]: (row+8, col) 和 (row+8, col+2) 位置的 2 个 half
3419 // d[m_it][g][2]: (row, col+8) 和 (row, col+10) 位置的 2 个 half
3420 // d[m_it][g][3]: (row+8, col+8) 和 (row+8, col+10) 位置的 2 个 half
3421 //
3422 // 其中:
3423 // row = warp * 16 + lane / 4 (0~63, 4 warps * 16 rows)
3424 // col = g * 16 + 2 * (lane % 4) (0~126, step 2, 8 g's * 16 cols)
3425 //
3426 // 每个线程覆盖一个 16x16 子块中的 16 个 half:
3427 // +-----+-----+
3428 // | reg[0] | reg[2] | <- rows [row, row+8)
3429 // |(col,+2)|(col+8)|
3430 // +-----+-----+
3431 // | reg[1] | reg[3] | <- rows [row+8, row+16)
3432 // |(col,+2)|(col+8)|
3433 // +-----+-----+
3434 // 每个 reg 包含 2 个连续 half (col 和 col+2), 用 uint32 存储。
3435 //
3436 // 三维遍历:
3437 // m_it=0: rows 0~63, m_it=1: rows 64~127
3438 // g=0..7: 每个覆盖 16 列, 8*16=128 列 ok
3439 // 每个线程写 4 uint32 * 2 half/uint32 * 8 g * 2 m_it = 128 half。
3440 // 128 线程 * 128 half = 16384 half = 128 * 128 ok
3441
3442 const int lane = wg_tid % 32;
3443 const int warp = wg_tid / 32;
3444 // row: 当前线程在当前 WGMMMA atom 中负责的起始行。
3445 // warp 0~3 各负责 16 行: rows [warp*16, warp*16+15]。
3446 // lane/4: 每 4 个连续 lane 负责同一行 (因为 WGMMMA fragment 中每行有 4 个 uint32,
3447 // 分别覆盖列对 {c0,c1}、{c2,c3}、{c8,c9}、{c10,c11}, 由 lane%4 区分)。
3448 // 因此 lane/4 给出该行在 16-row block 内的行号 (0~7 对应 rows 0~7,
3449 // 同一 warp 中 lane/4==0 的有 lane 0,1,2,3, 都负责 row 0)。
3450 const int row = warp * 16 + lane / 4;
3451 // block_C: 当前 C tile 在 global memory 中的起始地址
3452 // by*BM 是 M 方向偏移, bx*BN 是 N 方向偏移
3453 half *block_C = C + by * BM * N + bx * BN;
3454
3455 // 2 * (8 * 4) = 2 * 32 = 64 uint32 = 128 half
3456 // NOTE: 这里可以考虑通过 R->S, S->G 的方式做一次 128 bits写回。
3457 #pragma unroll
3458 for (int m = 0; m < kWarpgroupM / kWgmmaM; ++m) {
3459     int yo = m * kWgmmaM; // M 方向行偏移 (0 或 64)
3460 #pragma unroll
3461     for (int g = 0; g < kWgmmaN / 16; ++g) {
3462         int col = g * 16 + 2 * (lane % 4); // N 方向列偏移 (0,2,4,...,14 then 16,18,...)
3463         // 一次 uint32 store 写入 2 个 half (连续列 col 和 col+2)
3464         // 左上象限: (row, col)

```

```

3465     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) = d[m][g][0];
3466     // 左下象限: (row+8, col)
3467     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) = d[m][g][1];
3468     // 右上象限: (row, col+8)
3469     *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) = d[m][g][2];
3470     // 右下象限: (row+8, col+8)
3471     *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) = d[m][g][3];
3472 }
3473 }
3474 }
3475 }
3476
3477 #endif /* NOTES_V2_ENABLE_WGMMMA */
3478
3479 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
3480 // SM120 不支持 WGMMMA, 但支持相同的 TMA 生产者协议和 warp 级 mma.sync。
3481 // 保持 128B TMA swizzle 并显式声明物理布局供 ldmatrix 消费者使用。
3482 template <int BM, int BN, int BK, int QSIZE> struct TmaMmaWSSMem {
3483     static_assert(BK == 64, "The 128B swizzle helper below is specialized for BK=64");
3484     half A[BM * BK * QSIZE];
3485     half B[BN * BK * QSIZE];
3486 };
3487
3488 // tma_swizzle_128B: 计算 128B TMA swizzle 下 smem 中 (row, k) 的物理 K 偏移。
3489 //
3490 // TMA descriptor 使用 CU_TENSOR_MAP_SWIZZLE_128B 时, 硬件会将写入 smem 的
3491 // 数据按 128B 粒度做 XOR 重映射以消除 bank conflict。消费者在 ldmatrix 前必须
3492 // 对地址施加相同的 swizzle 变换, 否则读到的数据与 TMA 写入的物理位置不一致。
3493 //
3494 // 位级公式与 CuTe 对照:
3495 //   swizzle_k = (((k >> 3) ^ (row & 7)) << 3) | (k & 7)
3496 //
3497 //   等价于 CuTe Swizzle<3,4,3>:
3498 //   M=3 → 基本元素宽 2^3=8 个 half (16 bytes)
3499 //   S=4 → 每行 2^4=16 个基本元素 (32 个 half) ...不, BK=64...
3500 //
3501 // 实际让我们从函数出发:
3502 //   - k & 7: 低 3 位保持不变。即 8 个 half (16 bytes) 为一组, 组内顺序不变。
3503 //   - k >> 3: K 被划分为 8 个 chunk (BK=64, 每 chunk 8 个 half)。
3504 //   - row & 7: 取行号的低 3 位 (swizzle 周期为 8 行)。
3505 //   - XOR: 将 chunk 索引与行号的低 3 位做异或。
3506 //
3507 // ★ swizzle 映射图 (BK=64, 每 row 8 chunk à 8 half; 周期 = 8 rows × 512 half = 1024B) :
3508 //
3509 //   下图表示同一 chunk 索引在各行被映射到的物理 chunk 位置。例如 chunk 0 在
3510 //   row 0 位于物理 chunk 0, 在 row 1 被 XOR 到物理 chunk 1, 以此类推。
3511 //
3512 //   chunk idx:      0      1      2      3      4      5      6      7
3513 //   ───────────────────────────────────────────────────────────────────────────────────
3514 //   row 0 (0^c):    0      1      2      3      4      5      6      7      ← 恒等
3515 //   row 1 (1^c):    1      0      3      2      5      4      7      6      ← 相邻对交换
3516 //   row 2 (2^c):    2      3      0      1      6      7      4      5
3517 //   row 3 (3^c):    3      2      1      0      7      6      5      4
3518 //   row 4 (4^c):    4      5      6      7      0      1      2      3
3519 //   row 5 (5^c):    5      4      7      6      1      0      3      2
3520 //   row 6 (6^c):    6      7      4      5      2      3      0      1
3521 //   row 7 (7^c):    7      6      5      4      3      2      1      0
3522 //
3523 //   — 8 行一周期 —
3524 //   row 8 (0^c): 同 row 0 映射; row 9 同 row 1; 依此类推。
3525 //
3526 //   例子——ldmatrix 读取 row 1、逻辑 k=0 处的数据:
3527 //       swizzle_k = (((0 >> 3) ^ (1 & 7)) << 3) | (0 & 7) = ((0 ^ 1) << 3) | 0 = 8

```



```

3528 // 即 row 1 的 chunk 0 被 TMA 写到了物理 K=8 的位置, ldmatrix 地址需从 K=8 读。
3529 //
3530 // 例子——ldmatrix 读取 row 3、逻辑 k=16 处的数据:
3531 // swizzle_k = (((16 >> 3) ^ (3 & 7)) << 3) | (16 & 7)
3532 //           = ((2 ^ 3) << 3) | 0 = (1 << 3) | 0 = 8
3533 // 即 row 3 的 chunk 2 被映射到物理 chunk 1 (K=8) 。
3534 //
3535 // 关键结论: TMA 和 ldmatrix 两侧必须使用相同的 swizzle; 任何一侧不用或用了
3536 // 不同的 pattern, 整个 tile 的输出都会被破坏。128B = 128 BYTES = 64 half
3537 // = 8 chunk × 8 half/chunk = 8 rows × 512 half/row。
3538 __device__ __forceinline__ int tma_swizzle_128B(int row, int k) {
3539     return (((k >> 3) ^ (row & 7)) << 3) | (k & 7);
3540 }
3541
3542 // 消费者 MMA 层级映射参考 hgemm_mma_stages_tn_swizzle。与那个
3543 // 八 warp kernel 不同, 本 warp-specialized 消费者仅拥有四个 warp。
3544 template <const int kMmaM = 16,           // MMA atom M dim (m16n8k16)
3545           const int kMmaN = 8,           // MMA atom N dim
3546           const int kMmaK = 16,          // MMA atom K dim
3547           const int kMmaTileM = 2,       // consumer warps along M, warp tile M = 32
3548           const int kMmaTileN = 2,       // consumer warps along N, warp tile N = 16
3549           const int kValTileM = 4,       // value-repeat along M, BM = 16*2*4 = 128
3550           const int kValTileN = 8,       // value-repeat along N, BN = 8*2*8 = 128
3551           const int kValTileK = 4,       // MMA_K slices per BK tile, BK = 16*4 = 64
3552           const int kStages = 2,         // TMA full/empty pipeline depth
3553           const int kNumThreads = 256,   // 128 producer + 128 consumer threads
3554           const int kBlockSwizzle = 0>   // 1 enables 3D grid swizzle for L2 locality
3555 __global__ void __launch_bounds__(kNumThreads)
3556 hgemm_tma_mma_ws_tn(
3557     int M, int N, int K, half *C,
3558     const CUtensorMap *__restrict__ tensorMapA,
3559     const CUtensorMap *__restrict__ tensorMapB) {
3560
3561     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
3562     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
3563     constexpr int BK = kMmaK * kValTileK;
3564     static_assert(kMmaM == 16 && kMmaN == 8 && kMmaK == 16, "This kernel uses mma.sync.m16n8k16");
3565     static_assert(kMmaTileM * kMmaTileN == 4, "The consumer warpgroup has exactly four warps");
3566     static_assert(BM == 128 && BN == 128 && BK == 64, "TMA desc and 128B swizzle require 128x128x64");
3567     static_assert(kNumThreads == 256, "Use one producer and one consumer warpgroup");
3568     static_assert(kBlockSwizzle == 0 || kBlockSwizzle == 1, "kBlockSwizzle must be 0 or 1");
3569
3570     const int bx = kBlockSwizzle * blockIdx.z * gridDim.x + blockIdx.x;
3571     const int by = blockIdx.y;
3572     constexpr int kConsumerThreads = kNumThreads / 2;
3573     static_assert(kConsumerThreads == kMmaTileM * kMmaTileN * kWarpSize,
3574         "Consumer threads must cover the MMA warp grid");
3575
3576     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
3577         return;
3578
3579     // TMA CU_TENSOR_MAP_SWIZZLE_128B 要求 smem 基地址 1024 字节对齐,
3580     // 以确保硬件 swizzle phase 从零开始。消费者 tma_swizzle_128B()
3581     // 假设零 phase; 其他对齐 (如 128) 会导致 phase 偏移, 产生不匹配的
3582     // 物理地址, 破坏整个输出。
3583     //
3584     // 为什么 hgemm_wgmma_stages_tn 只需要 __align__(128), 而本 kernel
3585     // 必须 __align__(1024)? 核心区别在于消费者如何感知 swizzle phase:
3586     //
3587     // WGMMA 消费者: 通过 make_smem_desc() 构造 64-bit WGMMA descriptor,
3588     // 其中 bits [49,52) 的 base_offset 字段编码了 smem 基址相对于
3589     // 1024B 边界的偏移量 ((addr >> 7) & 0x7)。硬件在读取 smem 时会
3590     // 用这个 base_offset 自动补偿 swizzle phase, 因此 WGMMA 路径不

```

```

3591 // 需要 1024 字节对齐也能得到正确数据。
3592 //
3593 // 本 kernel 消费者：使用 ldmatrix + 手写 tma_swizzle_128B() 来
3594 // 计算 smem 物理地址。这个函数是纯软件公式，没有任何 base_offset
3595 // 补偿机制。它假设 smem 基址恰好落在 1024B 边界上 (phase=0)。
3596 // 如果基址只满足 128B 对齐，TMA 硬件会以非零 phase 写入数据，
3597 // 而 ldmatrix 以零 phase 读取 → 物理地址彻底错位 → 全输出错误。
3598 //
3599 // 简言之：WGMMMA 用硬件 descriptor base_offset 自动解决 phase 偏移；
3600 // ldmatrix 路径没有等价机制，必须靠 1024B 对齐来保证 phase=0。
3601 extern __shared__ __align__(1024) uint8_t smem_tma_mma_ws[];
3602 TmaMmaWSSMem<BM, BN, BK, kStages> &s =
3603     *reinterpret_cast<TmaMmaWSSMem<BM, BN, BK, kStages> *>(smem_tma_mma_ws);
3604 half *s_a = s.A;
3605 half *s_b = s.B;
3606
3607 #pragma nv_diag_suppress static_var_with_dynamic_init
3608 __shared__ cuda::barrier<cuda::thread_scope_block> full[kStages];
3609 __shared__ cuda::barrier<cuda::thread_scope_block> empty[kStages];
3610 #pragma nv_diag_default static_var_with_dynamic_init
3611
3612 const int NUM_K_TILES = div_ceil(K, BK);
3613 const int wg_idx = threadIdx.x / kConsumerThreads;
3614 const int wg_tid = threadIdx.x % kConsumerThreads;
3615
3616 if (threadIdx.x == 0) {
3617     for (int stage = 0; stage < kStages; ++stage) {
3618         init(&full[stage], kConsumerThreads + 1);
3619         init(&empty[stage], kConsumerThreads + 1);
3620     }
3621     tma_fence_proxy_async_shared_cta();
3622 }
3623 __syncthreads();
3624
3625 if (wg_idx == 0) {
3626     // 生产者 Warpgroup (wg_idx==0)
3627     if (wg_tid == 0) {
3628         int stage = 0;
3629         for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3630             if (stage == kStages)
3631                 stage = 0;
3632
3633             empty[stage].wait(empty[stage].arrive());
3634
3635             tma_load_2d(&s_a[stage * BM * BK], tensorMapA, k * BK, by * BM,
3636                 full[stage]);
3637             tma_load_2d(&s_b[stage * BN * BK], tensorMapB, k * BK, bx * BN,
3638                 full[stage]);
3639             tma_arrive_expect_tx(full[stage], (BM * BK + BN * BK) * sizeof(half));
3640         }
3641     }
3642 } else {
3643     // 消费者 Warpgroup (wg_idx==1)
3644     // 初始化：标记所有 stage 为"空" (可被 Producer 写入)
3645     for (int stage = 0; stage < kStages; ++stage) {
3646         [[maybe_unused]] auto token = empty[stage].arrive();
3647     }
3648
3649     const int warp_id = wg_tid / kWarpSize;
3650     const int lane_id = wg_tid % kWarpSize;
3651     // WG1 内形成 2x2 warp grid, warp_m / warp_n 分别表示 warp 在 M/N 方向的索引。
3652     const int warp_m = warp_id % kMmaTileM; // kMmaTileM = 2, {0,1}
3653     const int warp_n = warp_id / kMmaTileM; // kMmaTileN = 2, {0,1}

```

```

3654 uint32_t RA[kValTileM][4];
3655 uint32_t RB[kValTileN][2];
3656 uint32_t RC[kValTileM][kValTileN][2] = {0};
3657
3658 const uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
3659 const uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
3660 int stage = 0;
3661 for (int k = 0; k < NUM_K_TILES; ++k, ++stage) {
3662     if (stage == kStages)
3663         stage = 0;
3664
3665     full[stage].wait(full[stage].arrive());
3666     tma_fence_proxy_async_shared_cta();
3667
3668     // 与 cp.async pipeline (hgemm_mma_stages_tn 等) 不同, 这里 ldmatrix +
3669     // mma.sync 之前/后 不需要 __syncthreads, 原因在于 TMA + async proxy 的
3670     // 同步机制与 warp specialization 的线程分工:
3671     //
3672     // 1. 生产者-消费者解耦: TMA 拷贝由 producer (wg_idx==0 的单个线程)
3673     //    通过 tma_load_2d + tma_arrive_expect_tx 提交, 而非所有 256 个线程
3674     //    各自做 cp.async。cp.async 需要 __syncthreads 确保所有线程的异步
3675     //    拷贝都完成后再进入计算; 而 TMA 路径用 cuda::barrier 的 full/empty
3676     //    协议完成 CTA 级同步——full[stage].wait() 返回时, producer 已通过
3677     //    expect_tx 声明了完整的传输字节数, 且硬件保证这些字节已全部写入 smem。
3678     //
3679     // 2. async proxy fence 替代 __syncthreads: fence_proxy_async(space_shared)
3680     //    在 async proxy (TMA 写入通道) 和后续 smem load (ldmatrix 读取通道)
3681     //    之间建立内存序。它确保 fence 之前的所有 async proxy 写操作 (TMA)
3682     //    对 fence 之后的所有 smem 读操作 (ldmatrix) 可见。这是一个轻量级
3683     //    proxy 间 ordering, 不需要阻塞线程, 也不会让 warp 等待其他 warp。
3684     //
3685     // 3. 消费者内部 warp 间无依赖: WG1 内的 4 个 warp (2x2 grid) 各自独立
3686     //    读取 smem 中互不重叠的 A/B 区域 (warp_m 和 warp_n 分别索引不同的
3687     //    行区间)。它们之间不需要读写共享数据, 因此不需要 __syncthreads 来
3688     //    对齐 warp 间的执行进度。mma.sync 本身是 warp 级同步指令, 保证同一
3689     //    warp 内 32 个线程的 ldmatrix 和 mma 操作正确协作。
3690
3691     // 注意: 这里已经没有NUM_K_TILES循环了, 因为K loop load已经被WG0生产者处理了
3692 #pragma unroll
3693     for (int k_step = 0; k_step < kValTileK; ++k_step) {
3694
3695         #pragma unroll
3696         for (int i = 0; i < kValTileM; ++i) { // kValTileM = 4
3697             // kMmaM * kValTileM = 16*4 = 64, 每个消费者 warp 在 M 方向覆盖 64 行。
3698             const int warp_smem_a_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3699             const int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
3700             // 与 hgemm_mma_stages_tn_swizzle 不同: k_step * kMmaK 必须放在
3701             // lane_smem_a_k 中传给 tma_swizzle_128B(), 而非像 swizzle kernel
3702             // 那样作为 smem_k_offset 加在 swizzle 外部。原因:
3703             // tma_swizzle_128B 是 128B TMA swizzle, 作用于完整 BK=64,
3704             // swizzle 周期覆盖全部 64 列, k_step=0 和 k_step=1 的 chunk 会
3705             // 被 XOR 交叉混合 → k_step 偏移必须在 swizzle 内部参与 chunk 计算。
3706             // swizzle<kMmaK> 作用于 kMmaK=16, 每个 kMmaK slice 独立
3707             // swizzle, slice 之间互不跨越 → k_step * kMmaK 可以加在外部。
3708             const int lane_smem_a_k = (k_step * kMmaK) + (lane_id / 16) * 8;
3709             const uint32_t lane_smem_a_ptr = smem_a_base_ptr +
3710                 (stage * BM * BK + lane_smem_a_m * BK +
3711                  tma_swizzle_128B(lane_smem_a_m, lane_smem_a_k)) *
3712                 sizeof(half);
3713             LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
3714         }
3715
3716 #pragma unroll

```

```

3717     for (int j = 0; j < kValTileN; ++j) { // kValTileN = 8
3718         // kMmaN * kValTileN = 8*8 = 64, 每个消费者 warp 在 B^T 的 N 方向覆盖 64 行。
3719         const int warp_smem_b_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3720         const int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
3721         const int lane_smem_b_k = (k_step * kMmaK) + ((lane_id / 8) % 2) * 8;
3722         const uint32_t lane_smem_b_ptr = smem_b_base_ptr +
3723             (stage * BN * BK + lane_smem_b_n * BK +
3724              tma_swizzle_128B(lane_smem_b_n, lane_smem_b_k)) *
3725             sizeof(half);
3726         LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
3727     }
3728
3729 #pragma unroll
3730     for (int i = 0; i < kValTileM; ++i) {
3731 #pragma unroll
3732         for (int j = 0; j < kValTileN; ++j) {
3733             HMMMA16816(RC[i][j][0], RC[i][j][1],
3734                        RA[i][0], RA[i][1], RA[i][2], RA[i][3],
3735                        RB[j][0], RB[j][1],
3736                        RC[i][j][0], RC[i][j][1]);
3737         }
3738     }
3739 }
3740 [[maybe_unused]] auto token = empty[stage].arrive();
3741 }
3742
3743 // Epilogue: 复用 RA[0] 和 RA[1] (已在寄存器中) 作为 shuffle 缓冲,
3744 // 与 hgemmm_mma_stages_tn_swizzle 的 epilogue 模式一致。四个相邻
3745 // lane 各持有两个 uint32 fragment; shuffle 汇聚为每行一个 float4,
3746 // 由 lane 0 发出对齐的 128-bit store.
3747 #pragma unroll
3748     for (int i = 0; i < kValTileM; ++i) {
3749         const int store_warp_smem_c_m = warp_m * (kMmaM * kValTileM) + i * kMmaM;
3750         const int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
3751 #pragma unroll
3752         for (int j = 0; j < kValTileN; ++j) {
3753             const int store_warp_smem_c_n = warp_n * (kMmaN * kValTileN) + j * kMmaN;
3754             RA[0][0] = RC[i][j][0];
3755             RA[1][0] = RC[i][j][1];
3756             RA[0][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
3757             RA[0][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
3758             RA[0][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
3759             RA[1][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
3760             RA[1][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
3761             RA[1][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
3762             if (lane_id % 4 == 0) {
3763                 const int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
3764                 const int store_gmem_c_addr_0 =
3765                     store_lane_gmem_c_m * N + store_lane_gmem_c_n;
3766                 const int store_gmem_c_addr_1 =
3767                     (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;
3768                 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_0]) =
3769                     *reinterpret_cast<float4*>(&RA[0][0]);
3770                 *reinterpret_cast<float4*>(&C[store_gmem_c_addr_1]) =
3771                     *reinterpret_cast<float4*>(&RA[1][0]);
3772             }
3773         }
3774     }
3775 }
3776 }
3777 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
3778
3779 #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)

```

```

3780 // ---- Host-side TMA Tensor Map helpers (WGMMMA test) ----
3781 // 面试要点 (TMA descriptor 创建 — cuTensorMapEncodeTiled) :
3782 //   - TMA descriptor 描述 global memory 中矩形的 shape/stride/dtype,
3783 //     以及硬件搬运的 box (tile) 大小和 smem swizzle 模式
3784 //   - 关键易错点: TMA shape 参数写的是 (W,H) 而不是 (H,W)!
3785 //     对 row-major [M,K] 矩阵, TMA shape = (K,M), minor=K, major=M
3786 //   - cuTensorMapEncodeTiled 是 CUDA Driver API, 需 #include <cuda.h> 并链接 -lcuda
3787 //   - smem_box 维度顺序也是 (minor, major) = (BK, BM) 或 (BK, BN)
3788 //   - Swizzle 模式通常选 CU_TENSOR_MAP_SWIZZLE_128B 配合 WGMMMA 的 128B swizzle
3789 //
3790 // ★ gmem_prob_stride + 1 的含义:
3791 //   语法层面 — 数组名退化 + 指针算术:
3792 //     gmem_prob_stride 类型为 uint64_t[5], 在表达式中退化为 uint64_t*
3793 //     (指向首元素 gmem_prob_stride[0])。+ 1 偏移 1 个 uint64_t, 指向
3794 //     gmem_prob_stride[1], 等价于 &gmem_prob_stride[1]。
3795 //   语义层面 — 跳过隐式的最内维 stride:
3796 //     cuTensorMapEncodeTiled 的 globalStrides 参数只接收 tensorRank-1 个
3797 //     stride 值。最内维 (fastest-changing dimension) 的 stride 是隐式的,
3798 //     固定等于 sizeof(element_type), 不需要显式传入。
3799 //     对于 tensorRank=2 的 FP16 矩阵:
3800 //       dim 0 (内维, K) : stride = sizeof(half)    ← 隐式, API 不需要
3801 //       dim 1 (外维, M) : stride = sizeof(half)*K  ← 需要传给 API
3802 //     gmem_prob_stride 数组的布局是内维在前:
3803 //       [0] = sizeof(half)                        ← 内维, 不传
3804 //       [1] = sizeof(half) * BlockMinorSize * blocks_width ← 外维, 传给 API
3805 //     所以 + 1 的作用是跳过 [0], 让 API 从 [1] 开始读取, 正好得到外维 stride。
3806 //
3807 // ★ smem_box_stride 为什么不需要 +1?
3808 //   与 globalStrides 不同, smemBoxStrides 参数接收完整的 tensorRank 个 stride,
3809 //   最内维 stride 也必须显式传入 (不隐式)。
3810 //   smem_box_stride[5] = {1, 1, 1, 1, 1} 表示 smem tile 中所有维度元素
3811 //   都是连续存放的, 维度间没有 padding/stride。
3812 //   对于 tensorRank=2: strides[0]=1 (内维 stride), strides[1]=1 (外维 stride),
3813 //   即 smem 中第 (i,j) 元素的偏移 = i*1 + j*1 = i+j (元素连续排列)。
3814 //   所以这里直接传 smem_box_stride (即 &smem_box_stride[0]), API 会读到全部
3815 //   两个 stride 值, 不需要跳过任何一个。
3816 //
3817 // 参考: CUDA Programming Guide §TMA, PTX ISA §9.7.15.5, CUTLASS GmmaDescriptor
3818
3819 template <int BlockMajorSize, int BlockMinorSize>
3820 __host__ static inline void create_tensor_map(CUtensorMap *tma_map,
3821                                               half *gmem_ptr,
3822                                               int blocks_height,
3823                                               int blocks_width) {
3824     void *gmem_address = (void *)gmem_ptr;
3825     uint64_t gmem_prob_shape[5] = {(uint64_t)BlockMinorSize * blocks_width,
3826                                     (uint64_t)BlockMajorSize * blocks_height,
3827                                     1, 1, 1};
3828     uint64_t gmem_prob_stride[5] = {
3829         sizeof(half), sizeof(half) * BlockMinorSize * blocks_width, 0, 0, 0};
3830     uint32_t smem_box_shape[5] = {uint32_t(BlockMinorSize),
3831                                    uint32_t(BlockMajorSize), 1, 1, 1};
3832     uint32_t smem_box_stride[5] = {1, 1, 1, 1, 1};
3833     CUresult result = cuTensorMapEncodeTiled(
3834         tma_map, CU_TENSOR_MAP_DATA_TYPE_FLOAT16, 2, gmem_address,
3835         gmem_prob_shape, gmem_prob_stride + 1, smem_box_shape, smem_box_stride,
3836         CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
3837         CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);
3838     if (result != CUDA_SUCCESS)
3839         printf("cuTensorMapEncodeTiled failed: %d\n", (int)result);
3840 }
3841
3842 __host__ static inline CUtensorMap *allocate_and_create_tensor_map(

```

```

3843     half *src, int blocks_height, int blocks_width) {
3844     CUTensorMap *tma_map_d;
3845     cudaMalloc(&tma_map_d, sizeof(CUTensorMap));
3846     CUTensorMap tma_map_host;
3847     create_tensor_map<128, 64>(&tma_map_host, src, blocks_height, blocks_width);
3848     cudaMemcpy(tma_map_d, &tma_map_host, sizeof(CUTensorMap),
3849               cudaMemcpyHostToDevice);
3850     return tma_map_d;
3851 }
3852 #endif /* NOTES_V2_ENABLE_WGMMA || NOTES_V2_ENABLE_TMA_MMA_WS */
3853
3854 // =====
3855 // Phase 8: FlashAttention-2 + MMA m16n8k16)
3856 // =====
3857 // 面试题点 (FlashAttention-2算法) :
3858 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
3859 //    但 HBM 带宽是瓶颈 → FlashAttention-2用 tiling + online softmax 避免写回
3860 // 2. FA 三板斧:
3861 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqlen 分块 [Bc,d]
3862 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
3863 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
3864 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
3865 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
3866 //    - 优点: 减少 warp 间通信和 shuffle
3867 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
3868 //    for each K,V tile:
3869 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
3870 //        m_new = max(m_old, row_max(S_cur * scale))
3871 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
3872 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
3873 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
3874 //        O_final = O_new / l_final
3875 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
3876 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
3877 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
3878 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
3879 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
3880 //      都天然同构”的通用结论
3881 //
3882 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
3883 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
3884 // Grid: ((N + 63) / 64, B * H, 1), Br=64
3885 // Block: (128, 1, 1), kNumThreads=kWarpSize×kMmaTileSeqLenQ×kMmaTileSeqLenK=128
3886 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
3887
3888 // ---- 寄存器填充辅助函数 ----
3889 template <typename T, int M, const int N, const int K = 2>
3890 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
3891     #pragma unroll
3892     for (int i = 0; i < M; ++i)
3893         #pragma unroll
3894         for (int j = 0; j < N; ++j)
3895             #pragma unroll
3896             for (int k = 0; k < K; ++k)
3897                 R[i][j][k] = val;
3898 }
3899
3900 template <typename T, int M, const int N = 2>
3901 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
3902     #pragma unroll
3903     for (int i = 0; i < M; ++i)
3904         #pragma unroll
3905         for (int j = 0; j < N; ++j)

```



```

3906     R[i][j] = val;
3907 }
3908
3909 // =====
3910 // FlashAttention-2 Split-Q Kernel (完整实现)
3911 // =====
3912 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
3913 //
3914 // Tile 设计 (以 kHeadDim=64 为例) :
3915 //   Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ = 16*4*1 = 64
3916 //   Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK = 8*1*8 = 64
3917 //   Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
3918 //
3919 // 执行流程:
3920 //   1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
3921 //   2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
3922 //   3)   3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
3923 //   4)   3b: Q@K^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMMA16816
3924 //   5)   3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
3925 //       → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
3926 //   6)   3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
3927 //       → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
3928 //   7)   3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
3929 //   8) 最终 rescale: O_final = (1/l_final) * O_final
3930 //   9) Epilogue: warp shuffle + 128-bit collective store
3931 //
3932 // ★ Pad 与手动 XOR swizzle 的 profile 结论 (SM120, B=1,H=32,N=4096,D=64) :
3933 //   - XOR 能消除目标 ldmatrix lane pattern 的 bank conflict, 但收益是局部的;
3934 //     XOR 本身及 `x2` 的计算不是主要瓶颈, nvcc 已将该式化简为 bit operations。
3935 //   - compact Q/K/V XOR 改变了 cp.async 的 shared destination pattern: LDGSTS
3936 //     wavefronts 从 pad 的 30.15M 增至 68.16M (2.26x), long-scoreboard、
3937 //     LG-throttle、MIO-throttle 分别约为 pad 的 3.25x、2.74x、1.60x。
3938 //   - compact XOR 虽节省约 5 KiB smem, 但没有提高此 kernel 的 occupancy; 最终
3939 //     约 120.0 TFLOPS, 显著低于 Q/K/V kPad=8 的 166.6 TFLOPS。
3940 //   - 所以当前 kernel 默认对 Q/K/V 都用 kPad=8。保留 swizzle 路径是为了学习和消融: 评价
3941 //     shared layout 必须同时观察 ldmatrix reads 与 cp.async/LDGSTS writes, 不能只看
3942 //     bank-conflict counter, 也不能只优化 XOR 地址算术。
3943
3944 template <
3945     const int kHeadDim,           // head dim: 32, 64, 128
3946     const int kMmaAtomM,          // 16 (MMA instruction M dimension)
3947     const int kMmaAtomN,          // 8 (MMA instruction N dimension)
3948     const int kMmaAtomK,          // 16 (MMA instruction K dimension)
3949     const int kMmaTileSeqLenQ,    // MMA tiles along Q's M dim, 4 → Br=16*4=64
3950     const int kMmaTileSeqLenK,    // MMA tiles along K's N dim, 1 → Bc basis=8
3951     const int kMmaTileSeqLenP,    // MMA tiles for P@V M dim, must equal kMmaTileSeqLenQ
3952     const int kMmaTileHeadDimV,   // MMA tiles for P@V N dim (head dim direction)
3953     const int kValTileSeqLenQ,    // value tiles along Q's M, 1 → Br per warp=16
3954     const int kValTileSeqLenK,    // value tiles along K's N, 8 → Bc_warp=8*8=64
3955     const int kValTileSeqLenP,    // value tiles for P@V M dim, 1
3956     const int kValTileHeadDimV,   // value tiles for P@V N dim, kHeadDim/(8*kMmaTileHeadDimV)
3957     const int kStagesK,           // pipeline stages for K: >= 1; NO stages required for Q/V
3958     const int kPadQ,              // Q row padding; 0 selects compact XOR swizzle
3959     const int kPadK,              // K row padding; 0 selects compact XOR swizzle
3960     const int kPadV>             // V row padding; 0 selects compact XOR swizzle
3961 _global__ void flash_attn_mma_stages_split_q(
3962     half *Q, half *K, half *V, half *O, int N, int H) {
3963     static_assert(kStagesK >= 1, "kStagesK must be >= 1");
3964     static_assert(kPadQ >= 0 && kPadK >= 0 && kPadV >= 0,
3965         "Q/K/V padding must be non-negative");
3966     constexpr bool kSwizzleQ = kPadQ == 0;
3967     constexpr bool kSwizzleK = kPadK == 0;
3968     constexpr bool kSwizzleV = kPadV == 0;

```

```

3969 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ; // 64
3970 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK; // 64
3971 constexpr int kNumThreads = kWarpSize * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
3972 const int Tc = (N + Bc - 1) / Bc;
3973 // 原始实现默认 seqLen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
3974 // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
3975 const float scale = 1.0f / sqrtf((float)kHeadDim);
3976
3977 // Block indexing
3978 const int Nb_id = blockIdx.y / H; // batch id
3979 const int Nh_id = blockIdx.y % H; // head id
3980 const int Q_tile_id = blockIdx.x; // tile id along Q's M dim (Br)
3981 const int tid = threadIdx.x;
3982 const int warp_id = tid / kWarpSize;
3983 const int lane_id = tid % kWarpSize;
3984 const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
3985 const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
3986
3987 // Global memory base offsets for this (batch, head)
3988 // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
3989 const int Q_gmem_offset = (Nb_id * H * N + Nh_id * N) * kHeadDim;
3990 const int K_gmem_offset = Q_gmem_offset;
3991 const int V_gmem_offset = Q_gmem_offset;
3992 const int O_gmem_offset = Q_gmem_offset;
3993
3994 // Thread-to-smem mapping for cooperative load
3995 int load_smem_Q_Br = tid / (kNumThreads / Br);
3996 int load_smem_Q_d = (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
3997 int load_smem_K_Bc = tid / (kNumThreads / Bc);
3998 int load_smem_K_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
3999 int load_smem_V_Bc = tid / (kNumThreads / Bc);
4000 int load_smem_V_d = (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
4001
4002 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
4003 if (load_gmem_Q_Br >= N)
4004     return;
4005
4006 // ---- Shared memory layout ----
4007 // Q/K/V independently use padded row-major when kPad* > 0 and compact XOR
4008 // swizzle when kPad* == 0. Padding and XOR are never combined per operand.
4009 // The swizzled physical layout is [col / 16][row][16]; swizzle<16>() selects
4010 // the 0/8 phase inside the final 16-half tile.
4011 extern __shared__ half smem[];
4012 constexpr int Q_tile_size = Br * (kHeadDim + kPadQ);
4013 constexpr int K_tile_size = Bc * (kHeadDim + kPadK);
4014 constexpr int kSmemStrideQ = kHeadDim + kPadQ;
4015 constexpr int kSmemStrideK = kHeadDim + kPadK;
4016 constexpr int kSmemStrideV = kHeadDim + kPadV;
4017 half *Q_tile_smem = smem;
4018 half *K_tile_smem = Q_tile_smem + Q_tile_size;
4019 half *V_tile_smem = K_tile_smem + kStagesK * K_tile_size;
4020
4021 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
4022 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
4023 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
4024
4025 // ---- Online Softmax persistent state ----
4026 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
4027 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile i
4028 float lane_block_row_max_old[kValTileSeqLenQ][2];
4029 float lane_block_row_sum_old[kValTileSeqLenQ][2];
4030 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
4031 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);

```

```

4032 // ---- Register allocation ----
4033 uint32_t R_Q[kValTileSeqLenQ][4]; // Q regs
4034 uint32_t R_K[kValTileSeqLenK][2]; // K regs
4035 uint32_t R_V[kValTileHeadDimV][2]; // V regs
4036 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
4037 // 对单个 m16n8k16 tile 而言：
4038 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
4039 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
4040 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
4041 uint32_t R_S[kValTileSeqLenQ][kValTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
4042 uint32_t R_O[kValTileSeqLenP][kValTileHeadDimV][2]; // O for current tile
4043 uint32_t R_D[kValTileSeqLenP][kValTileHeadDimV]
4044         [2]; // O accumulator (final output)
4045
4046 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
4047 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_D, 0);
4048
4049 // =====
4050 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
4051 // =====
4052 {
4053     int load_gmem_Q_addr =
4054         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
4055     if constexpr (kSwizzleQ) {
4056 #pragma unroll
4057         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4058             int col = load_smem_Q_d + i;
4059             uint32_t ptr = smem_Q_base_ptr +
4060                 ((col / kMmaAtomK) * Br * kMmaAtomK +
4061                  load_smem_Q_Br * kMmaAtomK +
4062                  swizzle<kMmaAtomK>(load_smem_Q_Br, col % kMmaAtomK)) *
4063                 sizeof(half);
4064             CP_ASYNC_CG(ptr, &Q[load_gmem_Q_addr + i], 16);
4065         }
4066     } else {
4067         uint32_t load_smem_Q_ptr =
4068             smem_Q_base_ptr +
4069             (load_smem_Q_Br * kSmemStrideQ + load_smem_Q_d) * sizeof(half);
4070 #pragma unroll
4071         for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
4072             CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
4073         }
4074     }
4075 }
4076 CP_ASYNC_COMMIT_GROUP();
4077 }
4078
4079 // =====
4080 // Step 2: 预加载前 (kStagesK-1) 个 K tile (多 stage pipeline 预热)
4081 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqLen 遍历始终从 tile 0 开始,
4082 // 后续在外循环里不断递增至 tile 1/2/3/.../Tc-1。
4083 // =====
4084 if constexpr (kStagesK > 1) {
4085 #pragma unroll
4086     for (int stage = 0; stage < (kStagesK - 1); ++stage) {
4087         int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
4088         int load_gmem_K_addr =
4089             K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4090         if constexpr (kSwizzleK) {
4091 #pragma unroll
4092             for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4093                 int col = load_smem_K_d + i;
4094                 uint32_t ptr = smem_K_base_ptr +

```

```

4095         (stage * K_tile_size +
4096         (col / kMmaAtomK) * Bc * kMmaAtomK +
4097         load_smem_K_Bc * kMmaAtomK +
4098         swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4099         sizeof(half);
4100     CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4101 }
4102 } else {
4103     uint32_t load_smem_K_ptr =
4104         smem_K_base_ptr +
4105         (stage * K_tile_size + load_smem_K_Bc * kSmemStrideK +
4106         load_smem_K_d) *
4107         sizeof(half);
4108 #pragma unroll
4109     for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4110         CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4111     }
4112 }
4113 CP_ASYNC_COMMIT_GROUP();
4114 }
4115 CP_ASYNC_WAIT_GROUP(kStagesK - 2);
4116 __syncthreads();
4117 }
4118
4119 // =====
4120 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
4121 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
4122 // =====
4123 #pragma unroll 1
4124 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
4125     int smem_sel = tile_K_seqlen % kStagesK;
4126     int smem_sel_next = (tile_K_seqlen + (kStagesK - 1)) % kStagesK;
4127
4128     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
4129     if constexpr (kStagesK > 1) {
4130         // 只有 kStagesK>1 才能真正做 K 的 pipeline:
4131         // smem_sel 负责 “当前正在计算” 的 K tile, smem_sel_next 负责 “下一轮预取” 的 K tile。
4132         // 若 kStagesK=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
4133         // Load current V tile (no pipeline for V — one stage is enough)
4134         {
4135             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
4136             int load_gmem_V_addr =
4137                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
4138             if constexpr (kSwizzleV) {
4139 #pragma unroll
4140                 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4141                     int col = load_smem_V_d + i;
4142                     uint32_t ptr = smem_V_base_ptr +
4143                         ((col / kMmaAtomK) * Bc * kMmaAtomK +
4144                         load_smem_V_Bc * kMmaAtomK +
4145                         swizzle<kMmaAtomK>(load_smem_V_Bc, col % kMmaAtomK)) *
4146                         sizeof(half);
4147                     CP_ASYNC_CG(ptr, &V[load_gmem_V_addr + i], 16);
4148                 }
4149             } else {
4150                 uint32_t load_smem_V_ptr =
4151                     smem_V_base_ptr +
4152                     (load_smem_V_Bc * kSmemStrideV + load_smem_V_d) * sizeof(half);
4153 #pragma unroll
4154                 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4155                     CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
4156                 }
4157             }

```

```

4158     CP_ASYNC_COMMIT_GROUP();
4159 }
4160
4161 // Prefetch next K tile (pipelined)
4162 if ((tile_K_seqlen + 1) < Tc) {
4163     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
4164     int load_gmem_K_addr =
4165         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
4166     if constexpr (kSwizzleK) {
4167 #pragma unroll
4168         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4169             int col = load_smem_K_d + i;
4170             uint32_t ptr = smem_K_base_ptr +
4171                 (smem_sel_next * K_tile_size +
4172                 (col / kMmaAtomK) * Bc * kMmaAtomK +
4173                 load_smem_K_Bc * kMmaAtomK +
4174                 swizzle<kMmaAtomK>(load_smem_K_Bc, col % kMmaAtomK)) *
4175                 sizeof(half);
4176             CP_ASYNC_CG(ptr, &K[load_gmem_K_addr + i], 16);
4177         }
4178     } else {
4179         uint32_t load_smem_K_ptr =
4180             smem_K_base_ptr +
4181             (smem_sel_next * K_tile_size +
4182             load_smem_K_Bc * kSmemStrideK +
4183             load_smem_K_d) *
4184             sizeof(half);
4185 #pragma unroll
4186         for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
4187             CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
4188         }
4189     }
4190     CP_ASYNC_COMMIT_GROUP();
4191 }
4192 }
4193
4194 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
4195 fill_3D_regs<uint32_t, kValTileSeqLenQ, kValTileSeqLenK, 2>(R_S, 0);
4196 #pragma unroll
4197 for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
4198     // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
4199 #pragma unroll
4200     for (int i = 0; i < kValTileSeqLenQ; ++i) {
4201         int warp_smem_Q_Br =
4202             warp_QP * (kMmaAtomM * kValTileSeqLenQ) + i * kMmaAtomM;
4203         int lane_smem_Q_Br =
4204             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
4205         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
4206         uint32_t lane_smem_Q_ptr;
4207         if constexpr (kSwizzleQ) {
4208             lane_smem_Q_ptr = smem_Q_base_ptr +
4209                 ((lane_smem_Q_d / kMmaAtomK) * Br * kMmaAtomK +
4210                 lane_smem_Q_Br * kMmaAtomK +
4211                 swizzle<kMmaAtomK>(lane_smem_Q_Br,
4212                 lane_smem_Q_d % kMmaAtomK)) *
4213                 sizeof(half);
4214         } else {
4215             lane_smem_Q_ptr = smem_Q_base_ptr +
4216                 (lane_smem_Q_Br * kSmemStrideQ + lane_smem_Q_d) * sizeof(half);
4217         }
4218         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
4219             lane_smem_Q_ptr);
4220     }

```

```

4221 // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
4222 // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
4223 #pragma unroll
4224 for (int j = 0; j < kValTileSeqLenK; ++j) {
4225     int warp_smem_K_Bc =
4226         warp_KV * (kMmaAtomN * kValTileSeqLenK) + j * kMmaAtomN;
4227     int lane_smem_K_Bc =
4228         warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
4229     int lane_smem_K_d =
4230         tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
4231     uint32_t lane_smem_K_ptr;
4232     if constexpr (kSwizzleK) {
4233         lane_smem_K_ptr = smem_K_base_ptr +
4234             (smem_sel * K_tile_size +
4235              (lane_smem_K_d / kMmaAtomK) * Bc * kMmaAtomK +
4236              lane_smem_K_Bc * kMmaAtomK +
4237              swizzle<kMmaAtomK>(lane_smem_K_Bc,
4238                               lane_smem_K_d % kMmaAtomK)) *
4239             sizeof(half);
4240     } else {
4241         lane_smem_K_ptr = smem_K_base_ptr +
4242             (smem_sel * K_tile_size + lane_smem_K_Bc * kSmemStrideK +
4243              lane_smem_K_d) *
4244             sizeof(half);
4245     }
4246     LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
4247 }
4248
4249 // MMA: S[tile] += Q[tile] @ K^T[tile]
4250 #pragma unroll
4251 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4252     #pragma unroll
4253     for (int j = 0; j < kValTileSeqLenK; ++j) {
4254         HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
4255                  R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
4256                  R_S[i][j][1]);
4257     }
4258 }
4259 } // end loop over d
4260 __syncthreads();
4261
4262 // =====
4263 // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to R_S
4264 // =====
4265 // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
4266 // - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
4267 // - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
4268 // - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
4269 // - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
4270 // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
4271 // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
4272 // Each (i, j) pair = one 16x8 MMA tile; there are kValTileSeqLenQ x
4273 // kValTileSeqLenK tiles.
4274
4275 float lane_row_max_new[kValTileSeqLenQ][2];
4276 float lane_row_sum_new[kValTileSeqLenQ][2];
4277 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
4278 fill_2D_regs<float, kValTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
4279
4280 // Pass 1: Thread-level reduce max across all columns (kValTileSeqLenK tiles)
4281 #pragma unroll
4282 for (int i = 0; i < kValTileSeqLenQ; ++i) {

```



```

4284 #pragma unroll
4285 for (int j = 0; j < kValTileSeqLenK; ++j) {
4286     // Extract half values from R_S registers (C matrix fragment layout)
4287     float2 t_reg_S_0 =
4288         __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
4289     float2 t_reg_S_1 =
4290         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
4291     // S = (Q@K^T) * scale
4292     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
4293     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
4294     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
4295     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
4296 }
4297 // Warp-level reduce max (kWarpWidth = 4 for Q@K^T — only lanes
4298 // {0,4,8,...,28} hold valid data)
4299 lane_row_max_new[i][0] =
4300     warp_reduce_max<4, float>(lane_row_max_new[i][0]);
4301 lane_row_max_new[i][1] =
4302     warp_reduce_max<4, float>(lane_row_max_new[i][1]);
4303 }
4304
4305 // Pass 2: Compute P = exp(S*scale - m_new), store back to R_S
4306 // 面试关键点: 这里将 P 写回 R_S 寄存器!
4307 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
4308 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
4309 #pragma unroll
4310 for (int i = 0; i < kValTileSeqLenQ; ++i) {
4311     // m_new = max(m_old, m_cur)
4312     float block_row_max_new_0 =
4313         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
4314     float block_row_max_new_1 =
4315         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
4316
4317     #pragma unroll
4318     for (int j = 0; j < kValTileSeqLenK; ++j) {
4319         float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
4320         float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
4321
4322         // P = exp(S * scale - m_new), 用 fma 保证精度
4323         t_reg_S_0.x =
4324             __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
4325         t_reg_S_0.y =
4326             __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
4327         t_reg_S_1.x =
4328             __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
4329         t_reg_S_1.y =
4330             __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
4331
4332         // Accumulate row sums
4333         lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
4334         lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
4335
4336         // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
4337         HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
4338         HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
4339     }
4340
4341     // Warp-level reduce sum (kWarpWidth = 4, same as max)
4342     lane_row_sum_new[i][0] =
4343         warp_reduce_sum<4, float>(lane_row_sum_new[i][0]);
4344     lane_row_sum_new[i][1] =
4345         warp_reduce_sum<4, float>(lane_row_sum_new[i][1]);
4346 }

```

```

4347 __syncthreads();
4348
4349 // =====
4350 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = 0[Br,d]
4351 // =====
4352 // Wait for V to be ready before computing P@V
4353 if constexpr (kStagesK > 1) {
4354     if ((tile_K_seqlen + 1) < Tc) {
4355         CP_ASYNC_WAIT_GROUP(1);
4356     } else {
4357         CP_ASYNC_WAIT_GROUP(0);
4358     }
4359 } else {
4360     CP_ASYNC_WAIT_GROUP(0);
4361 }
4362 __syncthreads();
4363
4364 fill_3D_regs<uint32_t, kValTileSeqLenP, kValTileHeadDimV, 2>(R_0, 0);
4365
4366 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
4367 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
4368 #pragma unroll
4369 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
4370     // ldmatrix.x2.trans: load V[Bc,d] with transposition
4371     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
4372     // transposed ldmatrix
4373 #pragma unroll
4374 for (int j = 0; j < kValTileHeadDimV; ++j) {
4375     int warp_smem_V_d =
4376         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4377     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
4378     int lane_smem_V_d = warp_smem_V_d;
4379     uint32_t lane_smem_V_ptr;
4380     if constexpr (kSwizzleV) {
4381         lane_smem_V_ptr = smem_V_base_ptr +
4382             ((lane_smem_V_d / kMmaAtomK) * Bc * kMmaAtomK +
4383              lane_smem_V_Bc * kMmaAtomK +
4384              swizzle<kMmaAtomK>(lane_smem_V_Bc,
4385                               lane_smem_V_d % kMmaAtomK)) *
4386             sizeof(half);
4387     } else {
4388         lane_smem_V_ptr = smem_V_base_ptr +
4389             (lane_smem_V_Bc * kSmemStrideV + lane_smem_V_d) * sizeof(half);
4390     }
4391     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
4392 }
4393
4394 // P matrix layout in R_S[i][j][2]:
4395 // MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
4396 // | 64x64 | warp_KV 0 |
4397 // | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
4398 // | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
4399 // | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
4400 // | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
4401 // tile_V_Bc selects which 16-column slice of P to use:
4402 // tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
4403 // tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
4404 // tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
4405 // tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
4406 // 对应的 MMA A fragment 布局可以这样记:
4407 // - rows 0~7: lane 0~3 -> {a0,a1} 与 {a4,a5}, lane 4~7 -> 下一行, 依次类推
4408 // - rows 8~15: lane 0~3 -> {a2,a3} 与 {a6,a7}, lane 4~7 -> 下一行, 依次类推
4409 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,

```

```

4410 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
4411 // fragment 都无条件成立的通用结论。layout转换逻辑:
4412 // C layout: 8 x (16 x 8) layout -> A layout: 4 x (16 x 16) layout
4413 int w = tile_V_Bc * 2;
4414 #pragma unroll
4415 for (int i = 0; i < kValTileSeqLenP; ++i) {
4416 #pragma unroll
4417     for (int j = 0; j < kValTileHeadDimV; ++j) {
4418         HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
4419                 R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
4420                 R_V[j][0], R_V[j][1], // B fragment = V
4421                 R_0[i][j][0], R_0[i][j][1]); // C fragment output
4422     }
4423 }
4424 } // end for tile_V_Bc
4425 __syncthreads();
4426
4427 // =====
4428 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
4429 // =====
4430 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 l 的 rescale
4431 #pragma unroll
4432 for (int i = 0; i < kValTileSeqLenP; ++i) {
4433     float block_row_max_new_0 = lane_row_max_new[i][0];
4434     float block_row_max_new_1 = lane_row_max_new[i][1];
4435     float block_row_sum_new_0 = lane_row_sum_new[i][0];
4436     float block_row_sum_new_1 = lane_row_sum_new[i][1];
4437
4438     float block_row_max_old_0 = lane_block_row_max_old[i][0];
4439     float block_row_max_old_1 = lane_block_row_max_old[i][1];
4440
4441     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
4442     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
4443
4444     // Handle first iteration: m_old = -inf, need to use m_new directly
4445     block_row_max_old_0 =
4446         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
4447     block_row_max_old_1 =
4448         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
4449
4450     float rescale_o_factor_0 =
4451         __expf(block_row_max_old_0 - block_row_max_new_0);
4452     float rescale_o_factor_1 =
4453         __expf(block_row_max_old_1 - block_row_max_new_1);
4454
4455     // Rescale 0_old + Add P@V in one fused step
4456 #pragma unroll
4457     for (int j = 0; j < kValTileHeadDimV; ++j) {
4458         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
4459         // reg[0] -> rows 0~7 的 {c0,c1}
4460         // reg[1] -> rows 8~15 的 {c2,c3}
4461         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
4462         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
4463         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4464         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4465
4466         // 0_new = exp(m_old - m_new) * 0_old + P@V (fused multiply-add)
4467         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
4468         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
4469         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
4470         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
4471
4472         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);

```

```

4473     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4474 }
4475
4476 // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
4477 float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
4478 float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
4479 lane_block_row_sum_old[i][0] = __fmaf_rn(
4480     rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
4481 lane_block_row_sum_old[i][1] = __fmaf_rn(
4482     rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
4483
4484 // Update m
4485 lane_block_row_max_old[i][0] = block_row_max_new_0;
4486 lane_block_row_max_old[i][1] = block_row_max_new_1;
4487 }
4488
4489 // Wait for next K tile to be ready in smem before next iteration
4490 if constexpr (kStagesK > 1) {
4491     if ((tile_K_seqlen + 1) < Tc) {
4492         CP_ASYNC_WAIT_GROUP(0);
4493     }
4494     __syncthreads();
4495 }
4496 } // end outer loop over K seqlen
4497 __syncthreads();
4498
4499 // =====
4500 // Step 4: 最终 rescale — 0_final = (1/l_final) * 0_final
4501 // =====
4502 #pragma unroll
4503 for (int i = 0; i < kValTileSeqLenP; ++i) {
4504     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
4505     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
4506 #pragma unroll
4507     for (int j = 0; j < kValTileHeadDimV; ++j) {
4508         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
4509         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
4510         t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
4511         t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
4512         t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
4513         t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
4514         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
4515         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
4516     }
4517 }
4518
4519 // =====
4520 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
4521 // =====
4522 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
4523 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
4524 #pragma unroll
4525 for (int i = 0; i < kValTileSeqLenP; ++i) {
4526 #pragma unroll
4527     for (int j = 0; j < kValTileHeadDimV; ++j) {
4528         uint32_t R_Z[2][4];
4529         R_Z[0][0] = R_D[i][j][0];
4530         R_Z[1][0] = R_D[i][j][1];
4531         R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
4532         R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
4533         R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
4534         R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
4535         R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);

```

```

4536 R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
4537
4538 // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
4539 if (lane_id % 4 == 0) {
4540     int store_warp_regs_0_Br =
4541         warp_QP * (kMmaAtomM * kValTileSeqLenP) + i * kMmaAtomM;
4542     int store_lane_gmem_0_Br =
4543         Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
4544     int store_warp_regs_0_d =
4545         warp_KV * (kMmaAtomN * kValTileHeadDimV) + j * kMmaAtomN;
4546     int store_lane_gmem_0_d = store_warp_regs_0_d;
4547     int store_gmem_0_addr_0 = 0_gmem_offset +
4548         (store_lane_gmem_0_Br + 0) * kHeadDim +
4549         store_lane_gmem_0_d;
4550     int store_gmem_0_addr_1 = 0_gmem_offset +
4551         (store_lane_gmem_0_Br + 8) * kHeadDim +
4552         store_lane_gmem_0_d;
4553     // LDST128BITS = reinterpret_cast<float4*>
4554     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
4555         *reinterpret_cast<float4*>(&R_Z[0][0]);
4556     *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
4557         *reinterpret_cast<float4*>(&R_Z[1][0]);
4558 }
4559 }
4560 }
4561 }
4562
4563 // =====
4564 // 以下是测试代码，验证 Phase 1 - Phase 8 的kernel的正确性，不评估性能。
4565 // =====
4566
4567 static inline void check(cudaError_t err, const char *msg) {
4568     if (err != cudaSuccess) {
4569         fprintf(stderr, "[ERROR] %s: %s\n", msg, cudaGetErrorString(err));
4570         exit(EXIT_FAILURE);
4571     }
4572 }
4573
4574
4575 static void test_block_reduce(int N) {
4576
4577     srand(42);
4578     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4579     for (int i = 0; i < N; i++)
4580         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4581
4582     // CPU reference: sum of all elements
4583     double ref = 0.0;
4584     for (int i = 0; i < N; i++) ref += (double)h_a[i];
4585
4586     float *d_a, *d_y;
4587     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "blockreduce alloc A");
4588     check(cudaMalloc(&d_y, sizeof(float)), "blockreduce alloc Y");
4589
4590     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "blockreduce H2D A");
4591     check(cudaMemset(d_y, 0, sizeof(float)), "blockreduce zero Y");
4592
4593     dim3 block(128);
4594     dim3 grid((N + 127) / 128);
4595     block_reduce_all<128><<<grid, block>>>(d_a, d_y, N);
4596     check(cudaGetLastError(), "blockreduce launch");
4597     check(cudaDeviceSynchronize(), "blockreduce sync");
4598

```

```

4599 float result;
4600 check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "blockreduce D2H");
4601
4602 float err = fabsf(result - (float)ref);
4603 printf("| %-42s | %.6e | %-4s | %-8s |\n", "BlockReduce", err,
4604         err < 1e-2f ? "PASS" : "FAIL", "None");
4605
4606 free(h_a);
4607 cudaFree(d_a); cudaFree(d_y);
4608 }
4609
4610
4611 static void test_dot(int N) {
4612
4613     srand(42);
4614     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4615     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4616     for (int i = 0; i < N; i++) {
4617         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4618         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4619     }
4620
4621     // CPU reference
4622     double ref = 0.0;
4623     for (int i = 0; i < N; i++) ref += (double)h_a[i] * (double)h_b[i];
4624
4625     float *d_a, *d_b, *d_y;
4626     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "dot alloc A");
4627     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "dot alloc B");
4628     check(cudaMalloc(&d_y, sizeof(float)), "dot alloc Y");
4629
4630     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D A");
4631     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "dot H2D B");
4632     check(cudaMemset(d_y, 0, sizeof(float)), "dot zero Y");
4633
4634     dim3 block(128);
4635     dim3 grid((N + 127) / 128);
4636     dot<128><<<grid, block>>>(d_a, d_b, d_y, N);
4637     check(cudaGetLastError(), "dot launch");
4638     check(cudaDeviceSynchronize(), "dot sync");
4639
4640     float result;
4641     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot D2H");
4642
4643     float err = fabsf(result - (float)ref);
4644     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot", err,
4645         err < 1e-2f ? "PASS" : "FAIL", "None");
4646
4647     // ---- Dot Vec4 ----
4648     check(cudaMemset(d_y, 0, sizeof(float)), "dot_vec4 zero Y");
4649     dim3 block_v4(32);
4650     dot_vec4<32><<<grid, block_v4>>>(d_a, d_b, d_y, N);
4651     check(cudaGetLastError(), "dot_vec4 launch");
4652     check(cudaDeviceSynchronize(), "dot_vec4 sync");
4653
4654     check(cudaMemcpy(&result, d_y, sizeof(float), cudaMemcpyDeviceToHost), "dot_vec4 D2H");
4655     float err_v4 = fabsf(result - (float)ref);
4656     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Dot-Vec4", err_v4, err_v4 < 1e-2f ? "PASS" : "FAIL", "None");
4657
4658     free(h_a); free(h_b);
4659     cudaFree(d_a); cudaFree(d_b); cudaFree(d_y);
4660 }
4661

```



```

4662 static void test_relu(int N) {
4663     srand(42);
4664     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4665     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4666     for (int i = 0; i < N; i++)
4667         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4668
4669     float *d_x, *d_y;
4670     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "relu alloc X");
4671     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "relu alloc Y");
4672     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "relu H2D");
4673
4674     for (int i = 0; i < N; i++) h_y[i] = fmaxf(0.0f, h_x[i]);
4675     dim3 block256(256);
4676     dim3 grid256((N + 255) / 256);
4677     relu<<<grid256, block256>>>(d_x, d_y, N);
4678     check(cudaGetLastError(), "relu launch");
4679     check(cudaDeviceSynchronize(), "relu sync");
4680     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu D2H");
4681     float max_err = 0.0f;
4682     for (int i = 0; i < N; i++) {
4683         float expected = fmaxf(0.0f, h_x[i]);
4684         float err = fabsf(h_y[i] - expected);
4685         if (err > max_err) max_err = err;
4686     }
4687     printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4688
4689     dim3 block64(64);
4690     relu_vec4<<<grid256, block64>>>(d_x, d_y, N);
4691     check(cudaGetLastError(), "relu_vec4 launch");
4692     check(cudaDeviceSynchronize(), "relu_vec4 sync");
4693     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "relu_vec4 D2H");
4694     max_err = 0.0f;
4695     for (int i = 0; i < N; i++) {
4696         float expected = fmaxf(0.0f, h_x[i]);
4697         float err = fabsf(h_y[i] - expected);
4698         if (err > max_err) max_err = err;
4699     }
4700     printf("| %-42s | %.6e | %-4s | %-8s |\n", "ReLU-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
4701     );
4702
4703     free(h_x); free(h_y);
4704     cudaFree(d_x); cudaFree(d_y);
4705 }
4706
4707 static void test_elementwise(int N) {
4708     srand(42);
4709     float *h_a = (float *)malloc((size_t)N * sizeof(float));
4710     float *h_b = (float *)malloc((size_t)N * sizeof(float));
4711     for (int i = 0; i < N; i++) {
4712         h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4713         h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4714     }
4715
4716     float *d_a, *d_b, *d_c;
4717     check(cudaMalloc(&d_a, (size_t)N * sizeof(float)), "eadd alloc A");
4718     check(cudaMalloc(&d_b, (size_t)N * sizeof(float)), "eadd alloc B");
4719     check(cudaMalloc(&d_c, (size_t)N * sizeof(float)), "eadd alloc C");
4720     check(cudaMemcpy(d_a, h_a, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D A");
4721     check(cudaMemcpy(d_b, h_b, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "eadd H2D B");
4722
4723

```

```

4724 dim3 block256(256);
4725 dim3 grid256((N + 255) / 256);
4726 elementwise_add<<<grid256, block256>>>(d_a, d_b, d_c, N);
4727 check(cudaGetLastError(), "eadd launch");
4728 check(cudaDeviceSynchronize(), "eadd sync");
4729 float *h_c = (float *)malloc((size_t)N * sizeof(float));
4730 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd D2H");
4731 float max_err = 0.0f;
4732 for (int i = 0; i < N; i++) {
4733     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4734     if (err > max_err) max_err = err;
4735 }
4736 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4737
4738 dim3 block64(64);
4739 check(cudaMemset(d_c, 0, (size_t)N * sizeof(float)), "eadd_vec4 zero C");
4740 elementwise_add_vec4<<<grid256, block64>>>(d_a, d_b, d_c, N);
4741 check(cudaGetLastError(), "eadd_vec4 launch");
4742 check(cudaDeviceSynchronize(), "eadd_vec4 sync");
4743 check(cudaMemcpy(h_c, d_c, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "eadd_vec4 D2H");
4744 max_err = 0.0f;
4745 for (int i = 0; i < N; i++) {
4746     float err = fabsf(h_c[i] - (h_a[i] + h_b[i]));
4747     if (err > max_err) max_err = err;
4748 }
4749 printf("| %-42s | %.6e | %-4s | %-8s |\n", "ElemwiseAdd-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
4750
4751 free(h_a); free(h_b); free(h_c);
4752 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
4753 }
4754
4755
4756 static void test_histogram(int N) {
4757     srand(42);
4758     int BINS = 16;
4759     int *h_hist = (int *)calloc(BINS, sizeof(int));
4760     int *h_hist_ref = (int *)calloc(BINS, sizeof(int));
4761     int *h_idx = (int *)malloc((size_t)N * sizeof(int));
4762     for (int i = 0; i < N; i++) h_idx[i] = rand() % BINS;
4763     for (int i = 0; i < N; i++) h_hist_ref[h_idx[i]]++;
4764
4765     int *d_idx, *d_hist;
4766     check(cudaMalloc(&d_idx, (size_t)N * sizeof(int)), "hist alloc idx");
4767     check(cudaMalloc(&d_hist, BINS * sizeof(int)), "hist alloc hist");
4768     check(cudaMemcpy(d_idx, h_idx, (size_t)N * sizeof(int), cudaMemcpyHostToDevice), "hist H2D idx");
4769     check(cudaMemset(d_hist, 0, BINS * sizeof(int)), "hist zero");
4770
4771     dim3 block256(256);
4772     dim3 grid256((N + 255) / 256);
4773     histogram<<<grid256, block256>>>(d_idx, d_hist, N);
4774     check(cudaGetLastError(), "histogram launch");
4775     check(cudaDeviceSynchronize(), "histogram sync");
4776     check(cudaMemcpy(h_hist, d_hist, BINS * sizeof(int), cudaMemcpyDeviceToHost), "hist D2H");
4777
4778     float max_err = 0.0f;
4779     for (int i = 0; i < BINS; i++) {
4780         float err = fabsf((float)(h_hist[i] - h_hist_ref[i]));
4781         if (err > max_err) max_err = err;
4782     }
4783     printf("| %-42s | %.6e | %-4s | %-8s |\n", "Histogram", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
    );

```

```

4784 free(h_hist); free(h_hist_ref); free(h_idx);
4785 cudaFree(d_idx); cudaFree(d_hist);
4786 }
4787
4788
4789
4790 static void test_merge_attn_states(int num_tokens, int num_heads,
4791                                   int head_size) {
4792
4793     size_t out_size = (size_t)num_tokens * num_heads * head_size * sizeof(float);
4794     size_t lse_size = (size_t)num_heads * num_tokens * sizeof(float);
4795
4796     float *h_prefix_out = (float *)malloc(out_size);
4797     float *h_suffix_out = (float *)malloc(out_size);
4798     float *h_prefix_lse = (float *)malloc(lse_size);
4799     float *h_suffix_lse = (float *)malloc(lse_size);
4800     float *h_output_ref = (float *)malloc(out_size);
4801
4802     srand(42);
4803     for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4804         h_prefix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4805         h_suffix_out[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4806     }
4807     for (int i = 0; i < num_heads * num_tokens; i++) {
4808         h_prefix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4809         h_suffix_lse[i] = ((float)rand() / RAND_MAX) * 20.0f - 10.0f;
4810     }
4811
4812     // CPU reference: 与 kernel 完全相同的浮点计算
4813     for (int t = 0; t < num_tokens; t++) {
4814         for (int h = 0; h < num_heads; h++) {
4815             float p_lse = h_prefix_lse[h * num_tokens + t];
4816             float s_lse = h_suffix_lse[h * num_tokens + t];
4817             p_lse = isinf(p_lse) ? -INFINITY : p_lse;
4818             s_lse = isinf(s_lse) ? -INFINITY : s_lse;
4819
4820             float max_lse = fmaxf(p_lse, s_lse);
4821             p_lse -= max_lse;
4822             s_lse -= max_lse;
4823             float p_se = expf(p_lse);
4824             float s_se = expf(s_lse);
4825             float p_scale = p_se / (p_se + s_se);
4826             float s_scale = s_se / (p_se + s_se);
4827
4828             int head_off = t * num_heads * head_size + h * head_size;
4829             for (int d = 0; d < head_size; d++) {
4830                 h_output_ref[head_off + d] =
4831                     h_prefix_out[head_off + d] * p_scale +
4832                     h_suffix_out[head_off + d] * s_scale;
4833             }
4834         }
4835     }
4836
4837     float *d_prefix_out, *d_suffix_out, *d_prefix_lse, *d_suffix_lse, *d_output;
4838     check(cudaMalloc(&d_prefix_out, out_size), "merge_attn alloc p_out");
4839     check(cudaMalloc(&d_suffix_out, out_size), "merge_attn alloc s_out");
4840     check(cudaMalloc(&d_prefix_lse, lse_size), "merge_attn alloc p_lse");
4841     check(cudaMalloc(&d_suffix_lse, lse_size), "merge_attn alloc s_lse");
4842     check(cudaMalloc(&d_output, out_size), "merge_attn alloc output");
4843
4844     check(cudaMemcpy(d_prefix_out, h_prefix_out, out_size,
4845                     cudaMemcpyHostToDevice),
4846           "merge_attn H2D p_out");

```

```

4847 check(cudaMemcpy(d_suffix_out, h_suffix_out, out_size,
4848               cudaMemcpyHostToDevice),
4849       "merge_attn H2D s_out");
4850 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4851               cudaMemcpyHostToDevice),
4852       "merge_attn H2D p_lse");
4853 check(cudaMemcpy(d_suffix_lse, h_suffix_lse, lse_size,
4854               cudaMemcpyHostToDevice),
4855       "merge_attn H2D s_lse");
4856
4857 int threads_per_head = head_size / 4;
4858 int total_threads = num_tokens * num_heads * threads_per_head;
4859 dim3 block(128);
4860 dim3 grid((total_threads + 127) / 128);
4861 merge_attn_states<<<grid, block>>>(<
4862     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4863     num_tokens, num_heads, head_size);
4864 check(cudaGetLastError(), "merge_attn launch");
4865 check(cudaDeviceSynchronize(), "merge_attn sync");
4866
4867 float *h_output = (float *)malloc(out_size);
4868 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4869       "merge_attn D2H");
4870
4871 float max_err = 0.0f;
4872 for (int i = 0; i < num_tokens * num_heads * head_size; i++) {
4873     float err = fabsf(h_output[i] - h_output_ref[i]);
4874     if (err > max_err) max_err = err;
4875 }
4876 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates", max_err,
4877       max_err < 1e-4f ? "PASS" : "FAIL", "None");
4878
4879 // 边界测试: +inf LSE → 权重退化为 0 (空 attention 段)
4880 h_prefix_lse[0] = INFINITY; // head 0, token 0 的 LSE = +inf
4881 h_suffix_lse[0] = 0.0f;
4882 check(cudaMemcpy(d_prefix_lse, h_prefix_lse, lse_size,
4883               cudaMemcpyHostToDevice),
4884       "merge_attn H2D inf lse");
4885 merge_attn_states<<<grid, block>>>(<
4886     d_output, d_prefix_out, d_prefix_lse, d_suffix_out, d_suffix_lse,
4887     num_tokens, num_heads, head_size);
4888 check(cudaGetLastError(), "merge_attn inf launch");
4889 check(cudaDeviceSynchronize(), "merge_attn inf sync");
4890 check(cudaMemcpy(h_output, d_output, out_size, cudaMemcpyDeviceToHost),
4891       "merge_attn inf D2H");
4892
4893 // token 0, head 0 的所有元素应等于 suffix_output (prefix 权重  $\alpha=0$ )
4894 float inf_err = 0.0f;
4895 for (int d = 0; d < head_size; d++) {
4896     float err = fabsf(h_output[d] - h_suffix_out[d]);
4897     if (err > inf_err) inf_err = err;
4898 }
4899 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MergeAttnStates-inf", inf_err,
4900       inf_err < 1e-4f ? "PASS" : "FAIL", "None");
4901
4902 free(h_prefix_out);
4903 free(h_suffix_out);
4904 free(h_prefix_lse);
4905 free(h_suffix_lse);
4906 free(h_output_ref);
4907 free(h_output);
4908 cudaFree(d_prefix_out);
4909 cudaFree(d_suffix_out);

```

```

4910 cudaFree(d_prefix_lse);
4911 cudaFree(d_suffix_lse);
4912 cudaFree(d_output);
4913 }
4914
4915
4916 static void test_softmax(int N) {
4917     // Use N = blockDim.x so one block processes all elements as one token.
4918     constexpr int kNumThreads = 256;
4919     if (N != kNumThreads) {
4920         printf("    OnlineSafeSoftmax: N must be %d (got %d), skipping.\n", kNumThreads, N);
4921         return;
4922     }
4923
4924     srand(42);
4925     float *h_x = (float *)malloc((size_t)N * sizeof(float));
4926     float *h_y_ref = (float *)malloc((size_t)N * sizeof(float));
4927     for (int i = 0; i < N; i++)
4928         h_x[i] = ((float)rand() / RAND_MAX) * 10.0f - 5.0f; // [-5, 5]
4929
4930     // CPU reference: softmax(x_i) = exp(x_i - max) / sum(exp(x_j - max))
4931     float max_val = -FLT_MAX;
4932     for (int i = 0; i < N; i++) if (h_x[i] > max_val) max_val = h_x[i];
4933     double sum_exp = 0.0;
4934     for (int i = 0; i < N; i++) sum_exp += (double)expf(h_x[i] - max_val);
4935     for (int i = 0; i < N; i++)
4936         h_y_ref[i] = expf(h_x[i] - max_val) / (float)sum_exp;
4937
4938     float *d_x, *d_y;
4939     check(cudaMalloc(&d_x, (size_t)N * sizeof(float)), "softmax alloc X");
4940     check(cudaMalloc(&d_y, (size_t)N * sizeof(float)), "softmax alloc Y");
4941
4942     check(cudaMemcpy(d_x, h_x, (size_t)N * sizeof(float), cudaMemcpyHostToDevice), "softmax H2D X");
4943
4944     dim3 block(256);
4945     dim3 grid(1); // one token covering all N elements
4946     online_safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4947     check(cudaGetLastError(), "softmax launch");
4948     check(cudaDeviceSynchronize(), "softmax sync");
4949
4950     float *h_y = (float *)malloc((size_t)N * sizeof(float));
4951     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "softmax D2H");
4952
4953     float max_err = 0.0f;
4954     for (int i = 0; i < N; i++) {
4955         float err = fabsf(h_y[i] - h_y_ref[i]);
4956         if (err > max_err) max_err = err;
4957     }
4958     printf("| %-42s | %.6e | %-4s | %-8s |\n", "OnlineSafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL",
4959         , "None");
4960
4961     // ---- Safe Softmax ----
4962     safe_softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4963     check(cudaGetLastError(), "safe_softmax launch");
4964     check(cudaDeviceSynchronize(), "safe_softmax sync");
4965     check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "safe_softmax D2H");
4966     max_err = 0.0f;
4967     for (int i = 0; i < N; i++) {
4968         float err = fabsf(h_y[i] - h_y_ref[i]);
4969         if (err > max_err) max_err = err;
4970     }
4971     printf("| %-42s | %.6e | %-4s | %-8s |\n", "SafeSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");

```

```

4971 // ---- Naive Softmax ----
4972 softmax_per_token<<<grid, block>>>(d_x, d_y, N);
4973 check(cudaGetLastError(), "naive_softmax launch");
4974 check(cudaDeviceSynchronize(), "naive_softmax sync");
4975 check(cudaMemcpy(h_y, d_y, (size_t)N * sizeof(float), cudaMemcpyDeviceToHost), "naive_softmax D2H");
4976 max_err = 0.0f;
4977 for (int i = 0; i < N; i++) {
4978     float err = fabsf(h_y[i] - h_y_ref[i]);
4979     if (err > max_err) max_err = err;
4980 }
4981 printf("| %-42s | %.6e | %-4s | %-8s |\n", "NaiveSoftmax", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
4982     None");
4983
4984 free(h_x); free(h_y); free(h_y_ref);
4985 cudaFree(d_x); cudaFree(d_y);
4986 }
4987
4988 static void test_rms_norm(int N, int K) {
4989
4990     srand(42);
4991     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
4992     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
4993     for (int i = 0; i < N * K; i++)
4994         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
4995     float g = 1.5f; // gain
4996
4997     // CPU reference: y = (x / rms(x)) * g
4998     float epsilon = 1e-5f;
4999     for (int n = 0; n < N; n++) {
5000         double sum_sq = 0.0;
5001         for (int k = 0; k < K; k++) sum_sq += (double)h_x[n * K + k] * (double)h_x[n * K + k];
5002         float rms = sqrtf((float)sum_sq / (float)K + epsilon);
5003         for (int k = 0; k < K; k++)
5004             h_y_ref[n * K + k] = (h_x[n * K + k] / rms) * g;
5005     }
5006
5007     float *d_x, *d_y;
5008     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "rmsnorm alloc X");
5009     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "rmsnorm alloc Y");
5010     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "rmsnorm H2D X");
5011
5012     dim3 block(128);
5013     dim3 grid(N);
5014     rms_norm<<<grid, block>>>(d_x, d_y, g, N, K);
5015     check(cudaGetLastError(), "rmsnorm launch");
5016     check(cudaDeviceSynchronize(), "rmsnorm sync");
5017
5018     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
5019     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm D2H");
5020
5021     float max_err = 0.0f;
5022     for (int i = 0; i < N * K; i++) {
5023         float err = fabsf(h_y[i] - h_y_ref[i]);
5024         if (err > max_err) max_err = err;
5025     }
5026     printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5027
5028     // ---- RMS Norm Vec4 ----
5029     dim3 block_rv4(32);
5030     rms_norm_vec4<<<grid, block_rv4>>>(d_x, d_y, g, N, K);
5031     check(cudaGetLastError(), "rmsnorm_vec4 launch");

```



```

5033 check(cudaDeviceSynchronize(), "rmsnorm_vec4 sync");
5034 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "rmsnorm_vec4 D2H");
5035 max_err = 0.0f;
5036 for (int i = 0; i < N * K; i++) {
5037     float err = fabsf(h_y[i] - h_y_ref[i]);
5038     if (err > max_err) max_err = err;
5039 }
5040 printf("| %-42s | %.6e | %-4s | %-8s |\n", "RMSNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
5041
5042 free(h_x); free(h_y); free(h_y_ref);
5043 cudaFree(d_x); cudaFree(d_y);
5044 }
5045
5046
5047 static void test_layer_norm(int N, int K) {
5048
5049     srand(42);
5050     float *h_x = (float *)malloc((size_t)N * K * sizeof(float));
5051     float *h_y_ref = (float *)malloc((size_t)N * K * sizeof(float));
5052     for (int i = 0; i < N * K; i++)
5053         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5054     float g = 1.5f, b = 0.3f; // gain and bias
5055
5056     // CPU reference: y = ((x - mean) / std) * g + b
5057     float epsilon = 1e-5f;
5058     for (int n = 0; n < N; n++) {
5059         double sum = 0.0;
5060         for (int k = 0; k < K; k++) sum += (double)h_x[n * K + k];
5061         float mean = (float)sum / (float)K;
5062         double sum_sq = 0.0;
5063         for (int k = 0; k < K; k++) {
5064             float diff = h_x[n * K + k] - mean;
5065             sum_sq += (double)diff * (double)diff;
5066         }
5067         float std = sqrtf((float)sum_sq / (float)K + epsilon);
5068         for (int k = 0; k < K; k++)
5069             h_y_ref[n * K + k] = ((h_x[n * K + k] - mean) / std) * g + b;
5070     }
5071
5072     float *d_x, *d_y;
5073     check(cudaMalloc(&d_x, (size_t)N * K * sizeof(float)), "layernorm alloc X");
5074     check(cudaMalloc(&d_y, (size_t)N * K * sizeof(float)), "layernorm alloc Y");
5075     check(cudaMemcpy(d_x, h_x, (size_t)N * K * sizeof(float), cudaMemcpyHostToDevice), "layernorm H2D X");
5076
5077     dim3 block(128);
5078     dim3 grid(N);
5079     layer_norm<<<grid, block>>>(d_x, d_y, g, b, N, K);
5080     check(cudaGetLastError(), "layernorm launch");
5081     check(cudaDeviceSynchronize(), "layernorm sync");
5082
5083     float *h_y = (float *)malloc((size_t)N * K * sizeof(float));
5084     check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm D2H");
5085
5086     float max_err = 0.0f;
5087     for (int i = 0; i < N * K; i++) {
5088         float err = fabsf(h_y[i] - h_y_ref[i]);
5089         if (err > max_err) max_err = err;
5090     }
5091     printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None"
        );
5092
5093     // ---- Layer Norm Vec4 ----

```

```

5094 dim3 block_lv4(32);
5095 layer_norm_vec4<<<grid, block_lv4>>>(d_x, d_y, g, b, N, K);
5096 check(cudaGetLastError(), "layernorm_vec4 launch");
5097 check(cudaDeviceSynchronize(), "layernorm_vec4 sync");
5098 check(cudaMemcpy(h_y, d_y, (size_t)N * K * sizeof(float), cudaMemcpyDeviceToHost), "layernorm_vec4 D2H");
5099 max_err = 0.0f;
5100 for (int i = 0; i < N * K; i++) {
5101     float err = fabsf(h_y[i] - h_y_ref[i]);
5102     if (err > max_err) max_err = err;
5103 }
5104 printf("| %-42s | %.6e | %-4s | %-8s |\n", "LayerNorm-Vec4", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "
    None");
5105
5106 free(h_x); free(h_y); free(h_y_ref);
5107 cudaFree(d_x); cudaFree(d_y);
5108 }
5109
5110
5111 static void test_rope(int seq_len, int N) {
5112
5113     int total_pairs = seq_len * N;
5114     int total_elems = total_pairs * 2;
5115     size_t size = (size_t)total_elems * sizeof(float);
5116
5117     float *h_x = (float *)malloc(size);
5118     float *h_y_ref = (float *)malloc(size);
5119
5120     srand(42);
5121     for (int i = 0; i < total_elems; i++)
5122         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5123
5124     // CPU reference: 2D rotation for each pair
5125     for (int idx = 0; idx < total_pairs; idx++) {
5126         int token_pos = idx / N;
5127         int token_idx = idx % N;
5128         float x1 = h_x[idx * 2];
5129         float x2 = h_x[idx * 2 + 1];
5130         float theta = 1.0f / powf(10000.0f, 2.0f * token_idx / (N * 2.0f));
5131         float angle = (float)token_pos * theta;
5132         float cos_v = cosf(angle);
5133         float sin_v = sinf(angle);
5134         h_y_ref[idx * 2] = x1 * cos_v - x2 * sin_v;
5135         h_y_ref[idx * 2 + 1] = x1 * sin_v + x2 * cos_v;
5136     }
5137
5138     float *d_x, *d_y;
5139     check(cudaMalloc(&d_x, size), "rope alloc X");
5140     check(cudaMalloc(&d_y, size), "rope alloc Y");
5141     check(cudaMemcpy(d_x, h_x, size, cudaMemcpyHostToDevice), "rope H2D");
5142
5143     dim3 block(256);
5144     dim3 grid((total_pairs + 255) / 256);
5145     rope<<<grid, block>>>(d_x, d_y, seq_len, N);
5146     check(cudaGetLastError(), "rope launch");
5147     check(cudaDeviceSynchronize(), "rope sync");
5148
5149     float *h_y = (float *)malloc(size);
5150     check(cudaMemcpy(h_y, d_y, size, cudaMemcpyDeviceToHost), "rope D2H");
5151
5152     float max_err = 0.0f;
5153     for (int i = 0; i < total_elems; i++) {
5154         float err = fabsf(h_y[i] - h_y_ref[i]);
5155         if (err > max_err) max_err = err;

```

```

5156 }
5157 printf("| %-42s | %.6e | %-4s | %-8s |\n", "RoPE", max_err, max_err < 1e-4f ? "PASS" : "FAIL", "None");
5158
5159 free(h_x);
5160 free(h_y);
5161 free(h_y_ref);
5162 cudaFree(d_x);
5163 cudaFree(d_y);
5164 }
5165
5166
5167 static void test_mat_transpose(int row, int col) {
5168
5169     size_t size_in = (size_t)row * col * sizeof(float);
5170     size_t size_out = (size_t)col * row * sizeof(float);
5171
5172     float *h_x = (float *)malloc(size_in);
5173     float *h_y_ref = (float *)malloc(size_out);
5174
5175     srand(42);
5176     for (int i = 0; i < row * col; i++)
5177         h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5178
5179     // CPU reference: y[j][i] = x[i][j] (row-major)
5180     for (int i = 0; i < row; i++)
5181         for (int j = 0; j < col; j++)
5182             h_y_ref[j * row + i] = h_x[i * col + j];
5183
5184     float *d_x, *d_y;
5185     check(cudaMalloc(&d_x, size_in), "mattrans alloc X");
5186     check(cudaMalloc(&d_y, size_out), "mattrans alloc Y");
5187     check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans H2D");
5188
5189     dim3 block(16, 16);
5190     dim3 grid((col + 15) / 16, (row + 15) / 16);
5191     mat_transpose<<<grid, block>>>(d_x, d_y, row, col);
5192     check(cudaGetLastError(), "mattrans launch");
5193     check(cudaDeviceSynchronize(), "mattrans sync");
5194
5195     float *h_y = (float *)malloc(size_out);
5196     check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans D2H");
5197
5198     float max_err = 0.0f;
5199     for (int i = 0; i < col * row; i++) {
5200         float err = fabsf(h_y[i] - h_y_ref[i]);
5201         if (err > max_err) max_err = err;
5202     }
5203     printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTranspose", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "None");
5204
5205     free(h_x);
5206     free(h_y);
5207     free(h_y_ref);
5208     cudaFree(d_x);
5209     cudaFree(d_y);
5210 }
5211
5212
5213 static void test_mat_transpose_padded(int row, int col) {
5214
5215     size_t size_in = (size_t)row * col * sizeof(float);
5216     size_t size_out = (size_t)col * row * sizeof(float);
5217

```

```

5218 float *h_x = (float *)malloc(size_in);
5219 float *h_y_ref = (float *)malloc(size_out);
5220
5221 srand(42);
5222 for (int i = 0; i < row * col; i++)
5223     h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5224
5225 // CPU reference: y[j][i] = x[i][j] (row-major)
5226 for (int i = 0; i < row; i++)
5227     for (int j = 0; j < col; j++)
5228         h_y_ref[j * row + i] = h_x[i * col + j];
5229
5230 float *d_x, *d_y;
5231 check(cudaMalloc(&d_x, size_in), "mattrans_padded alloc X");
5232 check(cudaMalloc(&d_y, size_out), "mattrans_padded alloc Y");
5233 check(cudaMemcpy(d_x, h_x, size_in, cudaMemcpyHostToDevice), "mattrans_padded H2D");
5234
5235 dim3 block(16, 16);
5236 dim3 grid((col + 15) / 16, (row + 63) / 64);
5237 mat_transpose_padded<<<grid, block>>>(d_x, d_y, row, col);
5238 check(cudaGetLastError(), "mattrans_padded launch");
5239 check(cudaDeviceSynchronize(), "mattrans_padded sync");
5240
5241 float *h_y = (float *)malloc(size_out);
5242 check(cudaMemcpy(h_y, d_y, size_out, cudaMemcpyDeviceToHost), "mattrans_padded D2H");
5243
5244 float max_err = 0.0f;
5245 for (int i = 0; i < col * row; i++) {
5246     float err = fabsf(h_y[i] - h_y_ref[i]);
5247     if (err > max_err) max_err = err;
5248 }
5249 printf("| %-42s | %.6e | %-4s | %-8s |\n", "MatTransposePadded", max_err, max_err < 1e-6f ? "PASS" : "FAIL", "None");
5250
5251 free(h_x);
5252 free(h_y);
5253 free(h_y_ref);
5254 cudaFree(d_x);
5255 cudaFree(d_y);
5256 }
5257
5258
5259 static void test_sgemv(int M, int K) {
5260
5261     srand(42);
5262     float *h_a = (float *)malloc((size_t)M * K * sizeof(float));
5263     float *h_x = (float *)malloc((size_t)K * sizeof(float));
5264     float *h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5265     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5266     for (int i = 0; i < K; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5267
5268     // CPU reference: y[m] = sum_k A[m,k] * x[k]
5269     for (int m = 0; m < M; m++) {
5270         double sum = 0.0;
5271         for (int k = 0; k < K; k++) sum += (double)h_a[m * K + k] * (double)h_x[k];
5272         h_y_ref[m] = (float)sum;
5273     }
5274
5275     float *d_a, *d_x, *d_y;
5276     check(cudaMalloc(&d_a, (size_t)M * K * sizeof(float)), "sgemv alloc A");
5277     check(cudaMalloc(&d_x, (size_t)K * sizeof(float)), "sgemv alloc X");
5278     check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv alloc Y");
5279

```

```

5280 check(cudaMemcpy(d_a, h_a, (size_t)M * K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D A");
5281 check(cudaMemcpy(d_x, h_x, (size_t)K * sizeof(float), cudaMemcpyHostToDevice), "sgemv H2D X");
5282
5283 dim3 block(32, 4);
5284 dim3 grid((M + 3) / 4);
5285 sgemv_k128<<<grid, block>>>(d_a, d_x, d_y, M, K);
5286 check(cudaGetLastError(), "sgemv launch");
5287 check(cudaDeviceSynchronize(), "sgemv sync");
5288
5289 float *h_y = (float *)malloc((size_t)M * sizeof(float));
5290 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv D2H");
5291
5292 float max_err = 0.0f;
5293 for (int m = 0; m < M; m++) {
5294     float err = fabsf(h_y[m] - h_y_ref[m]);
5295     if (err > max_err) max_err = err;
5296 }
5297 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K128", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5298
5299 // ---- SGEMV K32 ----
5300 check(cudaMemset(d_y, 0, M * sizeof(float)), "sgemv_k32 zero Y");
5301 sgemv_k32<<<grid, block>>>(d_a, d_x, d_y, M, K);
5302 check(cudaGetLastError(), "sgemv_k32 launch");
5303 check(cudaDeviceSynchronize(), "sgemv_k32 sync");
5304
5305 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k32 D2H");
5306
5307 max_err = 0.0f;
5308 for (int m = 0; m < M; m++) {
5309     float err = fabsf(h_y[m] - h_y_ref[m]);
5310     if (err > max_err) max_err = err;
5311 }
5312 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K32", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5313
5314 // ---- SGEMV K16 ----
5315 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5316 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5317
5318 int K16 = 16;
5319 h_a = (float *)malloc((size_t)M * K16 * sizeof(float));
5320 h_x = (float *)malloc((size_t)K16 * sizeof(float));
5321 h_y_ref = (float *)malloc((size_t)M * sizeof(float));
5322 for (int i = 0; i < M * K16; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5323 for (int i = 0; i < K16; i++) h_x[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5324
5325 for (int m = 0; m < M; m++) {
5326     double sum = 0.0;
5327     for (int k = 0; k < K16; k++) sum += (double)h_a[m * K16 + k] * (double)h_x[k];
5328     h_y_ref[m] = (float)sum;
5329 }
5330
5331 check(cudaMalloc(&d_a, (size_t)M * K16 * sizeof(float)), "sgemv_k16 alloc A");
5332 check(cudaMalloc(&d_x, (size_t)K16 * sizeof(float)), "sgemv_k16 alloc X");
5333 check(cudaMalloc(&d_y, (size_t)M * sizeof(float)), "sgemv_k16 alloc Y");
5334
5335 check(cudaMemcpy(d_a, h_a, (size_t)M * K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D A");
5336 check(cudaMemcpy(d_x, h_x, (size_t)K16 * sizeof(float), cudaMemcpyHostToDevice), "sgemv_k16 H2D X");
5337
5338 dim3 grid_k16((M + 7) / 8);
5339 sgemv_k16<2><<<grid_k16, block>>>(d_a, d_x, d_y, M, K16);
5340 check(cudaGetLastError(), "sgemv_k16 launch");

```

```

5341 check(cudaDeviceSynchronize(), "sgemv_k16 sync");
5342
5343 h_y = (float *)malloc((size_t)M * sizeof(float));
5344 check(cudaMemcpy(h_y, d_y, (size_t)M * sizeof(float), cudaMemcpyDeviceToHost), "sgemv_k16 D2H");
5345
5346 max_err = 0.0f;
5347 for (int m = 0; m < M; m++) {
5348     float err = fabsf(h_y[m] - h_y_ref[m]);
5349     if (err > max_err) max_err = err;
5350 }
5351 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMV-K16", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None"
5352 );
5353
5354 free(h_a); free(h_x); free(h_y); free(h_y_ref);
5355 cudaFree(d_a); cudaFree(d_x); cudaFree(d_y);
5356 }
5357
5358 static void test_sgemm(int M, int N, int K) {
5359
5360     size_t size_a = (size_t)M * K * sizeof(float);
5361     size_t size_b = (size_t)K * N * sizeof(float);
5362     size_t size_c = (size_t)M * N * sizeof(float);
5363
5364     float *h_a = (float *)malloc(size_a);
5365     float *h_b = (float *)malloc(size_b);
5366     float *h_c_ref = (float *)malloc(size_c);
5367
5368     srand(42);
5369     for (int i = 0; i < M * K; i++) h_a[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5370     for (int i = 0; i < K * N; i++) h_b[i] = ((float)rand() / RAND_MAX) * 2.0f - 1.0f;
5371
5372     float *d_a, *d_b, *d_c;
5373     check(cudaMalloc(&d_a, size_a), "sgemm alloc A");
5374     check(cudaMalloc(&d_b, size_b), "sgemm alloc B");
5375     check(cudaMalloc(&d_c, size_c), "sgemm alloc C");
5376
5377     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "sgemm H2D A");
5378     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "sgemm H2D B");
5379
5380     // cuBLAS reference (row-major idiom: swap M/N, swap A/B)
5381     cublasHandle_t handle;
5382     cublasCreate(&handle);
5383     float alpha = 1.0f, beta = 0.0f;
5384     cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5385                 &alpha, d_b, N, d_a, K, &beta, d_c, N);
5386     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H ref");
5387
5388     // Kernel
5389     cudaEvent_t start, stop;
5390     cudaEventCreate(&start);
5391     cudaEventCreate(&stop);
5392
5393     dim3 block(32, 32);
5394     dim3 grid((N + 31) / 32, (M + 31) / 32);
5395     sgemm<<<grid, block>>>(d_a, d_b, d_c, M, N, K);
5396     check(cudaGetLastError(), "sgemm launch");
5397     check(cudaDeviceSynchronize(), "sgemm sync");
5398
5399     float *h_c = (float *)malloc(size_c);
5400     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm D2H");
5401
5402     // Verify

```



```

5403 float max_err = 0.0f;
5404 for (int i = 0; i < M * N; i++) {
5405     float err = fabsf(h_c[i] - h_c_ref[i]);
5406     if (err > max_err) max_err = err;
5407 }
5408 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5409
5410 // ---- SGEMM Vec4 (128x128 tile, 4x4 thread tile) ----
5411
5412 dim3 grid_vec4((N + 127) / 128, (M + 127) / 128);
5413 check(cudaMemset(d_c, 0, size_c), "sgemm_vec4 zero C");
5414 sgemm_vec4<<<grid_vec4, block>>>(d_a, d_b, d_c, M, N, K);
5415 check(cudaGetLastError(), "sgemm_vec4 launch");
5416 check(cudaDeviceSynchronize(), "sgemm_vec4 sync");
5417
5418 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "sgemm_vec4 D2H");
5419
5420 max_err = 0.0f;
5421 for (int i = 0; i < M * N; i++) {
5422     float err = fabsf(h_c[i] - h_c_ref[i]);
5423     if (err > max_err) max_err = err;
5424 }
5425 printf("| %-42s | %.6e | %-4s | %-8s |\n", "SGEMM-Vec4", max_err, max_err < 1e-2f ? "PASS" : "FAIL", "None");
5426
5427 free(h_a); free(h_b); free(h_c); free(h_c_ref);
5428 cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
5429 cublasDestroy(handle);
5430 cudaEventDestroy(start);
5431 cudaEventDestroy(stop);
5432 }
5433
5434
5435 static void test_hgemmm_mma(int M, int N, int K) {
5436
5437     size_t size_a = (size_t)M * K * sizeof(half);
5438     size_t size_b = (size_t)K * N * sizeof(half);
5439     size_t size_c = (size_t)M * N * sizeof(half);
5440
5441     half *h_a = (half *)malloc(size_a);
5442     half *h_b = (half *)malloc(size_b);
5443     half *h_c_ref = (half *)malloc(size_c);
5444
5445     srand(42);
5446     for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5447     for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5448
5449     // Kernel expects B^T [N^K] row-major layout (TN layout convention).
5450     // We store h_b as B [K^N] row-major for cuBLAS, then create B^T for kernel.
5451     size_t size_b_t = (size_t)N * K * sizeof(half);
5452     half *h_b_t = (half *)malloc(size_b_t);
5453     for (int n = 0; n < N; n++)
5454         for (int k = 0; k < K; k++)
5455             h_b_t[n * K + k] = h_b[k * N + n];
5456
5457     half *d_a, *d_b, *d_b_t, *d_c;
5458     check(cudaMalloc(&d_a, size_a), "hgemmm alloc A");
5459     check(cudaMalloc(&d_b, size_b), "hgemmm alloc B (cuBLAS)");
5460     check(cudaMalloc(&d_b_t, size_b_t), "hgemmm alloc B_t (kernel)");
5461     check(cudaMalloc(&d_c, size_c), "hgemmm alloc C");
5462
5463     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemmm H2D A");
5464     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemmm H2D B (cuBLAS)");

```

```

5465 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm H2D B_t (kernel)");
5466
5467 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5468 // Note: use CUBLAS_COMPUTE_16F to match the kernel's f16.f16.f16.f16 accumulation.
5469 cublasHandle_t handle;
5470 cublasCreate(&handle);
5471 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5472 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5473             &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5474             &beta_h, d_c, CUDA_R_16F, N,
5475             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5476 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H ref");
5477
5478 // MMA kernel (TN layout: A row-major, B^T row-major = B col-major)
5479 cudaEvent_t start, stop;
5480 cudaEventCreate(&start);
5481 cudaEventCreate(&stop);
5482
5483 constexpr int BM = 128, BN = 128, BK = 16, kStages = 3;
5484 size_t smem_bytes = kStages * (BM * BK + BN * BK) * sizeof(half); // 24576
5485 dim3 block(256);
5486 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5487 hgemm_mma_stages_tn<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5488 check(cudaGetLastError(), "hgemm launch");
5489 check(cudaDeviceSynchronize(), "hgemm sync");
5490
5491 half *h_c = (half *)malloc(size_c);
5492 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm D2H");
5493
5494 // Verify
5495 float max_err = 0.0f;
5496 for (int i = 0; i < M * N; i++) {
5497     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5498     if (err > max_err) max_err = err;
5499 }
5500 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM MMA", max_err, max_err < 1.0f ? "PASS" : "FAIL", "None")
5501 ;
5502
5503 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5504 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5505 cublasDestroy(handle);
5506 cudaEventDestroy(start);
5507 cudaEventDestroy(stop);
5508 }
5509
5510 static void test_hgemm_swizzle(int M, int N, int K) {
5511     // HGEMM MMA Swizzle — m16n8k16 + multistage pipeline + TN 布局 + XOR swizzle
5512     // + Register Double Buffering (kValTileK=2, BK=32)
5513     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5514     // Kernel: hgemm_mma_stages_tn_swizzle with default template params
5515     // (kValTileK=2, kStages=3, BK=32)
5516     // smem: kStages × (BM×BK + BN×BK) halves
5517
5518     size_t size_a = (size_t)M * K * sizeof(half);
5519     size_t size_b = (size_t)K * N * sizeof(half);
5520     size_t size_c = (size_t)M * N * sizeof(half);
5521
5522     half *h_a = (half *)malloc(size_a);
5523     half *h_b = (half *)malloc(size_b);
5524     half *h_c_ref = (half *)malloc(size_c);
5525
5526     srand(42);

```

```

5527 for (int i = 0; i < M * K; i++) h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5528 for (int i = 0; i < K * N; i++) h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5529
5530 // Kernel expects B^T [N×K] row-major layout (TN layout convention).
5531 size_t size_b_t = (size_t)N * K * sizeof(half);
5532 half *h_b_t = (half *)malloc(size_b_t);
5533 for (int n = 0; n < N; n++)
5534     for (int k = 0; k < K; k++)
5535         h_b_t[n * K + k] = h_b[k * N + n];
5536
5537 half *d_a, *d_b, *d_b_t, *d_c;
5538 check(cudaMalloc(&d_a, size_a), "hgemm_swizzle alloc A");
5539 check(cudaMalloc(&d_b, size_b), "hgemm_swizzle alloc B (cuBLAS)");
5540 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_swizzle alloc B_t (kernel)");
5541 check(cudaMalloc(&d_c, size_c), "hgemm_swizzle alloc C");
5542
5543 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_swizzle H2D A");
5544 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B (cuBLAS)");
5545 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_swizzle H2D B_t (kernel)");
5546
5547 // cuBLAS FP16 reference
5548 cublasHandle_t handle;
5549 cublasCreate(&handle);
5550 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5551 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K,
5552              &alpha_h, d_b, CUDA_R_16F, N, d_a, CUDA_R_16F, K,
5553              &beta_h, d_c, CUDA_R_16F, N,
5554              CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5555 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H ref");
5556
5557 // MMA swizzle kernel (default params: kStages=3, kValTileK=2, BK=32)
5558 constexpr int BM = 128, BN = 128, BK = 32, K_STAGE_S = 3;
5559 size_t smem_bytes = K_STAGE_S * (BM * BK + BN * BK) * sizeof(half);
5560 dim3 block(256);
5561 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5562 hgemm_mma_stages_tn_swizzle<<<grid, block, smem_bytes>>>(d_a, d_b_t, d_c, M, N, K);
5563 check(cudaGetLastError(), "hgemm_swizzle launch");
5564 check(cudaDeviceSynchronize(), "hgemm_swizzle sync");
5565
5566 half *h_c = (half *)malloc(size_c);
5567 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_swizzle D2H");
5568
5569 // Verify
5570 float max_err = 0.0f;
5571 for (int i = 0; i < M * N; i++) {
5572     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5573     if (err > max_err) max_err = err;
5574 }
5575 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM Swizzle + Reg2x", max_err, max_err < 1.0f ? "PASS" : "
5576 FAIL", "None");
5577
5578 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5579 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5580 cublasDestroy(handle);
5581 }
5582
5583 #if defined(NOTES_V2_ENABLE_CUTE)
5584 static void test_hgemm_cute(int M, int N, int K) {
5585     // HGEMM CuTe — SM80_16x8x16_F16F16F16F16_TN + Swizzle<3,3> + kStage=2
5586     // TN layout: C[M×N] = A[M×K] × B^T[N×K]
5587     // Kernel: hgemm_mma_stages_tn_cute via launch_hgemm_mma_stages_tn_cute
5588     // Tile: BM=128, BN=256, BK=32, 128 threads/block

```

```

5589
5590 size_t size_a = (size_t)M * K * sizeof(half);
5591 size_t size_b = (size_t)K * N * sizeof(half);
5592 size_t size_c = (size_t)M * N * sizeof(half);
5593
5594 half *h_a = (half *)malloc(size_a);
5595 half *h_b = (half *)malloc(size_b);
5596 half *h_c_ref = (half *)malloc(size_c);
5597
5598 srand(42);
5599 for (int i = 0; i < M * K; i++)
5600     h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5601 for (int i = 0; i < K * N; i++)
5602     h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5603
5604 // CuTe kernel expects B^T [N×K] row-major (TN layout).
5605 size_t size_b_t = (size_t)N * K * sizeof(half);
5606 half *h_b_t = (half *)malloc(size_b_t);
5607 for (int n = 0; n < N; n++)
5608     for (int k = 0; k < K; k++)
5609         h_b_t[n * K + k] = h_b[k * N + n];
5610
5611 half *d_a, *d_b, *d_b_t, *d_c;
5612 check(cudaMalloc(&d_a, size_a), "hgemm_cute alloc A");
5613 check(cudaMalloc(&d_b, size_b), "hgemm_cute alloc B (cuBLAS)");
5614 check(cudaMalloc(&d_b_t, size_b_t), "hgemm_cute alloc B_t (kernel)");
5615 check(cudaMalloc(&d_c, size_c), "hgemm_cute alloc C");
5616
5617 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "hgemm_cute H2D A");
5618 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "hgemm_cute H2D B (cuBLAS)");
5619 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "hgemm_cute H2D B_t (kernel)");
5620
5621 // cuBLAS FP16 reference (row-major idiom: swap M/N, swap A/B)
5622 cublasHandle_t handle;
5623 cublasCreate(&handle);
5624 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5625 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5626             CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5627             CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5628 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H ref");
5629
5630 // CuTe kernel (Stages=2, TN layout)
5631 launch_hgemm_mma_stages_tn_cute<half, 2, 0>(d_a, d_b_t, d_c, M, N, K);
5632 check(cudaGetLastError(), "hgemm_cute launch");
5633 check(cudaDeviceSynchronize(), "hgemm_cute sync");
5634
5635 half *h_c = (half *)malloc(size_c);
5636 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "hgemm_cute D2H");
5637
5638 // Verify
5639 float max_err = 0.0f;
5640 for (int i = 0; i < M * N; i++) {
5641     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5642     if (err > max_err) max_err = err;
5643 }
5644 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM CuTe Swizzle + Reg2x",
5645         max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5646
5647 free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5648 cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5649 cublasDestroy(handle);
5650 }
5651 #endif /* NOTES_V2_ENABLE_CUTE */

```

```

5652
5653
5654 #if defined(NOTES_V2_ENABLE_WGMMMA)
5655 static void test_hgemm_wgmma(int M, int N, int K) {
5656     // HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper SM90+)
5657     // TN layout: C[M×N] = A[M×K] × BT[N×K]
5658     // Kernel: hgemm_wgmma_stages_tn with default template params
5659
5660     constexpr int BM = 128, BN = 128, BK = 64, kStages = 3, kNumThreads = 256;
5661
5662     // M, K must be divisible by tile dims
5663     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5664         printf("| %-42s | %-12s | %-4s | %-8s |\n",
5665             "HGEMM WGMMMA", "SKIP", "SKIP", "None");
5666         return;
5667     }
5668
5669     size_t size_a = (size_t)M * K * sizeof(half);
5670     size_t size_b = (size_t)K * N * sizeof(half);
5671     size_t size_c = (size_t)M * N * sizeof(half);
5672
5673     half *h_a = (half *)malloc(size_a);
5674     half *h_b = (half *)malloc(size_b);
5675     half *h_c_ref = (half *)malloc(size_c);
5676
5677     srand(42);
5678     for (int i = 0; i < M * K; i++)
5679         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5680     for (int i = 0; i < K * N; i++)
5681         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5682
5683     // BT [N×K] row-major for TN layout (same as hgemm_mma kernel)
5684     size_t size_b_t = (size_t)N * K * sizeof(half);
5685     half *h_b_t = (half *)malloc(size_b_t);
5686     for (int n = 0; n < N; n++)
5687         for (int k = 0; k < K; k++)
5688             h_b_t[n * K + k] = h_b[k * N + n];
5689
5690     half *d_a, *d_b, *d_b_t, *d_c;
5691     check(cudaMalloc(&d_a, size_a), "wgmma alloc A");
5692     check(cudaMalloc(&d_b, size_b), "wgmma alloc B (cuBLAS)");
5693     check(cudaMalloc(&d_b_t, size_b_t), "wgmma alloc B_t (kernel)");
5694     check(cudaMalloc(&d_c, size_c), "wgmma alloc C");
5695
5696     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "wgmma H2D A");
5697     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice),
5698         "wgmma H2D B (cuBLAS)");
5699     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5700         "wgmma H2D B_t (kernel)");
5701
5702     // cuBLAS FP16 reference (row-major idiom, same as hgemm_mma)
5703     cublasHandle_t handle;
5704     cublasCreate(&handle);
5705     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
5706     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b,
5707         CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N,
5708         CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5709     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5710         "wgmma D2H ref");
5711
5712     // Create TMA tensor maps for A and BT
5713     // A[M×K] row-major: TMA box=(BK=64, BM=128), global shape=(K, M)
5714     // BT[N×K] row-major: TMA box=(BK=64, BN=128), global shape=(K, N)

```

```

5715 CUTensorMap *tma_a =
5716     allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5717 CUTensorMap *tma_b =
5718     allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5719
5720 // Launch WGMMMA kernel
5721 // kBlockSwizzle=false → 2D grid, no swizzle
5722 size_t smem_bytes =
5723     kStages * (BM * BK + BN * BK) * sizeof(half); // 3*16384*2 = 96KB
5724 cudaFuncSetAttribute(
5725     hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
5726     false>,
5727     cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
5728
5729 dim3 block(kNumThreads);
5730 dim3 grid((N + BN - 1) / BN, (M + BM - 1) / BM);
5731 hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages, false>
5732     <<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5733 check(cudaGetLastError(), "wgmma launch");
5734 check(cudaDeviceSynchronize(), "wgmma sync");
5735
5736 half *h_c = (half *)malloc(size_c);
5737 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "wgmma D2H");
5738
5739 // Verify
5740 float max_err = 0.0f;
5741 for (int i = 0; i < M * N; i++) {
5742     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
5743     if (err > max_err)
5744         max_err = err;
5745 }
5746 printf("| %-42s | %.6e | %-4s | %-8s |\n", "HGEMM TMA WGMMMA WS (3-stage)", max_err,
5747     max_err < 1.0f ? "PASS" : "FAIL", "None");
5748
5749 free(h_a);
5750 free(h_b);
5751 free(h_b_t);
5752 free(h_c);
5753 free(h_c_ref);
5754 cudaFree(d_a);
5755 cudaFree(d_b);
5756 cudaFree(d_b_t);
5757 cudaFree(d_c);
5758 cudaFree(tma_a);
5759 cudaFree(tma_b);
5760 cublasDestroy(handle);
5761 }
5762 #endif /* NOTES_V2_ENABLE_WGMMMA */
5763
5764 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
5765 template <int kStages, int kBlockSwizzle = 0>
5766 static bool launch_hgemm_tma_mma_ws(int M, int N, int K, half *d_a,
5767     half *d_b_t, half *d_c,
5768     CUTensorMap *tma_a, CUTensorMap *tma_b) {
5769     constexpr int kMmaM = 16, kMmaN = 8, kMmaK = 16;
5770     constexpr int kMmaTileM = 2, kMmaTileN = 2;
5771     constexpr int kValTileM = 4, kValTileN = 8, kValTileK = 4;
5772     constexpr int BM = kMmaM * kMmaTileM * kValTileM;
5773     constexpr int BN = kMmaN * kMmaTileN * kValTileN;
5774     constexpr int BK = kMmaK * kValTileK;
5775     constexpr int kNumThreads = 256;
5776     constexpr size_t payload_bytes =
5777         kStages * (BM * BK + BN * BK) * sizeof(half);

```



```

5778 constexpr size_t smem_bytes = payload_bytes;
5779 using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
5780                          const CUTensorMap *);
5781 Kernel kernel = hgemm_tma_mma_ws_tn<
5782     kMmaM, kMmaN, kMmaK, kMmaTileM, kMmaTileN, kValTileM, kValTileN,
5783     kValTileK, kStages, kNumThreads, kBlockSwizzle>;
5784
5785 int device = 0;
5786 int max_smem = 0;
5787 cudaFuncAttributes attributes{};
5788 check(cudaGetDevice(&device), "tma_mma_ws get device");
5789 check(cudaDeviceGetAttribute(&max_smem,
5790     cudaDevAttrMaxSharedMemoryPerBlockOptin,
5791     device),
5792     "tma_mma_ws max shared memory");
5793 check(cudaFuncGetAttributes(&attributes, kernel),
5794     "tma_mma_ws function attributes");
5795 if (smem_bytes + attributes.sharedSizeBytes > size_t(max_smem)) {
5796     printf("| %-42s | %-12s | %-4s | %-8s |\n",
5797         kStages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5798             : "HGEMM TMA MMA WS (S=3, SW=0)",
5799         "SMEM SKIP", "SKIP", "None");
5800     return false;
5801 }
5802
5803 check(cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
5804     smem_bytes),
5805     "tma_mma_ws set dynamic shared memory");
5806 dim3 block(kNumThreads);
5807 constexpr int kSwizzleN = 16;
5808 const int n_tiles = N / BN;
5809 const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
5810 const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
5811 dim3 grid(grid_x, M / BM, grid_z);
5812 kernel<<<grid, block, smem_bytes>>>(M, N, K, d_c, tma_a, tma_b);
5813 check(cudaGetLastError(), "tma_mma_ws launch");
5814 check(cudaDeviceSynchronize(), "tma_mma_ws sync");
5815 return true;
5816 }
5817
5818 static void test_hgemm_tma_mma_ws(int M, int N, int K) {
5819     constexpr int BM = 128, BN = 128, BK = 64;
5820     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
5821         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS", "SKIP",
5822             "SKIP", "None");
5823         return;
5824     }
5825
5826     const size_t size_a = size_t(M) * K * sizeof(half);
5827     const size_t size_b = size_t(K) * N * sizeof(half);
5828     const size_t size_b_t = size_t(N) * K * sizeof(half);
5829     const size_t size_c = size_t(M) * N * sizeof(half);
5830     half *h_a = static_cast<half *>(malloc(size_a));
5831     half *h_b = static_cast<half *>(malloc(size_b));
5832     half *h_b_t = static_cast<half *>(malloc(size_b_t));
5833     half *h_c = static_cast<half *>(malloc(size_c));
5834     half *h_c_ref = static_cast<half *>(malloc(size_c));
5835     srand(42);
5836     for (int i = 0; i < M * K; ++i)
5837         h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5838     for (int i = 0; i < K * N; ++i)
5839         h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
5840     for (int n = 0; n < N; ++n)

```

```

5841     for (int k = 0; k < K; ++k)
5842         h_b_t[n * K + k] = h_b[k * N + n];
5843
5844     half *d_a, *d_b, *d_b_t, *d_c;
5845     check(cudaMalloc(&d_a, size_a), "tma_mma_ws alloc A");
5846     check(cudaMalloc(&d_b, size_b), "tma_mma_ws alloc B");
5847     check(cudaMalloc(&d_b_t, size_b_t), "tma_mma_ws alloc B_t");
5848     check(cudaMalloc(&d_c, size_c), "tma_mma_ws alloc C");
5849     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "tma_mma_ws H2D A");
5850     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "tma_mma_ws H2D B");
5851     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
5852            "tma_mma_ws H2D B_t");
5853
5854     cublasHandle_t handle;
5855     cublasCreate(&handle);
5856     half alpha = __float2half(1.0f), beta = __float2half(0.0f);
5857     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b,
5858                  CUDA_R_16F, N, d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N,
5859                  CUBLAS_COMPUTE_16F, CUBLAS_GEMM_DEFAULT);
5860     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
5861            "tma_mma_ws D2H reference");
5862
5863     CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
5864     CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
5865     for (int block_swizzle : {0, 1}) {
5866         for (int stages : {2, 3}) {
5867             const bool launched = stages == 2
5868                 ? (block_swizzle
5869                    ? launch_hgemm_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c,
5870                                                    tma_a, tma_b)
5871                    : launch_hgemm_tma_mma_ws<2>(M, N, K, d_a, d_b_t, d_c,
5872                                                  tma_a, tma_b))
5873                 : (block_swizzle
5874                    ? launch_hgemm_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c,
5875                                                    tma_a, tma_b)
5876                    : launch_hgemm_tma_mma_ws<3>(M, N, K, d_a, d_b_t, d_c,
5877                                                  tma_a, tma_b));
5878             if (!launched)
5879                 continue;
5880             check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost),
5881                    "tma_mma_ws D2H");
5882             float max_err = 0.0f;
5883             for (int i = 0; i < M * N; ++i)
5884                 max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) -
5885                                                    __half2float(h_c_ref[i])));
5886             printf("| %-42s | %.6e | %-4s | %-8s |\n",
5887                    block_swizzle
5888                    ? (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=1)"
5889                                   : "HGEMM TMA MMA WS (S=3, SW=1)")
5890                    : (stages == 2 ? "HGEMM TMA MMA WS (S=2, SW=0)"
5891                                   : "HGEMM TMA MMA WS (S=3, SW=0)"),
5892                    max_err, max_err < 1.0f ? "PASS" : "FAIL", "None");
5893         }
5894     }
5895
5896     free(h_a); free(h_b); free(h_b_t); free(h_c); free(h_c_ref);
5897     cudaFree(d_a); cudaFree(d_b); cudaFree(d_b_t); cudaFree(d_c);
5898     cudaFree(tma_a); cudaFree(tma_b);
5899     cublasDestroy(handle);
5900 }
5901 #endif /* NOTES_V2_ENABLE_TMA_MMA_WS */
5902
5903

```

```

5904 static void test_flash_attn(int seqlen, int head_dim) {
5905     // FlashAttention-2 with split-Q, MMA m16n8k16
5906     int B = 1, H = 8;
5907
5908     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
5909
5910     srand(42);
5911     half *h_q = (half *)malloc(sz);
5912     half *h_k = (half *)malloc(sz);
5913     half *h_v = (half *)malloc(sz);
5914     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
5915         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5916         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5917         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
5918     }
5919
5920     // CPU reference in FP32: 0 = softmax(Q @ K^T / sqrt(d)) @ V
5921     float *ref_q = (float *)malloc(sz * 4 / sizeof(half)); // 4x for float
5922     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
5923     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
5924     float *ref_o = (float *)malloc(sz * 4 / sizeof(half));
5925     int count = B * H * seqlen * head_dim;
5926     for (int i = 0; i < count; i++) {
5927         ref_q[i] = __half2float(h_q[i]);
5928         ref_k[i] = __half2float(h_k[i]);
5929         ref_v[i] = __half2float(h_v[i]);
5930     }
5931
5932     float scale = 1.0f / sqrtf((float)head_dim);
5933     for (int bi = 0; bi < B * H; bi++) {
5934         for (int qi = 0; qi < seqlen; qi++) {
5935             // S[qi, kj] = Q[qi, :] @ K[kj, :]^T * scale
5936             float smax = -INFINITY;
5937             float *S = (float *)malloc((size_t)seqlen * sizeof(float));
5938             for (int kj = 0; kj < seqlen; kj++) {
5939                 float s = 0.0f;
5940                 for (int d = 0; d < head_dim; d++)
5941                     s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
5942                         ref_k[bi * seqlen * head_dim + kj * head_dim + d];
5943                 S[kj] = s * scale;
5944                 if (S[kj] > smax) smax = S[kj];
5945             }
5946             // softmax
5947             double sum_exp = 0.0;
5948             for (int kj = 0; kj < seqlen; kj++) sum_exp += (double)expf(S[kj] - smax);
5949             float inv_sum = 1.0f / (float)sum_exp;
5950             // O[qi, :] = sum_kj P[qi, kj] * V[kj, :]
5951             for (int d = 0; d < head_dim; d++) {
5952                 double o_acc = 0.0;
5953                 for (int kj = 0; kj < seqlen; kj++)
5954                     o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
5955                         ref_v[bi * seqlen * head_dim + kj * head_dim + d];
5956                 ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
5957             }
5958             free(S);
5959         }
5960     }
5961
5962     half *d_q, *d_k, *d_v, *d_o;
5963     check(cudaMalloc(&d_q, sz), "fa alloc Q");
5964     check(cudaMalloc(&d_k, sz), "fa alloc K");
5965     check(cudaMalloc(&d_v, sz), "fa alloc V");
5966     check(cudaMalloc(&d_o, sz), "fa alloc O");

```

```

5967 check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "fa H2D Q");
5968 check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "fa H2D K");
5969 check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "fa H2D V");
5970
5971 // Template params for kHeadDim=64, kStagesK=2
5972 constexpr int kHeadDim = 64;
5973 constexpr int kStagesK = 2;
5974 constexpr int kPadQ = 8;
5975 constexpr int kPadK = 8;
5976 constexpr int kPadV = 8;
5977 constexpr int kMmaAtomM = 16;
5978 constexpr int kMmaAtomN = 8;
5979 constexpr int kMmaAtomK = 16;
5980 constexpr int kMmaTileSeqLenQ = 8;
5981 constexpr int kMmaTileSeqLenK = 1;
5982 constexpr int kMmaTileSeqLenP = 8;
5983 constexpr int kMmaTileHeadDimV = 1;
5984 constexpr int kValTileSeqLenQ = 1;
5985 constexpr int kValTileSeqLenK = 8;
5986 constexpr int kValTileSeqLenP = 1;
5987 constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
5988
5989 constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
5990 constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
5991 if (seqlen < Br) return; // kernel requires seqlen >= tile size
5992 size_t smem_bytes =
5993     (Br * (kHeadDim + kPadQ) +
5994      kStagesK * Bc * (kHeadDim + kPadK) +
5995      Bc * (kHeadDim + kPadV)) * sizeof(half);
5996
5997 dim3 block(256);
5998 dim3 grid((seqlen + Br - 1) / Br, B * H);
5999
6000 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6001 FAKernel fa_k = flash_attn_mma_stages_split_q<kHeadDim, kMmaAtomM, kMmaAtomN,
6002     kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK, kMmaTileSeqLenP,
6003     kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK, kValTileSeqLenP,
6004     kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
6005 cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6006 fa_k<<grid, block, smem_bytes>>>(d_q, d_k, d_v, d_o, seqlen, H);
6007 check(cudaGetLastError(), "fa launch");
6008 check(cudaDeviceSynchronize(), "fa sync");
6009
6010 half *h_o = (half *)malloc(sz);
6011 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa D2H");
6012
6013 float max_err = 0.0f;
6014 for (int i = 0; i < count; i++) {
6015     float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
6016     if (err > max_err) max_err = err;
6017 }
6018 printf("| %-42s | %.6e | %-4s | %-8s |\n", "FlashAttention-2, (kStagesK=2, Pad)",
6019     max_err, max_err < 5e-1f ? "PASS" : "FAIL", "None");
6020
6021 // Also test swizzle mode (kPadQ=kPadK=kPadV=0)
6022 {
6023     constexpr int kPadQSwz = 0, kPadKSwz = 0, kPadVSwz = 0;
6024     size_t smem_bytes_swz =
6025         (Br * (kHeadDim + kPadQSwz) +
6026          kStagesK * Bc * (kHeadDim + kPadKSwz) +
6027          Bc * (kHeadDim + kPadVSwz)) * sizeof(half);
6028     check(cudaMemset(d_o, 0, sz), "fa memset 0");
6029     using FAKernelSwz = void (*)(half *, half *, half *, half *, int, int);

```

```

6030     FAKernelSwz fa_k_swz = flash_attn_mma_stages_split_q<
6031         kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK,
6032         kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
6033         kValTileSeqLenP, kValTileHeadDimV, kStagesK,
6034         kPadQSwz, kPadKSwz, kPadVSwz>;
6035     cudaFuncSetAttribute(fa_k_swz, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes_swz);
6036     fa_k_swz<<<grid, block, smem_bytes_swz>>>(d_q, d_k, d_v, d_o, seqlen, H);
6037     check(cudaGetLastError(), "fa swizzle launch");
6038     check(cudaDeviceSynchronize(), "fa swizzle sync");
6039     check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "fa swizzle D2H");
6040
6041     max_err = 0.0f;
6042     for (int i = 0; i < count; i++) {
6043         float err = fabsf(__half2float(h_o[i]) - ref_o[i]);
6044         if (err > max_err) max_err = err;
6045     }
6046     printf("| %-42s | %.6e | %-4s | %-8s |\n", "FlashAttention-2, (kStagesK=2, Swizzle)",
6047         max_err, max_err < 5e-1f ? "PASS" : "FAIL", "None");
6048 }
6049
6050 free(h_q); free(h_k); free(h_v); free(h_o);
6051 free(ref_q); free(ref_k); free(ref_v); free(ref_o);
6052 cudaFree(d_q); cudaFree(d_k); cudaFree(d_v); cudaFree(d_o);
6053 }
6054
6055
6056 // Bench mode globals and helpers
6057 static bool g_bench_mode = false;
6058 static bool g_bench_fa = false;
6059 static bool g_bench_only_fa = false;
6060 static bool g_fa_skip_check = false;
6061 enum class FALayout {All, Pad, SwizzleQ, SwizzleK, SwizzleV, SwizzleQK, SwizzleQV, SwizzleKV, Swizzle};
6062 static FALayout g_fa_layout = FALayout::Pad;
6063 static int g_bench_M = 4096, g_bench_N = 4096, g_bench_K = 4096;
6064 static int g_bench_B = 1, g_bench_H = 32, g_bench_Nfa = 4096, g_bench_D = 64;
6065 static int g_warmup = 5, g_repeat = 10;
6066
6067 static float bench_hgemm_tflops(int M, int N, int K, float time_ms) {
6068     double flops = 2.0 * M * N * K;
6069     return (float)(flops / (double)time_ms / 1e9);
6070 }
6071
6072 static float bench_fa_tflops(int B, int H, int N, int D, float time_ms) {
6073     double flops = 4.0 * B * H * N * N * D;
6074     return (float)(flops / (double)time_ms / 1e9);
6075 }
6076
6077 static bool check_smem_feasible(const void *kernel_func, size_t dyn_smem_bytes) {
6078     int device = 0;
6079     int max_smem = 0;
6080     cudaFuncAttributes attrs{};
6081     cudaGetDevice(&device);
6082     cudaDeviceGetAttribute(&max_smem, cudaDevAttrMaxSharedMemoryPerBlockOptin, device);
6083     cudaFuncGetAttributes(&attrs, kernel_func);
6084     return dyn_smem_bytes + attrs.sharedSizeBytes <= (size_t)max_smem;
6085 }
6086
6087 // =====
6088 // Bench: HGEMM MMA (basic m16n8k16 + multistage pipeline)
6089 // =====
6090 template <int kStages, int kBlockSwizzle>
6091 static bool launch_timed_hgemm_mma(
6092     half *d_a, half *d_b_t, half *d_c, half *h_c,

```

```

6093     half *h_c_ref, int M, int N, int K, size_t size_c,
6094     cudaEvent_t start, cudaEvent_t stop, float &max_err,
6095     float &time_ms
6096 ) {
6097     constexpr int BM = 128, BN = 128, BK = 16;
6098     using Kernel = void (*)(half *, half *, half *, int, int, int);
6099     Kernel k = hgemm_mma_stages_tn<16, 8, 16, 2, 4, 4, 4, kStages, kBlockSwizzle>;
6100     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6101     if (!check_smem_feasible((const void *)k, smem)) return false;
6102     dim3 block(256);
6103     const int tiles_n = (N + BN - 1) / BN;
6104     constexpr int kSwizzleN = 16;
6105     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6106     int gz = kBlockSwizzle ? kSwizzleN : 1;
6107     dim3 grid(gx, (M + BM - 1) / BM, gz);
6108     for (int w = 0; w < g_warmup; w++)
6109         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6110     cudaDeviceSynchronize();
6111     cudaEventRecord(start);
6112     for (int r = 0; r < g_repeat; r++)
6113         k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6114     cudaEventRecord(stop);
6115     cudaEventSynchronize(stop);
6116     cudaEventElapsedTime(&time_ms, start, stop);
6117     time_ms /= g_repeat;
6118     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H");
6119     max_err = 0.0f;
6120     for (int i = 0; i < M * N; i++) {
6121         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6122         if (err > max_err) max_err = err;
6123     }
6124     return true;
6125 }
6126
6127 static void bench_hgemm_mma(int M, int N, int K) {
6128     size_t size_a = (size_t)M * K * sizeof(half);
6129     size_t size_b = (size_t)K * N * sizeof(half);
6130     size_t size_c = (size_t)M * N * sizeof(half);
6131     half *h_a = (half *)malloc(size_a);
6132     half *h_b = (half *)malloc(size_b);
6133     half *h_c_ref = (half *)malloc(size_c);
6134     srand(42);
6135     for (int i = 0; i < M * K; i++)
6136         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6137     for (int i = 0; i < K * N; i++)
6138         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6139     size_t size_b_t = (size_t)N * K * sizeof(half);
6140     half *h_b_t = (half *)malloc(size_b_t);
6141     for (int n = 0; n < N; n++)
6142         for (int k = 0; k < K; k++)
6143             h_b_t[n * K + k] = h_b[k * N + n];
6144     half *d_a, *d_b, *d_b_t, *d_c;
6145     check(cudaMalloc(&d_a, size_a), "bench mma alloc A");
6146     check(cudaMalloc(&d_b, size_b), "bench mma alloc B");
6147     check(cudaMalloc(&d_b_t, size_b_t), "bench mma alloc B_t");
6148     check(cudaMalloc(&d_c, size_c), "bench mma alloc C");
6149     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench mma H2D A");
6150     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench mma H2D B");
6151     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench mma H2D B_t");
6152     cublasHandle_t handle;
6153     cublasCreate(&handle);
6154     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6155     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,

```



```

6156     d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6157     CUBLAS_GEMM_DEFAULT);
6158 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench mma D2H ref");
6159 half *h_c = (half *)malloc(size_c);
6160 cudaEvent_t start, stop;
6161 cudaEventCreate(&start);
6162 cudaEventCreate(&stop);
6163 for (int stages : {2, 3}) {
6164     for (int swizzle : {0, 1}) {
6165         float max_err = 0, time_ms = 0;
6166         bool ok = false;
6167         if (stages == 2)
6168             ok = swizzle ? launch_timed_hgemm_mma<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6169                 K, size_c, start, stop, max_err, time_ms)
6170             : launch_timed_hgemm_mma<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6171                 K, size_c, start, stop, max_err, time_ms);
6172         else
6173             ok = swizzle ? launch_timed_hgemm_mma<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6174                 K, size_c, start, stop, max_err, time_ms)
6175             : launch_timed_hgemm_mma<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M, N,
6176                 K, size_c, start, stop, max_err, time_ms);
6177         char label[64];
6178         snprintf(label, sizeof(label), "HGEMM MMA (S=%d, SW=%d)", stages, swizzle);
6179         if (!ok) {
6180             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6181         } else {
6182             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6183             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6184                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6185         }
6186     }
6187 }
6188 cudaEventDestroy(start);
6189 cudaEventDestroy(stop);
6190 free(h_a);
6191 free(h_b);
6192 free(h_b_t);
6193 free(h_c);
6194 free(h_c_ref);
6195 cudaFree(d_a);
6196 cudaFree(d_b);
6197 cudaFree(d_b_t);
6198 cudaFree(d_c);
6199 cublasDestroy(handle);
6200 }
6201
6202 // =====
6203 // Bench: HGEMM MMA Swizzle + Register Double Buffering (kValTileK=2, BK=32)
6204 // =====
6205 template <int kStages, int kBlockSwizzle>
6206 static bool launch_timed_hgemm_swizzle(
6207     half *d_a, half *d_b_t, half *d_c, half *h_c,
6208     half *h_c_ref, int M, int N, int K, size_t size_c,
6209     cudaEvent_t start, cudaEvent_t stop, float &max_err,
6210     float &time_ms
6211 ) {
6212     constexpr int BM = 128, BN = 128, BK = 32;
6213     using Kernel = void (*)(half *, half *, half *, int, int, int);
6214     Kernel k = hgemm_mma_stages_tn_swizzle<16, 8, 16, 2, 4, 4, 4, 2, kStages, kBlockSwizzle>;
6215     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6216     if (!check_smem_feasible((const void *)k, smem)) return false;
6217     dim3 block(256);
6218     const int tiles_n = (N + BN - 1) / BN;

```

```

6219 constexpr int kSwizzleN = 16;
6220 int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6221 int gz = kBlockSwizzle ? kSwizzleN : 1;
6222 dim3 grid(gx, (M + BM - 1) / BM, gz);
6223 for (int w = 0; w < g_warmup; w++)
6224     k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6225 cudaDeviceSynchronize();
6226 cudaEventRecord(start);
6227 for (int r = 0; r < g_repeat; r++)
6228     k<<<grid, block, smem>>>(d_a, d_b_t, d_c, M, N, K);
6229 cudaEventRecord(stop);
6230 cudaEventSynchronize(stop);
6231 cudaEventElapsedTime(&time_ms, start, stop);
6232 time_ms /= g_repeat;
6233 check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H");
6234 max_err = 0.0f;
6235 for (int i = 0; i < M * N; i++) {
6236     float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6237     if (err > max_err) max_err = err;
6238 }
6239 return true;
6240 }
6241
6242 static void bench_hgemm_swizzle(int M, int N, int K) {
6243     size_t size_a = (size_t)M * K * sizeof(half);
6244     size_t size_b = (size_t)K * N * sizeof(half);
6245     size_t size_c = (size_t)M * N * sizeof(half);
6246     half *h_a = (half *)malloc(size_a);
6247     half *h_b = (half *)malloc(size_b);
6248     half *h_c_ref = (half *)malloc(size_c);
6249     srand(42);
6250     for (int i = 0; i < M * K; i++)
6251         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6252     for (int i = 0; i < K * N; i++)
6253         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6254     size_t size_b_t = (size_t)N * K * sizeof(half);
6255     half *h_b_t = (half *)malloc(size_b_t);
6256     for (int n = 0; n < N; n++)
6257         for (int k = 0; k < K; k++)
6258             h_b_t[n * K + k] = h_b[k * N + n];
6259     half *d_a, *d_b, *d_b_t, *d_c;
6260     check(cudaMalloc(&d_a, size_a), "bench swizzle alloc A");
6261     check(cudaMalloc(&d_b, size_b), "bench swizzle alloc B");
6262     check(cudaMalloc(&d_b_t, size_b_t), "bench swizzle alloc B_t");
6263     check(cudaMalloc(&d_c, size_c), "bench swizzle alloc C");
6264     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench swizzle H2D A");
6265     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench swizzle H2D B");
6266     check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench swizzle H2D B_t");
6267     cublasHandle_t handle;
6268     cublasCreate(&handle);
6269     half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6270     cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6271         d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6272         CUBLAS_GEMM_DEFAULT);
6273     check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench swizzle D2H ref");
6274     half *h_c = (half *)malloc(size_c);
6275     cudaEvent_t start, stop;
6276     cudaEventCreate(&start);
6277     cudaEventCreate(&stop);
6278     for (int stages : {2, 3}) {
6279         for (int swizzle : {0, 1}) {
6280             float max_err = 0, time_ms = 0;
6281             bool ok = false;

```

```

6282     if (stages == 2)
6283         ok = swizzle ? launch_timed_hgemm_swizzle<2, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6284             N, K, size_c, start, stop, max_err, time_ms)
6285         : launch_timed_hgemm_swizzle<2, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6286             N, K, size_c, start, stop, max_err, time_ms);
6287     else
6288         ok = swizzle ? launch_timed_hgemm_swizzle<3, 1>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6289             N, K, size_c, start, stop, max_err, time_ms)
6290         : launch_timed_hgemm_swizzle<3, 0>(d_a, d_b_t, d_c, h_c, h_c_ref, M,
6291             N, K, size_c, start, stop, max_err, time_ms);
6292     char label[64];
6293     snprintf(label, sizeof(label), "HGEMM Swizzle+Reg2x (S=%d, SW=%d)", stages, swizzle);
6294     if (!ok) {
6295         printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6296     } else {
6297         float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6298         printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6299             max_err < 1.0f ? "PASS" : "FAIL", tflops);
6300     }
6301 }
6302 }
6303 cudaEventDestroy(start);
6304 cudaEventDestroy(stop);
6305 free(h_a);
6306 free(h_b);
6307 free(h_b_t);
6308 free(h_c);
6309 free(h_c_ref);
6310 cudaFree(d_a);
6311 cudaFree(d_b);
6312 cudaFree(d_b_t);
6313 cudaFree(d_c);
6314 cublasDestroy(handle);
6315 }
6316
6317 #if defined(NOTES_V2_ENABLE_CUTE)
6318 // =====
6319 // Bench: HGEMM CuTe
6320 // =====
6321 static void bench_hgemm_cute(int M, int N, int K) {
6322     size_t size_a = (size_t)M * K * sizeof(half);
6323     size_t size_b = (size_t)K * N * sizeof(half);
6324     size_t size_c = (size_t)M * N * sizeof(half);
6325     half *h_a = (half *)malloc(size_a);
6326     half *h_b = (half *)malloc(size_b);
6327     half *h_c_ref = (half *)malloc(size_c);
6328     srand(42);
6329     for (int i = 0; i < M * K; i++)
6330         h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6331     for (int i = 0; i < K * N; i++)
6332         h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6333     size_t size_b_t = (size_t)N * K * sizeof(half);
6334     half *h_b_t = (half *)malloc(size_b_t);
6335     for (int n = 0; n < N; n++)
6336         for (int k = 0; k < K; k++)
6337             h_b_t[n * K + k] = h_b[k * N + n];
6338     half *d_a, *d_b, *d_b_t, *d_c;
6339     check(cudaMalloc(&d_a, size_a), "bench cute alloc A");
6340     check(cudaMalloc(&d_b, size_b), "bench cute alloc B");
6341     check(cudaMalloc(&d_b_t, size_b_t), "bench cute alloc B_t");
6342     check(cudaMalloc(&d_c, size_c), "bench cute alloc C");
6343     check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench cute H2D A");
6344     check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench cute H2D B");

```

```

6345 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench cute H2D B_t");
6346 cublasHandle_t handle;
6347 cublasCreate(&handle);
6348 half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6349 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6350             d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6351             CUBLAS_GEMM_DEFAULT);
6352 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H ref");
6353 half *h_c = (half *)malloc(size_c);
6354 cudaEvent_t start, stop;
6355 cudaEventCreate(&start);
6356 cudaEventCreate(&stop);
6357 for (int stages : {2, 3}) {
6358     for (int swizzle : {0, 1}) {
6359         char label[64];
6360         snprintf(label, sizeof(label), "HGEMM CuTe Swizzle (S=%d, SW=%d)", stages, swizzle);
6361         bool ok = false;
6362         float time_ms = 0, max_err = 0;
6363         if (stages == 2) {
6364             auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 2, 1>
6365                             : launch_hgemm_mma_stages_tn_cute<half, 2, 0>;
6366             for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
6367             cudaDeviceSynchronize();
6368             cudaEventRecord(start);
6369             for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
6370             cudaEventRecord(stop);
6371             cudaEventSynchronize(stop);
6372             cudaEventElapsedTime(&time_ms, start, stop);
6373             time_ms /= g_repeat;
6374             ok = true;
6375         } else {
6376             auto k = swizzle ? launch_hgemm_mma_stages_tn_cute<half, 3, 1>
6377                             : launch_hgemm_mma_stages_tn_cute<half, 3, 0>;
6378             for (int w = 0; w < g_warmup; w++) k(d_a, d_b_t, d_c, M, N, K);
6379             cudaDeviceSynchronize();
6380             cudaEventRecord(start);
6381             for (int r = 0; r < g_repeat; r++) k(d_a, d_b_t, d_c, M, N, K);
6382             cudaEventRecord(stop);
6383             cudaEventSynchronize(stop);
6384             cudaEventElapsedTime(&time_ms, start, stop);
6385             time_ms /= g_repeat;
6386             ok = true;
6387         }
6388         if (ok) {
6389             check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench cute D2H");
6390             max_err = 0.0f;
6391             for (int i = 0; i < M * N; i++) {
6392                 float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6393                 if (err > max_err) max_err = err;
6394             }
6395             float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6396             printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6397                 max_err < 1.0f ? "PASS" : "FAIL", tflops);
6398         } else {
6399             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "LAUNCH ERR", "FAIL", "None");
6400         }
6401     }
6402 }
6403 cudaEventDestroy(start);
6404 cudaEventDestroy(stop);
6405 free(h_a);
6406 free(h_b);
6407 free(h_b_t);

```

```

6408     free(h_c);
6409     free(h_c_ref);
6410     cudaFree(d_a);
6411     cudaFree(d_b);
6412     cudaFree(d_b_t);
6413     cudaFree(d_c);
6414     cublasDestroy(handle);
6415 }
6416 #endif
6417
6418 #if defined(NOTES_V2_ENABLE_WGMMMA)
6419 // =====
6420 // Bench: HGEMM WGMMMA (m64n128k16 + TMA + Warp Specialization, SM90+)
6421 // =====
6422 template <int kStages, int kBlockSwizzle>
6423 static bool launch_timed_hgemm_wgmma(half *d_c, half *h_c, half *h_c_ref,
6424                                     CUTensorMap *tma_a, CUTensorMap *tma_b,
6425                                     int M, int N, int K, size_t size_c,
6426                                     cudaEvent_t start, cudaEvent_t stop,
6427                                     float &max_err, float &time_ms) {
6428     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6429     using Kernel = void (*)(int, int, int, half *, const CUTensorMap *,
6430                             const CUTensorMap *);
6431     Kernel k = hgemm_wgmma_stages_tn<64, 128, 16, BM, BN, BK, kNumThreads, kStages,
6432                kBlockSwizzle>;
6433     size_t smem = kStages * (BM * BK + BN * BK) * sizeof(half);
6434     if (!check_smem_feasible((const void *)k, smem)) return false;
6435     cudaFuncSetAttribute(k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem);
6436     dim3 block(kNumThreads);
6437     const int tiles_n = (N + BN - 1) / BN;
6438     constexpr int kSwizzleN = 16;
6439     int gx = kBlockSwizzle ? div_ceil(tiles_n, kSwizzleN) : tiles_n;
6440     int gz = kBlockSwizzle ? kSwizzleN : 1;
6441     dim3 grid(gx, (M + BM - 1) / BM, gz);
6442     for (int w = 0; w < g_warmup; w++)
6443         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
6444     cudaDeviceSynchronize();
6445     cudaEventRecord(start);
6446     for (int r = 0; r < g_repeat; r++)
6447         k<<<grid, block, smem>>>(M, N, K, d_c, tma_a, tma_b);
6448     cudaEventRecord(stop);
6449     cudaEventSynchronize(stop);
6450     cudaEventElapsedTime(&time_ms, start, stop);
6451     time_ms /= g_repeat;
6452     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H");
6453     max_err = 0.0f;
6454     for (int i = 0; i < M * N; i++) {
6455         float err = fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i]));
6456         if (err > max_err) max_err = err;
6457     }
6458     return true;
6459 }
6460
6461 static void bench_hgemm_wgmma(int M, int N, int K) {
6462     constexpr int BM = 128, BN = 128, BK = 64;
6463     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6464         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM WGMMMA (unaligned)", "SKIP", "SKIP",
6465             "None");
6466         return;
6467     }
6468     size_t size_a = (size_t)M * K * sizeof(half);
6469     size_t size_b = (size_t)K * N * sizeof(half);
6470     size_t size_c = (size_t)M * N * sizeof(half);

```

```

6471  half *h_a = (half *)malloc(size_a);
6472  half *h_b = (half *)malloc(size_b);
6473  half *h_c_ref = (half *)malloc(size_c);
6474  srand(42);
6475  for (int i = 0; i < M * K; i++)
6476      h_a[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6477  for (int i = 0; i < K * N; i++)
6478      h_b[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6479  size_t size_b_t = (size_t)N * K * sizeof(half);
6480  half *h_b_t = (half *)malloc(size_b_t);
6481  for (int n = 0; n < N; n++)
6482      for (int k = 0; k < K; k++)
6483          h_b_t[n * K + k] = h_b[k * N + n];
6484  half *d_a, *d_b, *d_b_t, *d_c;
6485  check(cudaMalloc(&d_a, size_a), "bench wgmma alloc A");
6486  check(cudaMalloc(&d_b, size_b), "bench wgmma alloc B");
6487  check(cudaMalloc(&d_b_t, size_b_t), "bench wgmma alloc B_t");
6488  check(cudaMalloc(&d_c, size_c), "bench wgmma alloc C");
6489  check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench wgmma H2D A");
6490  check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench wgmma H2D B");
6491  check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice), "bench wgmma H2D B_t");
6492  cublasHandle_t handle;
6493  cublasCreate(&handle);
6494  half alpha_h = __float2half(1.0f), beta_h = __float2half(0.0f);
6495  cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha_h, d_b, CUDA_R_16F, N,
6496      d_a, CUDA_R_16F, K, &beta_h, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6497      CUBLAS_GEMM_DEFAULT);
6498  check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost), "bench wgmma D2H ref");
6499  CUSensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6500  CUSensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6501  half *h_c = (half *)malloc(size_c);
6502  cudaEvent_t start, stop;
6503  cudaEventCreate(&start);
6504  cudaEventCreate(&stop);
6505  for (int stages : {1, 2, 3}) {
6506      for (int swizzle : {0, 1}) {
6507          float max_err = 0, time_ms = 0;
6508          bool ok = false;
6509          if (stages == 2)
6510              ok = swizzle
6511                  ? launch_timed_hgemm_wgmma<2, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6512                      K, size_c, start, stop, max_err, time_ms)
6513                  : launch_timed_hgemm_wgmma<2, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6514                      size_c, start, stop, max_err, time_ms);
6515          else
6516              ok = swizzle
6517                  ? launch_timed_hgemm_wgmma<3, 1>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N,
6518                      K, size_c, start, stop, max_err, time_ms)
6519                  : launch_timed_hgemm_wgmma<3, 0>(d_c, h_c, h_c_ref, tma_a, tma_b, M, N, K,
6520                      size_c, start, stop, max_err, time_ms);
6521          char label[64];
6522          snprintf(label, sizeof(label), "HGEMM TMA WGMMA WS (S=%d, SW=%d)", stages, swizzle);
6523          if (!ok) {
6524              printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");
6525          } else {
6526              float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6527              printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6528                  max_err < 1.0f ? "PASS" : "FAIL", tflops);
6529          }
6530      }
6531  }
6532  cudaEventDestroy(start);
6533  cudaEventDestroy(stop);

```



```

6534 free(h_a);
6535 free(h_b);
6536 free(h_b_t);
6537 free(h_c);
6538 free(h_c_ref);
6539 cudaFree(d_a);
6540 cudaFree(d_b);
6541 cudaFree(d_b_t);
6542 cudaFree(d_c);
6543 cudaFree(tma_a);
6544 cudaFree(tma_b);
6545 cublasDestroy(handle);
6546 }
6547 #endif
6548
6549 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
6550 // =====
6551 // Bench: HGEMM TMA MMA WS (mma.sync + TMA + Warp Specialization, SM120)
6552 // =====
6553 template <int kStages, int kBlockSwizzle>
6554 static bool bench_launch_tma_mma_ws(
6555     int M, int N, int K, half *d_a, half *d_b_t,
6556     half *d_c, half *h_c, half *h_c_ref, size_t size_c,
6557     CUTensorMap *tma_a, CUTensorMap *tma_b,
6558     cudaEvent_t start, cudaEvent_t stop,
6559     float &max_err, float &time_ms) {
6560     constexpr int BM = 128, BN = 128, BK = 64, kNumThreads = 256;
6561     constexpr size_t payload_bytes = kStages * (BM * BK + BN * BK) * sizeof(half);
6562     using Kernel = void (*)(int, int, half *, const CUTensorMap *,
6563         const CUTensorMap *);
6564     Kernel kernel = hgemm_tma_mma_ws_tn<16, 8, 16, 2, 2, 4, 8, 4, kStages,
6565         kNumThreads, kBlockSwizzle>;
6566     if (!check_smem_feasible((const void *)kernel, payload_bytes)) return false;
6567     cudaFuncSetAttribute(kernel, cudaFuncAttributeMaxDynamicSharedMemorySize,
6568         payload_bytes);
6569     dim3 block(kNumThreads);
6570     constexpr int kSwizzleN = 16;
6571     const int n_tiles = N / BN;
6572     const int grid_x = kBlockSwizzle ? div_ceil(n_tiles, kSwizzleN) : n_tiles;
6573     const int grid_z = kBlockSwizzle ? kSwizzleN : 1;
6574     dim3 grid(grid_x, M / BM, grid_z);
6575     for (int w = 0; w < g_warmup; w++)
6576         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6577     cudaDeviceSynchronize();
6578     cudaEventRecord(start);
6579     for (int r = 0; r < g_repeat; r++)
6580         kernel<<<grid, block, payload_bytes>>>(M, N, K, d_c, tma_a, tma_b);
6581     cudaEventRecord(stop);
6582     cudaEventSynchronize(stop);
6583     cudaEventElapsedTime(&time_ms, start, stop);
6584     time_ms /= g_repeat;
6585     check(cudaMemcpy(h_c, d_c, size_c, cudaMemcpyDeviceToHost), "bench tma mma ws D2H");
6586     max_err = 0.0f;
6587     for (int i = 0; i < M * N; ++i)
6588         max_err = fmaxf(max_err, fabsf(__half2float(h_c[i]) - __half2float(h_c_ref[i])));
6589     return true;
6590 }
6591
6592 static void bench_hgemm_tma_mma_ws(int M, int N, int K) {
6593     constexpr int BM = 128, BN = 128, BK = 64;
6594     if (M % BM != 0 || N % BN != 0 || K % BK != 0) {
6595         printf("| %-42s | %-12s | %-4s | %-8s |\n", "HGEMM TMA MMA WS (unaligned)", "SKIP",
6596             "SKIP", "None");

```

```

6597     return;
6598 }
6599 const size_t size_a = size_t(M) * K * sizeof(half);
6600 const size_t size_b = size_t(K) * N * sizeof(half);
6601 const size_t size_b_t = size_t(N) * K * sizeof(half);
6602 const size_t size_c = size_t(M) * N * sizeof(half);
6603 half *h_a = static_cast<half *>(malloc(size_a));
6604 half *h_b = static_cast<half *>(malloc(size_b));
6605 half *h_b_t = static_cast<half *>(malloc(size_b_t));
6606 half *h_c = static_cast<half *>(malloc(size_c));
6607 half *h_c_ref = static_cast<half *>(malloc(size_c));
6608 srand(42);
6609 for (int i = 0; i < M * K; ++i)
6610     h_a[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6611 for (int i = 0; i < K * N; ++i)
6612     h_b[i] = __float2half((float(rand()) / RAND_MAX) * 2.0f - 1.0f);
6613 for (int n = 0; n < N; ++n)
6614     for (int k = 0; k < K; ++k)
6615         h_b_t[n * K + k] = h_b[k * N + n];
6616 half *d_a, *d_b, *d_b_t, *d_c;
6617 check(cudaMalloc(&d_a, size_a), "bench tma mma ws alloc A");
6618 check(cudaMalloc(&d_b, size_b), "bench tma mma ws alloc B");
6619 check(cudaMalloc(&d_b_t, size_b_t), "bench tma mma ws alloc B_t");
6620 check(cudaMalloc(&d_c, size_c), "bench tma mma ws alloc C");
6621 check(cudaMemcpy(d_a, h_a, size_a, cudaMemcpyHostToDevice), "bench tma mma ws H2D A");
6622 check(cudaMemcpy(d_b, h_b, size_b, cudaMemcpyHostToDevice), "bench tma mma ws H2D B");
6623 check(cudaMemcpy(d_b_t, h_b_t, size_b_t, cudaMemcpyHostToDevice),
6624     "bench tma mma ws H2D B_t");
6625 cublasHandle_t handle;
6626 cublasCreate(&handle);
6627 half alpha = __float2half(1.0f), beta = __float2half(0.0f);
6628 cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, M, K, &alpha, d_b, CUDA_R_16F, N,
6629     d_a, CUDA_R_16F, K, &beta, d_c, CUDA_R_16F, N, CUBLAS_COMPUTE_16F,
6630     CUBLAS_GEMM_DEFAULT);
6631 check(cudaMemcpy(h_c_ref, d_c, size_c, cudaMemcpyDeviceToHost),
6632     "bench tma mma ws D2H reference");
6633 CUTensorMap *tma_a = allocate_and_create_tensor_map(d_a, M / BM, K / BK);
6634 CUTensorMap *tma_b = allocate_and_create_tensor_map(d_b_t, N / BN, K / BK);
6635 cudaEvent_t start, stop;
6636 cudaEventCreate(&start);
6637 cudaEventCreate(&stop);
6638 for (int stages : {1, 2, 3}) {
6639     for (int swizzle : {0, 1}) {
6640         float max_err = 0, time_ms = 0;
6641         bool ok = false;
6642         if (stages == 2)
6643             ok = swizzle ? bench_launch_tma_mma_ws<2, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6644                 h_c_ref, size_c, tma_a, tma_b,
6645                 start, stop, max_err, time_ms)
6646             : bench_launch_tma_mma_ws<2, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6647                 h_c_ref, size_c, tma_a, tma_b,
6648                 start, stop, max_err, time_ms);
6649         else
6650             ok = swizzle ? bench_launch_tma_mma_ws<3, 1>(M, N, K, d_a, d_b_t, d_c, h_c,
6651                 h_c_ref, size_c, tma_a, tma_b,
6652                 start, stop, max_err, time_ms)
6653             : bench_launch_tma_mma_ws<3, 0>(M, N, K, d_a, d_b_t, d_c, h_c,
6654                 h_c_ref, size_c, tma_a, tma_b,
6655                 start, stop, max_err, time_ms);
6656         char label[64];
6657         snprintf(label, sizeof(label), "HGEMM TMA MMA WS (S=%d, SW=%d)", stages, swizzle);
6658         if (!ok) {
6659             printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM SKIP", "SKIP", "None");

```

```

6660     } else {
6661         float tflops = bench_hgemm_tflops(M, N, K, time_ms);
6662         printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6663             max_err < 1.0f ? "PASS" : "FAIL", tflops);
6664     }
6665 }
6666 }
6667 cudaEventDestroy(start);
6668 cudaEventDestroy(stop);
6669 free(h_a);
6670 free(h_b);
6671 free(h_b_t);
6672 free(h_c);
6673 free(h_c_ref);
6674 cudaFree(d_a);
6675 cudaFree(d_b);
6676 cudaFree(d_b_t);
6677 cudaFree(d_c);
6678 cudaFree(tma_a);
6679 cudaFree(tma_b);
6680 cublasDestroy(handle);
6681 }
6682 #endif
6683
6684 // =====
6685 // Bench: FlashAttention-2Split-Q (template on kHeadDim for dispatch)
6686 // =====
6687 template <int kHeadDim, int kPadQ = 8, int kPadK = 8, int kPadV = 8>
6688 static void bench_fa_launch(int B, int H, int seqlen, int head_dim,
6689     half *h_o_ref, float *ref_o, half *d_q, half *d_k, half *d_v, half *d_o) {
6690     static_assert(kHeadDim == 64 || kHeadDim == 128, "Only D=64 and D=128 are supported");
6691     constexpr int kStagesK = 2;
6692     constexpr bool kSwizzleQ = kPadQ == 0;
6693     constexpr bool kSwizzleK = kPadK == 0;
6694     constexpr bool kSwizzleV = kPadV == 0;
6695     constexpr int kMmaAtomM = 16;
6696     constexpr int kMmaAtomN = 8;
6697     constexpr int kMmaAtomK = 16;
6698     constexpr int kMmaTileSeqLenQ = 8;
6699     constexpr int kMmaTileSeqLenK = 1;
6700     constexpr int kMmaTileSeqLenP = 8;
6701     constexpr int kMmaTileHeadDimV = 1;
6702     constexpr int kValTileSeqLenQ = 1;
6703     constexpr int kValTileSeqLenK = 8;
6704     constexpr int kValTileSeqLenP = 1;
6705     constexpr int kValTileHeadDimV = kHeadDim / (8 * kMmaTileHeadDimV);
6706     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kValTileSeqLenQ;
6707     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kValTileSeqLenK;
6708     constexpr const char *layout_name =
6709         kSwizzleQ && kSwizzleK && kSwizzleV ? "Swizzle" :
6710         kSwizzleQ && kSwizzleK ? "SwizzleQK" :
6711         kSwizzleQ && kSwizzleV ? "SwizzleQV" :
6712         kSwizzleK && kSwizzleV ? "SwizzleKV" :
6713         kSwizzleQ ? "SwizzleQ" :
6714         kSwizzleK ? "SwizzleK" :
6715         kSwizzleV ? "SwizzleV" : "Pad";
6716
6717     // Kernel requires seqlen >= Br (tile size); skip gracefully for short seqlen
6718     if (seqlen < Br) {
6719         char label[64];
6720         snprintf(label, sizeof(label), "FlashAttention-2, D=%d, %s)", kHeadDim,
6721             layout_name);
6722         printf("| %-42s | %-12s | %-4s | %-8s |\n", label,

```

```

6723         "seqlen<Br", "SKIP", "None");
6724     return;
6725 }
6726 size_t smem_bytes =
6727     (Br * (kHeadDim + kPadQ) +
6728      kStagesK * Bc * (kHeadDim + kPadK) +
6729      Bc * (kHeadDim + kPadV)) * sizeof(half);
6730
6731 dim3 block(256);
6732 dim3 grid((seqlen + Br - 1) / Br, B * H);
6733
6734 cudaStream_t timing_stream;
6735 cudaStreamCreate(&timing_stream);
6736 cudaEvent_t start, stop;
6737 cudaEventCreate(&start);
6738 cudaEventCreate(&stop);
6739
6740 using FAKernel = void (*)(half *, half *, half *, half *, int, int);
6741 FAKernel fa_k = flash_attn_mma_stages_split_q<
6742     kHeadDim, kMmaAtomM, kMmaAtomN, kMmaAtomK, kMmaTileSeqLenQ, kMmaTileSeqLenK,
6743     kMmaTileSeqLenP, kMmaTileHeadDimV, kValTileSeqLenQ, kValTileSeqLenK,
6744     kValTileSeqLenP, kValTileHeadDimV, kStagesK, kPadQ, kPadK, kPadV>;
6745 bool smem_ok = check_smem_feasible((const void *)fa_k, smem_bytes);
6746 if (smem_ok) {
6747     cudaFuncSetAttribute(fa_k, cudaFuncAttributeMaxDynamicSharedMemorySize, smem_bytes);
6748 }
6749
6750 // Warmup
6751 if (smem_ok) {
6752     for (int w = 0; w < g_warmup; w++)
6753         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
6754     check(cudaStreamSynchronize(timing_stream), "fa warmup sync");
6755 }
6756
6757 // Timed repeat
6758 cudaEventRecord(start, timing_stream);
6759 if (smem_ok) {
6760     for (int r = 0; r < g_repeat; r++)
6761         fa_k<<<grid, block, smem_bytes, timing_stream>>>(d_q, d_k, d_v, d_o, seqlen, H);
6762 }
6763 cudaEventRecord(stop, timing_stream);
6764 cudaEventSynchronize(stop);
6765
6766 float time_ms = 0;
6767 cudaEventElapsedTime(&time_ms, start, stop);
6768 time_ms /= g_repeat;
6769
6770 size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6771 half *h_o = (half *)malloc(sz);
6772 check(cudaMemcpy(h_o, d_o, sz, cudaMemcpyDeviceToHost), "bench fa D2H");
6773
6774 int count = B * H * seqlen * head_dim;
6775 char label[64];
6776 snprintf(label, sizeof(label), "FlashAttention-2, (S=2, D=%d, %s)", kHeadDim,
6777     layout_name);
6778 float max_err = 0.0f;
6779 bool checked = h_o_ref || ref_o;
6780 if (smem_ok && checked) {
6781     for (int i = 0; i < count; i++) {
6782         float ref_val = h_o_ref ? __half2float(h_o_ref[i]) : ref_o[i];
6783         float err = fabsf(__half2float(h_o[i]) - ref_val);
6784         if (err > max_err) max_err = err;
6785     }

```

```

6786 }
6787 if (smem_ok && checked) {
6788     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
6789     printf("| %-42s | %.6e | %-4s | %-8.1f |\n", label, max_err,
6790           max_err < 5e-1f ? "PASS" : "FAIL", tflops);
6791 } else if (smem_ok) {
6792     float tflops = bench_fa_tflops(B, H, seqlen, head_dim, time_ms);
6793     printf("| %-42s | %-12s | %-4s | %-8.1f |\n", label, "unchecked", "SKIP", tflops);
6794 } else {
6795     printf("| %-42s | %-12s | %-4s | %-8s |\n", label, "SMEM too large", "SKIP", "None");
6796 }
6797
6798 cudaEventDestroy(start);
6799 cudaEventDestroy(stop);
6800 cudaStreamDestroy(timing_stream);
6801 free(h_o);
6802 }
6803
6804 static void bench_flash_attn(int B, int H, int N, int D) {
6805     int seqlen = N, head_dim = D;
6806
6807     if (head_dim != 64 && head_dim != 128) {
6808         printf("| %-42s | %-12s | %-4s | %-8s |\n", "FlashAttention-2", "unsupported D", "SKIP", "None");
6809         return;
6810     }
6811
6812     size_t sz = (size_t)B * H * seqlen * head_dim * sizeof(half);
6813
6814     srand(42);
6815     half *h_q = (half *)malloc(sz);
6816     half *h_k = (half *)malloc(sz);
6817     half *h_v = (half *)malloc(sz);
6818     for (int i = 0; i < B * H * seqlen * head_dim; i++) {
6819         h_q[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6820         h_k[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6821         h_v[i] = __float2half(((float)rand() / RAND_MAX) * 2.0f - 1.0f);
6822     }
6823
6824     // Device allocations — shared by kernel and reference
6825     half *d_q, *d_k, *d_v, *d_o;
6826     check(cudaMalloc(&d_q, sz), "bench fa alloc Q");
6827     check(cudaMalloc(&d_k, sz), "bench fa alloc K");
6828     check(cudaMalloc(&d_v, sz), "bench fa alloc V");
6829     check(cudaMalloc(&d_o, sz), "bench fa alloc O");
6830     check(cudaMemcpy(d_q, h_q, sz, cudaMemcpyHostToDevice), "bench fa H2D Q");
6831     check(cudaMemcpy(d_k, h_k, sz, cudaMemcpyHostToDevice), "bench fa H2D K");
6832     check(cudaMemcpy(d_v, h_v, sz, cudaMemcpyHostToDevice), "bench fa H2D V");
6833
6834     // Reference output: either cuDNN (half) or CPU (float)
6835     half *h_o_ref = nullptr;
6836     float *ref_o = nullptr;
6837     int ref_count = B * H * seqlen * head_dim;
6838
6839     #if defined(NOTES_V2_ENABLE_CUDNN)
6840     if (!g_fa_skip_check) {
6841         // Try cuDNN SDPA first; fall back to CPU if unsupported on this SM
6842         bool cudnn_ok = false;
6843         half *d_o_ref;
6844         check(cudaMalloc(&d_o_ref, sz), "bench fa alloc O_ref");
6845         {
6846             cudnnHandle_t cudnn_handle;
6847             cudnnCreate(&cudnn_handle);
6848             auto graph = std::make_shared<fe::graph::Graph>();

```

```

6849 graph->set_io_data_type(fe::DataType_t::HALF)
6850     .set_intermediate_data_type(fe::DataType_t::FLOAT)
6851     .set_compute_data_type(fe::DataType_t::FLOAT);
6852
6853 auto Q = graph->tensor(fe::graph::Tensor_attributes()
6854     .set_uid(1).set_dim({B, H, seqlen, head_dim})
6855     .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6856 auto K = graph->tensor(fe::graph::Tensor_attributes()
6857     .set_uid(2).set_dim({B, H, seqlen, head_dim})
6858     .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6859 auto V = graph->tensor(fe::graph::Tensor_attributes()
6860     .set_uid(3).set_dim({B, H, seqlen, head_dim})
6861     .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1}));
6862
6863 auto [O_sdpa, Stats] = graph->sdpa(Q, K, V,
6864     fe::graph::SDPA_attributes()
6865     .set_name("sdpa_ref")
6866     .set_attn_scale(1.0f / sqrtf((float)head_dim)));
6867
6868 O_sdpa->set_output(true).set_uid(4)
6869     .set_dim({B, H, seqlen, head_dim})
6870     .set_stride({H * seqlen * head_dim, seqlen * head_dim, head_dim, 1});
6871
6872 // Try A first, then fallback (matching Python example: [cudnn.heur_mode.A, cudnn.heur_mode.FALLBACK])
6873 auto build_status = graph->build(cudnn_handle, {fe::HeurMode_t::A, fe::HeurMode_t::FALLBACK});
6874 if (build_status.is_good()) {
6875     std::unordered_map<fe::graph::Tensor_attributes::uid_t, void*> vp = {
6876         {1, d_q}, {2, d_k}, {3, d_v}, {4, d_o_ref}};
6877     int64_t ws_size = 0;
6878     if (graph->get_workspace_size(ws_size).is_good()) {
6879         int8_t *d_ws = nullptr;
6880         if (ws_size > 0) check(cudaMalloc(&d_ws, ws_size), "bench fa workspace");
6881         if (graph->execute(cudnn_handle, vp, d_ws).is_good()) {
6882             check(cudaDeviceSynchronize(), "bench fa cudnn sync");
6883             cudnn_ok = true;
6884         }
6885         if (d_ws) cudaFree(d_ws);
6886     }
6887 }
6888 cudnnDestroy(cudnn_handle);
6889 }
6890
6891 if (cudnn_ok) {
6892     h_o_ref = (half *)malloc(sz);
6893     check(cudaMemcpy(h_o_ref, d_o_ref, sz, cudaMemcpyDeviceToHost), "bench fa D2H ref");
6894 } else {
6895     fprintf(stderr, "cudnn SDPA unsupported on this SM, using CPU ref\n");
6896 }
6897 cudaFree(d_o_ref);
6898 }
6899 #endif
6900
6901 // CPU reference fallback
6902 if (!g_fa_skip_check && !h_o_ref) {
6903     float *ref_q = (float *)malloc(sz * 4 / sizeof(half));
6904     float *ref_k = (float *)malloc(sz * 4 / sizeof(half));
6905     float *ref_v = (float *)malloc(sz * 4 / sizeof(half));
6906     ref_o = (float *)malloc(sz * 4 / sizeof(half));
6907     for (int i = 0; i < ref_count; i++) {
6908         ref_q[i] = __half2float(h_q[i]);
6909         ref_k[i] = __half2float(h_k[i]);
6910         ref_v[i] = __half2float(h_v[i]);
6911     }

```



```

6912 float scale = 1.0f / sqrtf((float)head_dim);
6913 for (int bi = 0; bi < B * H; bi++) {
6914     for (int qi = 0; qi < seqlen; qi++) {
6915         float smax = -INFINITY;
6916         float *S = (float *)malloc((size_t)seqlen * sizeof(float));
6917         for (int kj = 0; kj < seqlen; kj++) {
6918             float s = 0.0f;
6919             for (int d = 0; d < head_dim; d++)
6920                 s += ref_q[bi * seqlen * head_dim + qi * head_dim + d] *
6921                     ref_k[bi * seqlen * head_dim + kj * head_dim + d];
6922             S[kj] = s * scale;
6923             if (S[kj] > smax) smax = S[kj];
6924         }
6925         double sum_exp = 0.0;
6926         for (int kj = 0; kj < seqlen; kj++)
6927             sum_exp += (double)expf(S[kj] - smax);
6928         float inv_sum = 1.0f / (float)sum_exp;
6929         for (int d = 0; d < head_dim; d++) {
6930             double o_acc = 0.0;
6931             for (int kj = 0; kj < seqlen; kj++)
6932                 o_acc += (double)(expf(S[kj] - smax) * inv_sum) *
6933                     ref_v[bi * seqlen * head_dim + kj * head_dim + d];
6934             ref_o[bi * seqlen * head_dim + qi * head_dim + d] = (float)o_acc;
6935         }
6936         free(S);
6937     }
6938 }
6939 free(ref_q);
6940 free(ref_k);
6941 free(ref_v);
6942 }
6943
6944 if (head_dim == 64) {
6945     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
6946         cudaDeviceSynchronize();
6947         bench_fa_launch<64, 8, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6948     }
6949     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
6950         cudaDeviceSynchronize();
6951         bench_fa_launch<64, 0, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6952     }
6953     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
6954         cudaDeviceSynchronize();
6955         bench_fa_launch<64, 8, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6956     }
6957     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
6958         cudaDeviceSynchronize();
6959         bench_fa_launch<64, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6960     }
6961     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
6962         cudaDeviceSynchronize();
6963         bench_fa_launch<64, 0, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6964     }
6965     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
6966         cudaDeviceSynchronize();
6967         bench_fa_launch<64, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6968     }
6969     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
6970         cudaDeviceSynchronize();
6971         bench_fa_launch<64, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6972     }
6973     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {

```

```

6975     cudaDeviceSynchronize();
6976     bench_fa_launch<64, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6977 }
6978 } else {
6979     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Pad) {
6980         cudaDeviceSynchronize();
6981         bench_fa_launch<128, 8, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6982     }
6983     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQ) {
6984         cudaDeviceSynchronize();
6985         bench_fa_launch<128, 0, 8, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6986     }
6987     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleK) {
6988         cudaDeviceSynchronize();
6989         bench_fa_launch<128, 8, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6990     }
6991     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleV) {
6992         cudaDeviceSynchronize();
6993         bench_fa_launch<128, 8, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6994     }
6995     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQK) {
6996         cudaDeviceSynchronize();
6997         bench_fa_launch<128, 0, 0, 8>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
6998     }
6999     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleQV) {
7000         cudaDeviceSynchronize();
7001         bench_fa_launch<128, 0, 8, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
7002     }
7003     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::SwizzleKV) {
7004         cudaDeviceSynchronize();
7005         bench_fa_launch<128, 8, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
7006     }
7007     if (g_fa_layout == FALayout::All || g_fa_layout == FALayout::Swizzle) {
7008         cudaDeviceSynchronize();
7009         bench_fa_launch<128, 0, 0, 0>(B, H, seqlen, head_dim, h_o_ref, ref_o, d_q, d_k, d_v, d_o);
7010     }
7011 }
7012 cudaDeviceSynchronize();
7013
7014 free(h_q);
7015 free(h_k);
7016 free(h_v);
7017 free(h_o_ref);
7018 free(ref_o);
7019 cudaFree(d_q);
7020 cudaFree(d_k);
7021 cudaFree(d_v);
7022 cudaFree(d_o);
7023 }
7024
7025
7026 int main(int argc, char *argv[]) {
7027     #if defined(NOTES_V2_ENABLE_WGMMMA) || defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7028         cuInit(0);
7029     #endif
7030
7031     for (int i = 1; i < argc; i++) {
7032         if (strcmp(argv[i], "--bench") == 0) {
7033             g_bench_mode = true;
7034         } else if (strcmp(argv[i], "--bench-fa") == 0) {
7035             g_bench_fa = true;
7036         } else if (strcmp(argv[i], "--bench-fa-all") == 0) {
7037             g_bench_fa = true;

```

```

7038     g_fa_layout = FALayout::All;
7039 } else if (strcmp(argv[i], "--only-fa") == 0) {
7040     g_bench_mode = true;
7041     g_bench_only_fa = true;
7042     g_bench_fa = true;
7043 } else if (strcmp(argv[i], "--mnk") == 0 && i + 1 < argc) {
7044     sscanf(argv[++i], "%d,%d,%d", &g_bench_M, &g_bench_N, &g_bench_K);
7045 } else if (strcmp(argv[i], "--bhnd") == 0 && i + 1 < argc) {
7046     sscanf(argv[++i], "%d,%d,%d,%d", &g_bench_B, &g_bench_H, &g_bench_Nfa, &g_bench_D);
7047 } else if (strcmp(argv[i], "--fa-layout") == 0 && i + 1 < argc) {
7048     const char *layout = argv[++i];
7049     if (strcmp(layout, "all") == 0)
7050         g_fa_layout = FALayout::All;
7051     else if (strcmp(layout, "pad") == 0)
7052         g_fa_layout = FALayout::Pad;
7053     else if (strcmp(layout, "swizzle-q") == 0)
7054         g_fa_layout = FALayout::SwizzleQ;
7055     else if (strcmp(layout, "swizzle-k") == 0)
7056         g_fa_layout = FALayout::SwizzleK;
7057     else if (strcmp(layout, "swizzle-v") == 0)
7058         g_fa_layout = FALayout::SwizzleV;
7059     else if (strcmp(layout, "swizzle-qk") == 0)
7060         g_fa_layout = FALayout::SwizzleQK;
7061     else if (strcmp(layout, "swizzle-qv") == 0)
7062         g_fa_layout = FALayout::SwizzleQV;
7063     else if (strcmp(layout, "swizzle-kv") == 0)
7064         g_fa_layout = FALayout::SwizzleKV;
7065     else if (strcmp(layout, "swizzle") == 0)
7066         g_fa_layout = FALayout::Swizzle;
7067     else {
7068         fprintf(stderr, "Unsupported FA layout: %s\n", layout);
7069         return EXIT_FAILURE;
7070     }
7071 } else if (strcmp(argv[i], "--fa-skip-check") == 0) {
7072     g_fa_skip_check = true;
7073 } else if (strcmp(argv[i], "--warmup") == 0 && i + 1 < argc) {
7074     g_warmup = atoi(argv[++i]);
7075 } else if (strcmp(argv[i], "--repeat") == 0 && i + 1 < argc) {
7076     g_repeat = atoi(argv[++i]);
7077 }
7078 }
7079
7080 if (g_bench_mode) {
7081     printf("=== notes-v2.cu bench mode ===\n");
7082     printf("HGEMM: M=%d N=%d K=%d   FA: B=%d H=%d N=%d D=%d\n",
7083           g_bench_M, g_bench_N, g_bench_K,
7084           g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
7085     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7086     printf("|-----|-----|-----|-----|\n");
7087
7088     if (!g_bench_only_fa) {
7089         test_block_reduce(g_bench_N);
7090         test_dot(g_bench_N);
7091         test_relu(1024);
7092         test_elementwise(1024);
7093         test_histogram(1024);
7094         test_merge_attn_states(512, 16, 128);
7095         test_softmax(256);
7096         test_rms_norm(8, 128);
7097         test_layer_norm(8, 128);
7098         test_rope(8, 128);
7099         test_mat_transpose(256, 256);
7100         test_mat_transpose_padded(256, 256);

```

```

7101     test_sgemv(256, 128);
7102     test_sgemm(g_bench_M, g_bench_N, g_bench_K);
7103     bench_hgemm_mma(g_bench_M, g_bench_N, g_bench_K);
7104     bench_hgemm_swizzle(g_bench_M, g_bench_N, g_bench_K);
7105     #if defined(NOTES_V2_ENABLE_CUTE)
7106         bench_hgemm_cute(g_bench_M, g_bench_N, g_bench_K);
7107     #endif
7108     #if defined(NOTES_V2_ENABLE_WGMMMA)
7109         bench_hgemm_wgmma(g_bench_M, g_bench_N, g_bench_K);
7110     #endif
7111     #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7112         bench_hgemm_tma_mma_ws(g_bench_M, g_bench_N, g_bench_K);
7113     #endif
7114     }
7115     if (g_bench_fa)
7116         bench_flash_attn(g_bench_B, g_bench_H, g_bench_Nfa, g_bench_D);
7117
7118     printf("=== Bench done ===\n");
7119     return 0;
7120 }
7121
7122 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7123     if (argc >= 2 && strcmp(argv[1], "--tma-mma-ws") == 0) {
7124         int M = 128, N = 128, K = 64;
7125         if (argc > 4) {
7126             M = atoi(argv[2]);
7127             N = atoi(argv[3]);
7128             K = atoi(argv[4]);
7129         }
7130         printf("=== SM120 TMA MMA WS validation ===\n");
7131         printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7132         printf("|-----|-----|-----|-----|\n");
7133         test_hgemm_tma_mma_ws(M, N, K);
7134         return 0;
7135     }
7136 #endif
7137     int M = 1024, N = 1024, K = 1024;
7138     if (argc > 3) { M = atoi(argv[1]); N = atoi(argv[2]); K = atoi(argv[3]); }
7139
7140     printf("=== notes-v2.cu verification harness ===\n");
7141     printf("| %-42s | %-12s | %-4s | %-8s |\n", "Kernel", "Max Err", "Pass", "TFLOPS");
7142     printf("|-----|-----|-----|-----|\n");
7143
7144     test_block_reduce(N);
7145     test_dot(N);
7146     test_relu(1024);
7147     test_elementwise(1024);
7148     test_histogram(1024);
7149     test_merge_attn_states(512, 16, 128);
7150     test_softmax(256);
7151     test_rms_norm(8, 128);
7152     test_layer_norm(8, 128);
7153     test_rope(8, 128);
7154     test_mat_transpose(256, 256);
7155     test_mat_transpose_padded(256, 256);
7156     test_sgemv(256, 128);
7157     test_sgemm(M, N, K);
7158     test_hgemm_mma(M, N, K);
7159     test_hgemm_swizzle(M, N, K);
7160     #if defined(NOTES_V2_ENABLE_CUTE)
7161         test_hgemm_cute(M, N, K);
7162     #endif
7163     #if defined(NOTES_V2_ENABLE_WGMMMA)

```

```

7164     test_hgemm_wgmma(M, N, K);
7165 #endif
7166 #if defined(NOTES_V2_ENABLE_TMA_MMA_WS)
7167     test_hgemm_tma_mma_ws(M, N, K);
7168 #endif
7169     test_flash_attn(1024, 64);
7170
7171     printf("=== All tests done ===\n");
7172     return 0;
7173 }
7174
7175 // =====
7176 // Quick build & run reference
7177 // =====
7178 // # sm_86 + CuTe (Ampere, RTX 30/40 系列, 无 WGMMMA) :
7179 // nvcc -std=c++20 -O2 -arch=sm_86 -DNOTES_V2_ENABLE_CUTE \
7180 // -I ../../third-party/cutlass/include \
7181 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm86.bin
7182 //
7183 // # sm_89 单独编译 + 运行:
7184 // nvcc -std=c++20 -O2 -arch=sm_89 -lcublas -lcuda notes-v2.cu -o notes_v2_sm89.bin
7185 //
7186 // # sm_89 + CuTe (需要 CUTLASS include 路径, 编译 CUDA_HOME 指向的 CUTLASS 或
7187 // 项目内置 third-party/cutlass) :
7188 // nvcc -std=c++20 -O2 -arch=sm_89 -DNOTES_V2_ENABLE_CUTE \
7189 // -I ../../third-party/cutlass/include \
7190 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm89.bin
7191 //
7192 // # sm_90a 单独编译 + 运行 (需要 Hopper GPU, H100/H200 均可) :
7193 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
7194 // -DNOTES_V2_ENABLE_WGMMMA -lcublas -lcuda notes-v2.cu -o notes_v2_sm90.bin
7195 //
7196 // # sm_90a + CuTe + WGMMMA:
7197 // nvcc -std=c++20 -O2 -gencode arch=compute_90a,code=sm_90a \
7198 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_WGMMMA \
7199 // -I ../../third-party/cutlass/include \
7200 // -lcublas -lcuda notes-v2.cu -o notes_v2_cute_sm90.bin
7201 //
7202 // # sm_120a TMA + warp-specialized mma.sync (CUDA Toolkit >= 13.2;
7203 // RTX PRO 5000 / RTX 5090):
7204 // nvcc -std=c++20 -O2 -arch=sm_120a -DNOTES_V2_ENABLE_TMA_MMA_WS \
7205 // -lcublas -lcuda notes-v2.cu -o notes_v2_tma_mma_ws_sm120.bin
7206 // ./notes_v2_tma_mma_ws_sm120.bin --tma-mma-ws 1024 1024 1024
7207 //
7208 // # sm_120a + CUTE + TMA_MMA_WS + cuDNN SDPA (CUDA Toolkit >= 13.2;
7209 // requires cudnn-frontend submodule: git submodule update --init):
7210 // nvcc -std=c++20 -O2 -arch=sm_120a \
7211 // -DNOTES_V2_ENABLE_CUTE -DNOTES_V2_ENABLE_TMA_MMA_WS -DNOTES_V2_ENABLE_CUDNN \
7212 // -I ../../third-party/cutlass/include -I ../../third-party/cudnn-frontend/include \
7213 // -L/usr/local/cuda-13.2/targets/x86_64-linux/lib/stubs \
7214 // -lcublas -lcudnn -lnvrtc -lcuda notes-v2.cu -o notes_v2_cute_ws_sm120a.bin
7215 // # Default FA bench (kPadQ=kPadK=kPadV=8 only):
7216 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa --bhnd 1,48,4096,64
7217 // # All FA layout variants:
7218 // ./notes_v2_cute_ws_sm120a.bin --bench --bench-fa-all --bhnd 1,48,4096,64

```